

Foundry Learning Loop Lab · Complete Print Edition

Create your own lab environment and follow ten steps through Foundry Evaluation, Agent Optimizer, comparison, and authorized cleanup or retention. The setup reference, lab checklist, troubleshooting, and data guide support the same self-service path. Cross-document links stay within this book; file references use package paths.

As of 2026-10-05

Included documents

1. [Hands-on guide](#)

2. [Appendix 1 · Lab checklist](#)

3. [Appendix 2 · Environment setup reference](#)

4. [Appendix 3 · Troubleshooting](#)

5. [Appendix 4 · Data guide](#)

Hands-on guide

Evaluate and improve a customer-support Agent · v1

CONTOSO ATLAS CLOUD · ONE LEARNING PATH

Score a support AI on 12 questions, improve its instructions, then compare its answers to the same questions. No Azure or Foundry experience is required. You do need to install tools on your computer and have an Azure environment with access and cost authorization.

Azure is Microsoft's cloud platform. **Microsoft Foundry** is the platform for building and evaluating AI models and Agents on Azure. An **Agent** combines a model with instructions and tools. Here, a fictional **Contoso Atlas Cloud support Agent** searches policy documents before answering.

WHAT YOU BUILD	A support Agent that answers using policy evidence.
WHY IT MATTERS	Judge quality from 12 answers and scoring reasons, not one convincing answer.
HOW YOU PROCEED	Keep questions, models, and scoring criteria fixed; change only instructions.
WHAT YOU KEEP	A before/after comparison, your decision, and resource-cleanup records.

Every participant completes the lab from beginning to end. Use your own account and computer to prepare the environment, Agent, dataset, and evaluation, then improve and clean up. No pre-provisioned environment is assumed. Open reference documents only when you need a setting explained.

Start at 01; when resuming, start with your own records. Use the same lab's `config.json`, manifest, and receipts to verify completed work and avoid duplicate creation or submission. Never copy another person's configuration, sign-in session, or ownership records.

Time and cost: Allow roughly half a day when starting from scratch, plus permission/quota approval waits and service execution time. Model calls, Search, and logs can incur charges. **Closing your browser does not end those charges; finish the cleanup in 10.** If you stop partway through after creating resources, still perform 10's record preservation and cleanup.

01 [Check your account, computer, and access](#)

02 [Create the Foundry environment](#)

03 [Connect policies and create the Agent](#)

04 [Register the dataset](#)

05 [Select evaluation criteria](#)

06 [Run Foundry Evaluation](#)

07 [Read scores and reasons](#)

08 [Improve instructions with Agent Optimizer](#)

09 [Reevaluate and compare v1/v2](#)

10 [Save results and delete resources](#)

Here, **learning means reading evaluation results and improving instructions**, not retraining model weights. Use only the supplied synthetic policies and questions. Do not upload private customer records or put the test answers into the instructions.

Where you work	What you do there
Azure Portal (https://portal.azure.com)	Verify identity, subscription, permissions, actual resources, and deletion scope.
Your computer's terminal	The window where you prepare files and run supplied commands. Step 01 explains how to open it.
Microsoft Foundry (https://ai.azure.com)	Inspect the project, Agent, and policy tool; run evaluations and Optimizer; read comparisons.

How to read a step: Use its **Action order** links, perform each action, then check **completion criteria**. Enter only **Run in your terminal** blocks on your computer; perform portal actions in the named screen. Keep Azure Portal and Foundry in separate tabs. **Use your own account/project values, not the names in the illustrations.**

Expand implementation only when needed. Actual implementation · read only panels show the command's original functions/settings, not programs to run separately. You can complete the lab with these panels closed. Use **Where to act** links beside each **code ↔ portal** table to find the command or screen; read source when you need the internal behavior. The print edition expands the complete source.

Do not recreate code-created objects in the portal. Inspect their results there. For evaluation and Optimizer stages that submit through the portal, submit once in that UI. Do not substitute invented SDK code for the portal's internal implementation.

Steps 01–03 prepare the environment, 04–09 evaluate and improve, and 10 cleans up. **Steps 04–06 continue in the same evaluation wizard.** Do not create a separate evaluation for each step. Reading marks are personal bookmarks, not evidence of completed operations.

V1 is the original Agent; v2 is the candidate with revised instructions. These are complete Agent versions. Keep their models, tools, and output format fixed. Retaining v1 when no improvement is found is a valid lab outcome; creating v2 is not production approval.

01. Check your account, computer, and access

- What:** Confirm the Azure subscription, access, and local execution environment.
- Why:** Different browser and CLI identities can cause access failures or create resources in the wrong environment.
- How · where:** Check subscription/access in Azure Portal, then compare tools, login, and language settings in the terminal. No Azure resources are created yet.

Action order: [Azure Portal · account/access](#) → [Terminal · tools](#) → [Terminal · sign-in](#) → [Terminal · language](#).

Six terms to recognize before opening the portal. You do not need to memorize them; distinguish their roles.

Tenant / Directory

The boundary that manages an organization's users and access. One account can access several organizations, so check the selected directory too.

Resource group

A group of Azure resources managed together. A dedicated lab group makes the eventual deletion scope easier to identify.

Model

The AI that generates answers. Deploying makes a model available to call; the deployment name identifies that callable deployment.

Subscription

The unit that groups Azure usage and billing. Confirm which subscription will pay for this lab.

Project

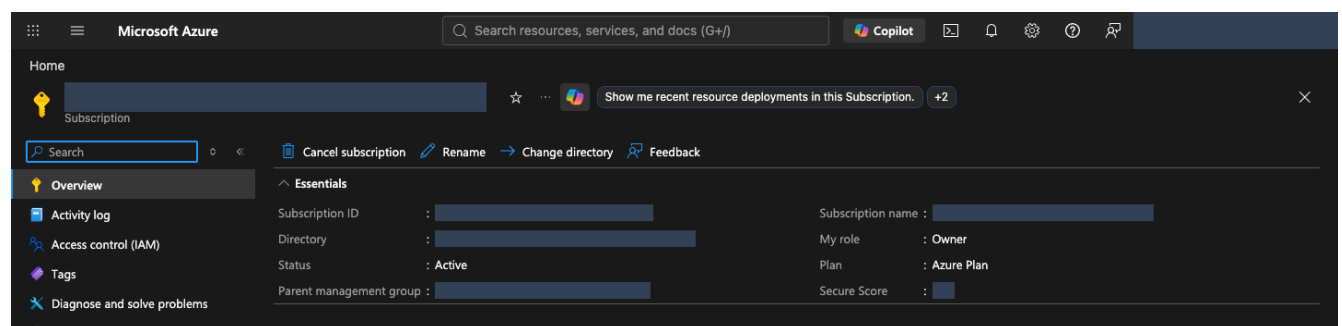
The Foundry workspace for your Agents, data, and evaluations. Stay in the same project throughout the lab.

Role / Scope

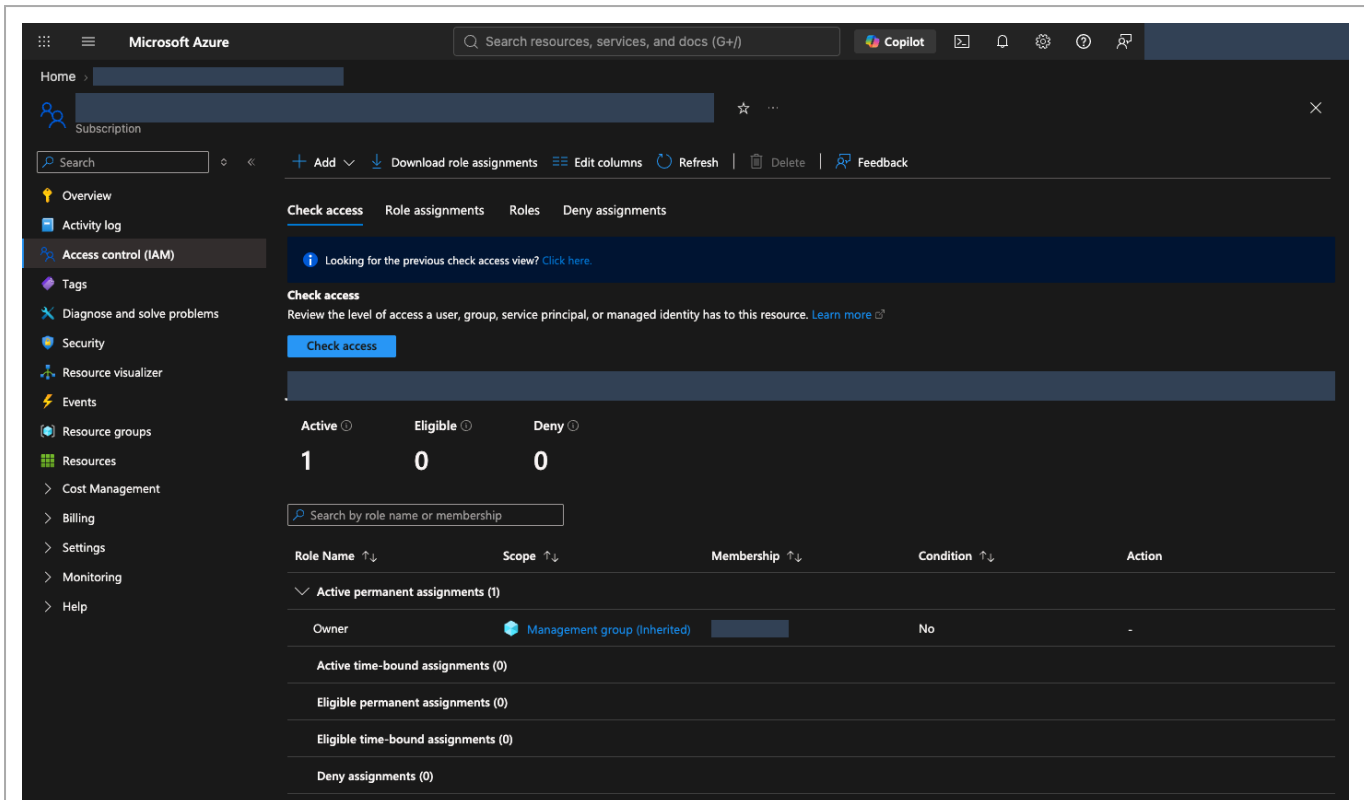
A role defines allowed actions; its scope defines where they apply. Signing in does not automatically grant creation, evaluation, and deletion access.

Confirm your Azure account and subscription

1. Sign in to the [Azure portal](https://portal.azure.com) (<https://portal.azure.com>) with **your own lab account**. If you have no account/subscription, follow the [Azure account instructions](https://azure.microsoft.com/pricing/purchase-options/azure-account) (<https://azure.microsoft.com/pricing/purchase-options/azure-account>). For an organizational subscription, first obtain access and spending authorization for your own lab scope. A free or trial subscription does not guarantee the required model quota.
2. Search for **Subscriptions** in the top search box and open the intended subscription. Confirm portal status **Active** and record its **Subscription ID** and **Directory/Tenant ID**. The CLI describes the same usable subscription as **Enabled**. If it is missing, check directory and subscription filters under your account.
3. Open the permissions menu **Access control (IAM) → Check access** and inspect your role and scope. Older UI versions may label this **View my access**. You create resources and resource-scoped role assignments in this lab, so **both provisioning and role-assignment permissions are required**. Contributor alone cannot assign roles. See [permissions by task and provider registration](#).
4. Set a budget, end time, and deletion or retention scope for your dedicated group. If access or authorization is missing, obtain an approved execution scope through your organization's procedures before continuing. Do not delegate the remaining exercises to another person, grant new subscription-wide Owner, or disable security controls as a shortcut.



Check the subscription and status. Find Subscription ID, Directory, Status, and My role under Essentials. Portal **Active** and CLI **Enabled** describe the same usable state. View full-size image (file: [web/assets/portal/shared/21-subscription-overview.png](#))



Read the role and its scope together. Open Access control (IAM) → Check access and inspect your active assignments. Confirm where each role applies; do not add broader access than the task requires. View full-size image (file: web/assets/porta1/shared/22-check-access.png)

Install tools and download the lab

Install [Python](https://www.python.org/downloads/) (<https://www.python.org/downloads/>) **3.11–3.14**, [Git](https://git-scm.com/downloads) (<https://git-scm.com/downloads>), and the [Azure CLI](https://learn.microsoft.com/cli/azure/install-azure-cli) (<https://learn.microsoft.com/cli/azure/install-azure-cli>). Python runs the supplied programs, Git downloads the lab, and Azure CLI provides the `az` sign-in/query commands. Obtain an approved installation path if your work computer restricts installation.

If the tools are already installed, start with [installation checks](#) and do not reinstall working tools. The Windows/macOS examples below explicitly select **3.13** so an unsupported newer Python is not selected. If you already use another supported version, replace `py -3.13` or `python3.13` with its version-specific command, using the same Python for verification and virtual-environment creation. Python 3.10 or earlier and 3.15 or later are outside this lab's supported range.

First open **your computer's terminal**. Search for **PowerShell** in Windows Start, **Terminal** in macOS Spotlight, or the terminal in your Linux app menu. This guide does not run in Azure Portal's Cloud Shell or the Python prompt showing `>>>`.

Reading commands: Run **one command at a time** and wait for it to finish. A long command can wrap across screen/PDF lines; do not insert a line break within that command.

Copy buttons copy the **whole block**. If it contains several commands, paste into an editor first and run them individually. Replace `YOUR_...` with your value, preserving the surrounding double quotes.

Blocks labeled `Shared terminal` work on all operating systems. For `macOS/Linux` or `PowerShell` blocks, read the preceding description and **run only the blocks for your OS. Stop at an error instead of running the next command**, and use that step's troubleshooting link.

Follow one of [Windows installation](#), [macOS installation](#), or [Linux installation](#), then continue to [installation checks](#).

Windows · install in PowerShell

Run `winget --version` in PowerShell. If it prints a version, use the Windows package manager **WinGet** to run the following commands one at a time. Personally review installation agreements and elevation prompts within your organization's authorization.

Run in your terminal · Windows PowerShell

```
winget install --exact --id Python.Python.3.13 --source winget
winget install --exact --id Git.Git --source winget
winget install --exact --id Microsoft.AzureCLI --source winget
```

If WinGet is missing or unavailable, use the official installers instead. You do not need to use both methods.

Tool	Download and installer choices
Python	From Windows downloads (https://www.python.org/downloads/windows/), choose the latest Python 3.13 patch release and an installer matching your computer. Use the regular installer, not the embeddable package; select Add python.exe to PATH and include pip and the Python Launcher .
Git	Download the appropriate installer from Git for Windows (https://git-scm.com/install/windows). Keep Git from the command line and also from 3rd-party software on the PATH selection screen so Git works in PowerShell.
Azure CLI	Follow Microsoft Installer (MSI) in the official Windows instructions (https://learn.microsoft.com/cli/azure/install-azure-cli-windows?pivots=msi). Choose the 64-bit MSI for a typical x64 computer.

Continue to [checking all three tools in a new terminal](#). If `py` is not recognized, check the Python installer's Launcher option.

macOS · install with Homebrew

Run `brew --version` in Terminal. If the command is missing, follow the [official Homebrew installation instructions \(https://brew.sh/\)](https://brew.sh/), complete the PATH configuration printed under **Next steps**, then check again in a new terminal. Do not bypass your organization's software or Command Line Tools approval process.

Run in your terminal · macOS/Linux

```
brew update
brew install python@3.13 git azure-cli
```

This uses [Homebrew's versioned Python package \(https://formulae.brew.sh/formula/python@3.13\)](https://formulae.brew.sh/formula/python@3.13) and [Microsoft's macOS Azure CLI instructions \(https://learn.microsoft.com/cli/azure/install-azure-cli-macos\)](https://learn.microsoft.com/cli/azure/install-azure-cli-macos). **Use python3.13 for this path.** An existing `python3` can still point to macOS's system Python or another version; do not overwrite or forcibly relink the system interpreter.

Linux · Ubuntu 24.04 LTS example

The following targets **Ubuntu 24.04 LTS**, which provides Python 3.12 by default. For another distribution, follow its Python instructions, [Git installation guidance \(https://git-scm.com/install/linux\)](https://git-scm.com/install/linux), and [official Azure CLI installation instructions \(https://learn.microsoft.com/cli/azure/install-azure-cli-linux\)](https://learn.microsoft.com/cli/azure/install-azure-cli-linux). If Python is outside the supported range, request a supported execution environment rather than replacing the system interpreter.

Run in your terminal · macOS/Linux

```
sudo apt update
sudo apt install python3 python3-venv python3-pip git curl
curl -fsSL 'https://azurecliprod.blob.core.windows.net/$root/deb_install.sh' -o install-azure-cli.sh
```

The last command **only downloads** [Microsoft's Ubuntu/Debian installation script \(https://learn.microsoft.com/cli/azure/install-azure-cli-linux?pivots=apt\)](https://learn.microsoft.com/cli/azure/install-azure-cli-linux?pivots=apt). Open `install-azure-cli.sh` in a text editor and inspect it before running the following command with the required system-installation permissions and authorization. Without `sudo` access, obtain an approved installation path before continuing.

Run in your terminal · macOS/Linux

```
sudo bash install-azure-cli.sh
```

Verify installation in a new terminal

After installation, close all existing terminal windows and open a new one. PATH is the list of locations used to find commands; an existing window might not receive the newly installed paths. If you use VS Code's terminal, exit and reopen VS Code too.

Windows PowerShell:

Run in your terminal · Windows PowerShell

```
py -3.13 --version
git --version
az version
```

macOS · the Homebrew path above:

Run in your terminal · macOS/Linux

```
python3.13 --version
git --version
az version
```

Linux · the Ubuntu path above:

Run in your terminal · macOS/Linux

```
python3 --version
git --version
az version
```

Check	Expected output and requirement
Lab Python	A version such as Python 3.13.x; x is the actual patch number. The minor version must be 3.11, 3.12, 3.13, or 3.14 .
Git	An installed version such as git version 2.x...
Azure CLI	JSON containing a version such as "azure-cli": "2.x...". This check does not require Azure sign-in.

Python information in `az version` describes Azure CLI's own runtime, not verification of the lab's Python installation. **All three commands must succeed before** downloading the lab or creating its virtual environment. For `command not found`, `not recognized`, or Microsoft Store opening instead of Python, see [installation and PATH troubleshooting](#).

Download the lab files

Run in your terminal · Shared terminal

```
git clone https://github.com/junwoojeong100/foundry-evaluation-labs-v1.git
cd foundry-evaluation-labs-v1
```

If you already downloaded the repository, enter its folder rather than cloning it again. With GitHub **Code** → **Download ZIP**, extract the archive first and open the folder containing `pyproject.toml` and `requirements.lock`. Downloading only an HTML file omits the code, data, and images.

Create a virtual environment with the verified Python

Run subsequent commands from the **lab folder containing** `pyproject.toml` **and** `requirements.lock`. `.venv` is an isolated Python package environment for this lab. Create it with **the same Python you verified above**, running only the one path matching your operating system.

macOS · the Homebrew path above:

Run in your terminal · macOS/Linux

```
python3.13 -m venv .venv
source .venv/bin/activate
```

Linux · the Ubuntu path above:

Run in your terminal · macOS/Linux

```
python3 -m venv .venv
source .venv/bin/activate
```

In Windows PowerShell, run:

Run in your terminal · Windows PowerShell

```
py -3.13 -m venv .venv
.\.venv\Scripts\Activate.ps1
```

If PowerShell policy blocks activation, do not change organizational policy. Replace `python` in subsequent commands with `.\.venv\Scripts\python.exe`. If `.venv` already exists, confirm that it belongs to this lab rather than overwriting another task's environment.

After preparing the virtual environment, run these shared commands. Confirm Python 3.11–3.14 and a pip path inside `.venv` from the first two commands before installing packages.

Run in your terminal · Shared terminal

```
python --version
python -m pip --version
python -m pip install -r requirements.lock
python -m lab --help
python scripts/build_datasets.py --language en --check
```

Confirm that the command list appears and the dataset check succeeds. For `ModuleNotFoundError`, use `python -m pip --version` to check that Python and pip belong to `.venv`. Do not install packages into an unrelated system interpreter.

Pin the CLI identity and lab language

First replace `YOUR_SUBSCRIPTION_ID` below with the subscription ID you recorded. Sign in with **the same account** in the browser opened by `az login`, then return to the terminal.

Run in your terminal · Shared terminal

```
az login
az account list --query "[].{name:name,id:id,state:state}" -o table
az account set --subscription "YOUR_SUBSCRIPTION_ID"
az account show --query "{user:user.name,tenant:tenantId,subscription:id,state:state}" -o json
```

Confirm that the final state **is** `Enabled`, compare `user`, `tenant`, and `subscription` with the portal, and record them for the plan. Browser and CLI sign-ins are separate. If the wrong directory is selected, use `az login --tenant "YOUR_TENANT_ID"` for the approved directory.

Set the language and execution-record folder

Use `lab-en` as this English lab's environment name. If that local plan already exists, resume its original records rather than creating it again. For a separate class, choose a new name such as `lab-en-02` and change every subsequent path and prefix consistently.

On macOS/Linux, set:

Run in your terminal · macOS/Linux

```
export LAB_LANGUAGE=en
export LAB_ARTIFACTS_DIR="$PWD/.lab/lab-en/artifacts"
```

In Windows PowerShell, set:

Run in your terminal · Windows PowerShell

```
$env:LAB_LANGUAGE = "en"
$env:LAB_ARTIFACTS_DIR = Join-Path (Get-Location).Path ".lab/lab-en/artifacts"
```

Enter the lab folder first in each new terminal, then repeat [virtual-environment activation](#) and these two settings. Never change `LAB_LANGUAGE` while reusing another language's run folder.

Resuming the same lab: Recheck your virtual environment, language variables, and identity, then use your original execution records. Inspect provisioning with [02's status and runtime checks](#). Step 01 does not create Azure resources or call the SDK.

Code ↔ portal · verify sign-in and local settings

Where to act: Run the [terminal sign-in commands](#) and [language/record-folder settings](#) above. In this step, compare the command output with the portal yourself.

Executed command/setting	What to check now	Portal actions and verification
az login, az account set, az account show	Record the signed-in account, tenant, subscription, and state: Enabled.	Compare Directory under your Azure Portal account and the IDs/state under Subscriptions → intended subscription → Overview yourself.
LAB_LANGUAGE, LAB_ARTIFACTS_DIR	Set corpus language and the execution-record path in this terminal.	These are local settings, not portal controls. The English lab uses en and .lab/lab-en/artifacts.

Completion criteria: Python 3.11–3.14, Git, and Azure CLI versions are verified; the user, tenant, and subscription match; the virtual environment's Python commands and English dataset check succeed. For blockers, see [environment troubleshooting](#).

02. Create the Foundry environment

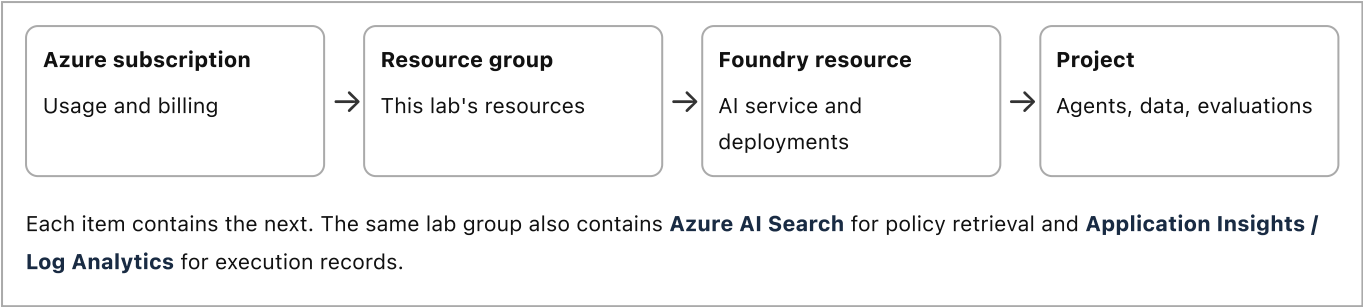
What: Prepare a Foundry project, models, Search, and monitoring in a dedicated resource group.

Why: The group bounds cost and cleanup; the project organizes Agents and evaluations. An empty project alone does not provide policy retrieval.

How · where: Plan, authorize, and provision from your terminal, then compare both portals with the outputs. When resuming, inspect the same plan and actual state.

Action order: [Terminal · plan](#) → [Check required TPM](#) → [Authorization/readiness](#) → [Terminal · provision](#) → [Portal/runtime checks](#).

Start with the plan below on your first attempt. If your same lab plan already exists, do not repeat plan; inspect bootstrap status with its original config.json. Even after provisioning, verify this step's portal and runtime checks before continuing to 03.



An **endpoint** is the address a program uses to connect to a service. Use the project's **Project endpoint** for this lab, not the adjacent Azure OpenAI endpoint.

Create a local provisioning plan

Replace the placeholders with the values recorded in 01. An **ID identifies a resource; it is not its display name**. Do not enter a subscription name in an ID field.

Placeholder	Your value
YOUR_SUBSCRIPTION_ID	subscription from az account show
YOUR_TENANT_ID	tenant from the same output
YOUR_SIGN_IN_NAME	user from the same output

This command writes only a plan and unapproved authorization example under `.lab/lab-en/` on your computer; it does not call Azure.

Run in your terminal · Shared terminal

```
python -m lab bootstrap plan --subscription "YOUR_SUBSCRIPTION_ID" --tenant "YOUR_TENANT_ID" --expected-user "YOUR_SIGN_IN_NAME" --environment lab-en --location northcentralus
```

Confirm `plan_status: CREATED_LOCAL_ONLY`, `mutations_performed: false`, and `config_path`. `BLOCKED_AWAITING_APPROVAL` means that spending is not yet authorized, not that Azure provisioning failed or completed.

Generated file	What to check
<code>.lab/lab-en/config.json</code>	Identity, generated resource names, and exact model/version/capacity selections. Do not edit this hashed plan.
<code>.lab/lab-en/approval.example.json</code>	The initially unapproved cost and change scope.
<code>.lab/lab-en/manifest.json</code>	Provisioning, ownership, and interruption records. Preserve this file.

Generated file	What to check
<code>.lab/lab-en/.env</code>	Not present yet. Successful provisioning creates this runtime configuration.

Use your text editor's **Open file** action to inspect these files. `.lab` stores lab records and may appear as a hidden folder. JSON stores settings as name/value pairs. Do not convert these files to Word documents or change their extensions.

In the same editor, create `.lab/lab-en/notes.md` and use the [lab worksheet](#) to record the identity values from 01 and your plan path. `.md` is a plain-text note file. Add actual deployment names, Agent, data, run IDs, and your decision as you proceed.

Implementation reference: `plan()` selects names/models, calculates hashes, and writes local files only. **It contains no Azure creation request.** Seeing no new resources in the portal after `plan` is expected.

Actual implementation · read only · `lab/bootstrap.py:plan`

Read-only source, not another command to run. The complete named function or object is shown as written, including its original messages; dependencies and imports are in the linked file. Run the lab command only once.

Full source file · 657–726 (file: `lab/bootstrap.py#L657–L726`) · [View on GitHub \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/bootstrap.py#L657-L726\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/bootstrap.py#L657-L726)

```
def plan(
    *, subscription_id: str, tenant_id: str, expected_user: str,
    environment: str | None = None, root: Path | str = Path(".lab"),
    location: str = REGION, retention_days: int = 30, models: list[dict] | None = None,
    resource_group: str | None = None, agent_sku: str | None = None,
) -> dict:
    """Create a private local plan only; never query Azure or overwrite .env/artifacts."""
    instance = uuid4()
    suffix = instance.hex[:8]
    date = _now().strftime("%Y%m%d")
    environment = environment or f"lab-{date}-{suffix}"
    root = Path(root)
    if root.is_symlink() or any(parent.is_symlink() for parent in root.parents):
        raise BootstrapError("Environment root must not traverse symlinks.")
    directory = root.resolve() / environment
    selected = deepcopy(list(DEFAULT_MODELS) if models is None else models)
    if agent_sku is not None and agent_sku not in {"Standard", "GlobalStandard"}:
        raise BootstrapError("Explicit agent SKU must be Standard or GlobalStandard.")
    for model in selected:
        if not isinstance(model, dict) or not isinstance(model.get("roles"), list) or not
model["roles"]:
            raise BootstrapError("Each model requires a nonempty roles array.")
        model.setdefault("sku", "GlobalStandard")
        if agent_sku is not None and "agent" in model["roles"]:
            model["sku"] = agent_sku
        model.setdefault("deployment", f"lab-{model.get('roles', ['model'])[0]}-{suffix}")
    config = {
```

```

        "schema_version": SCHEMA_VERSION, "instance_id": str(instance), "created_at": _stamp(),
        "environment": environment, "environment_dir": str(directory),
        "subscription_id": subscription_id, "tenant_id": tenant_id, "expected_user": expected_user,
        "location": location, "retention_days": retention_days, "models": selected,
        "deployment_name": f"lab-bootstrap-{suffix}",
        "template_sha256": _file_digest(TEMPLATE),
        "names": {
            "resource_group": resource_group or f"rg-foundry-eval-v11-{date}-{suffix}",
            "account": f"ai-fev11-{date}-{suffix}", "project": f"proj-fev11-{suffix}",
            "search": f"srch-fev11-{date}-{suffix}", "workspace": f"log-fev11-{date}-{suffix}",
            "insights": f"appi-fev11-{date}-{suffix}",
        },
    },
}
config["scope_sha256"] = _scope(config)
_validate(config)
try:
    root.mkdir(parents=True, exist_ok=True, mode=0o700)
    directory.mkdir(mode=0o700)
except FileExistsError as exc:
    raise BootstrapError("Environment already exists; use its config to inspect/resume, never
overwrite it.") from exc
_write(directory / ".gitignore", "*\n", text=True)
(directory / "artifacts").mkdir(mode=0o700)
(directory / "evidence").mkdir(mode=0o700)
manifest = {
    "schema_version": SCHEMA_VERSION, "scope_sha256": config["scope_sha256"],
    "phase": "planned", "created_at": _stamp(), "resources": _resources(config),
    "attempts": [], "operator_principal_id": None, "env_sha256": None,
}
_write(directory / "config.json", config)
_write(directory / "template.json", TEMPLATE.read_text(encoding="utf-8"), text=True)
_write(directory / "approval.example.json", approval_template(config))
_write(directory / "group-authorization.example.json", group_authorization_template(config))
_write(directory / "manifest.json", manifest)
_write(directory / "cost-ledger.json", cost_ledger(manifest))
return {
    "status": AWAITING_APPROVAL, "plan_status": "CREATED_LOCAL_ONLY",
    "approval_status": "MISSING", "live_status": "NOT_RUN", "mutations_performed": False,
    "scope_sha256": config["scope_sha256"],
    "config_path": str(directory / "config.json"),
    "approval_example_path": str(directory / "approval.example.json"),
    "env_path": str(directory / ".env"), "artifacts_dir": str(directory / "artifacts"),
    "models": selected,
    "warning": "No Azure calls made. Candidate versions/SKUs/quota require fresh preflight and
explicit approval.",
}

```

The model roles are **answer generation (Agent)**, **scoring (Judge)**, **instruction suggestions/search planning (Optimizer/planner)**, and **numerical representations for search (embedding)**. You do not develop a new model in this lab.

This repository's setup region is **North Central US** (northcentralus). Defaults are Agent gpt-6-sol, Judge gpt-6-luna, Optimizer/search planner gpt-5.5, and embedding text-embedding-3-small. Check the exact versions, SKUs (deployment types), and capacity units in the [model table](#) and plan. Do not silently substitute

a different region or model.

Allocate the recommended minimum TPM

TPM (Tokens Per Minute) is a per-deployment token rate limit, not the model's context window or guaranteed processing speed. These starting requirements cover **one environment running one 12-case evaluation or Optimizer job at a time**. They are recommended minimums with headroom for evaluation/retrieval calls, not proven absolute lower bounds or a guarantee against 429 errors.

Role/model	Recommended minimum TPM to start	ARM capacity in a new default plan
Agent · gpt-6-sol	100,000	100
Judge · gpt-6-luna	100,000	100
Optimizer / search planner · gpt-5.5	100,000	100, one deployment shared by both roles
Embedding · text-embedding-3-small	10,000	10

New bootstrap plan output requests these capacities. **ARM units depend on the model/SKU**; do not universally multiply capacity by 1,000. After deployment, confirm actual TPM with runtime preflight below. Unallocated subscription quota does not establish that an individual deployment has enough TPM assigned.

Existing plans and deployments are not automatically increased. Check your actual deployments and authorization with the [TPM setup reference](#). Do not edit config.json, approval hashes, or the manifest to bypass checks.

Shared users or overlapping jobs require additional capacity. Azure rate-limits estimated tokens based on inputs and maximum-output settings, not only billed tokens. **RPM (Requests Per Minute)** and short-window request bursts are separate constraints. See the [official TPM/RPM explanation](https://learn.microsoft.com/azure/foundry/openai/how-to/quota#understanding-rate-limits) (<https://learn.microsoft.com/azure/foundry/openai/how-to/quota#understanding-rate-limits>).

Check readiness and obtain cost authorization

Run in your terminal · Shared terminal

```
python -m lab bootstrap preflight --config .lab/lab-en/config.json
```

Preflight checks readiness before execution. Look for readiness_status: READY. Quota is your subscription's allowed usage; capacity is actual availability in a region. If blocked, read reason and follow [provisioning troubleshooting](#). Without an approval file, the overall status can still await approval even when readiness is READY. Do not keep creating new environments or switching regions.

Read the `cognitiveservices model list`, `usage list`, and ARM `modelCapacities` requests. This checks availability; it does not invoke or deploy the models.

Actual implementation · read only · `lab/bootstrap.py:_preflight`

Read-only source, not another command to run. The complete named function or object is shown as written, including its original messages; dependencies and imports are in the linked file. Run the lab command only once.

Full source file · 1307–1339 (file: `lab/bootstrap.py#L1307–L1339`) · [View on GitHub](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/bootstrap.py#L1307-L1339)
(<https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/bootstrap.py#L1307-L1339>)

```
def _preflight(config: dict, manifest: dict, run: Run) -> tuple[dict, dict, dict]:
    identity = _identity(config, manifest, run)
    prerequisites = _prerequisites(config, run)
    args = ["--location", REGION, "--subscription", config["subscription_id"]]
    catalog = run(["cognitiveservices", "model", "list", *args])
    usage = run(["cognitiveservices", "usage", "list", *args])
    if not isinstance(catalog, list) or not isinstance(usage, list):
        raise BootstrapError("Regional model catalog/quota response is invalid.")
    capacities = {}
    for model in config["models"]:
        query = urlencode({
            "api-version": "2024-10-01", "modelFormat": "OpenAI",
            "modelName": model["name"], "modelVersion": model["version"],
        })
        capacities[model["deployment"]] = run([
            "rest", "--method", "GET", "--url",
            f"
{ARM}/subscriptions/{config['subscription_id']}/providers/Microsoft.CognitiveServices/locations/{REGION}/modelCapacities?{query}",
        ])
    observed = _inspect(config, manifest, run)
    checks = _model_checks(config, catalog, usage, capacities, observed)
    report = {
        "status": "READY", "scope_sha256": config["scope_sha256"], "checked_at": _stamp(),
        "region": REGION, "identity_verified": True, **prerequisites, "models": checks,
        "owned_resource_count": len(observed), "mutations_performed": False,
        "not_verified": [
            "No paid inference, training, runtime data-plane access, or RBAC propagation tested.",
            "MCP principal is not inferred from matching subscription/tenant.",
            "Available quota is unallocated capacity, not free pricing or a spend cap.",
            "Fine-tuning region, training quota, deployment availability, and cost require separate
preflight/approval.",
        ],
    }
    raw = {"identity": identity, "catalog": catalog, "usage": usage, "capacities": capacities,
"observed": observed}
    return report, raw, observed
```

After confirming authorization for your spending and changes, open `.lab/lab-en/approval.example.json` in an editor and **Save As** `.lab/lab-en/approval.json`. If you can authorize spending on your own subscription, set and record its limits yourself. For an organizational subscription, follow its actual approval procedure. Preserve `scope_sha256`, `models`, and `retention_days`, and complete **every field in the [approval worksheet](#)**.

Set `approved_by` to **your sign-in name**, matching the plan's `expected_user`. This binds the execution identity; it is not an organizational approver's digital signature. Confirm actual authorization evidence, currently valid timestamps, currency/budget, wait/retention bounds, and resource/RBAC/global-processing consent. Changing only `approved` to `true` is insufficient; the budget value is not an Azure spending cutoff.

Run in your terminal · Shared terminal

```
python -m lab bootstrap preflight --config .lab/lab-en/config.json --approval .lab/lab-en/approval.json
```

Proceed only when both `readiness_status: READY` and `status: READY_FOR_APPROVED_APPLY` are present.

Provision and confirm the resources in the portal

Run in your terminal · Shared terminal

```
python -m lab bootstrap apply --config .lab/lab-en/config.json --approval .lab/lab-en/approval.json
python -m lab bootstrap status --config .lab/lab-en/config.json --approval .lab/lab-en/approval.json
```

The first command creates resources, deployments, connections, and resource-scoped roles. It can take time; do not launch a second `apply` in another terminal. Confirm **APPLIED**, a generated `.lab/lab-en/.env`, and status showing phase: succeeded with the expected resources present. After a timeout, inspect status first and follow the [resume procedure](#).

The `.env` created here supplies connection settings for subsequent commands. It records user, tenant, subscription, and actual project/model-deployment values. It is configuration, not a script: specify it with `--config`, never execute or source it. If it is absent, do not copy `.env.example` and call provisioning complete.

Code ↔ portal · inspect created resources and deployments

Where to act: After authorization, run [bootstrap apply and status above](#), then complete the [portal/runtime checks below](#). Inspect created resources in the portal rather than creating them again.

Code/input	Actual action	Portal actions and verification
<code>config["names"]</code> , ARM PUT location/tags	Check for collisions and create a new dedicated group.	Azure Portal → Resource groups → your group → Overview : inspect name, region, and Tags. Do not create another group.
deployment group create, <code>template.json</code> , <code>_parameters(...)</code>	Deploy the Foundry account/project, Search, monitoring, models, connections, and roles in the actual ARM template (file: <code>infra/bootstrap.json</code>).	Group Deployments → your deployment → Deployment details/Inputs/Outputs : compare inputs and created resources.

Code/input	Actual action	Portal actions and verification
Model/version/SKU/capacity in <code>config["models"]</code>	Create four planned model deployments.	Foundry Home → View deployments → deployment → Details : inspect model, version, type, and Tokens per Minute Rate Limit.
Existing group/deployment/resource reads in <code>bootstrap status</code>	Compare actual state with the same plan/local records; do not submit another ARM deployment.	Refresh the same group's Deployments → existing deployment and actual inventory, not Create.
<code>AZURE_AI_PROJECT_ENDPOINT</code> in <code>.env</code>	Save the actual project connection after successful creation.	Compare Foundry Home → Project endpoint , not Azure OpenAI endpoint.
Role assignments and project/Search managed identities	Assign distinct permissions to user and service identities.	Inspect each Azure resource's Identity and Access control (IAM) → Check access . Do not add the same roles again just to verify them.

Read **group creation** → **ARM validation/deployment** → **status polling** → **inventory verification** → `.env saving` in `apply()`. `bounded(...)` checks authorization/wait bounds before executing the Azure CLI argument lists shown in the source. Local records, resumption, and ownership protections are real parts of the implementation, not omitted details.

Actual implementation · read only · `lab/bootstrap.py:apply`

Read-only source, not another command to run. The complete named function or object is shown as written, including its original messages; dependencies and imports are in the linked file. Run the lab command only once.

Full source file · 1901–2092 (file: `lab/bootstrap.py#L1901–L2092`) · [View on GitHub](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/bootstrap.py#L1901-L2092)
<https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/bootstrap.py#L1901-L2092>

```
def apply(
    config_path: Path | str, approval_path: Path | str | None = None, *,
    run: Run = az_json, what_if: bool = False, retry: bool = False,
    sleep: Callable[[float], None] = time.sleep,
    clock: Callable[[], float] = time.monotonic,
) -> dict:
    """Provision only this plan's owned scope. Calling this function can incur cost."""
    directory, config, manifest = _load(config_path)
    if what_if and retry:
        raise BootstrapError("--retry cannot be combined with read-only --what-if.")
    if not what_if:
        approval = _approved_record(config, approval_path)
    else:
        approval = {}
    with _lock(directory):
        directory, config, manifest = _load(config_path)
        _check_outputs(directory, manifest)
    try:
        report, raw, observed = _preflight(config, manifest, run)
    except BootstrapError as exc:
        _record_failure(directory, "pre-apply", exc)
```

```

        raise
    _write(directory / "evidence" / f"pre-apply-{uuid4().hex}.local.json", raw)
    if what_if:
        if "resource_group" not in observed:
            raise BootstrapError("Read-only group what-if needs the already-owned group; it never
creates a group.")
        elif any(m["deprecated"] for m in report["models"]) and not
approval["accept_deprecated_models"]:
            raise ApprovalError("A selected model is deprecating; explicit acceptance or a newly
approved plan is required.")
        principal = raw["identity"]["user"]["id"]
        if what_if:
            path = directory / "evidence" / f"what-if-parameters-{uuid4().hex}.local.json"
            _write(path, _parameters(config, principal))
            result = run([
                "deployment", "group", "what-if", "--subscription", config["subscription_id"],
                "--resource-group", config["names"]["resource_group"], "--name",
config["deployment_name"],
                "--template-file", str(directory / "template.json"), "--parameters", f"@{path}", "--
mode", "Incremental",
                "--no-pretty-print",
            ])
            _write(directory / "evidence" / f"what-if-{uuid4().hex}.local.json", {"result": result})
            return {"status": "WHAT_IF_ONLY", "scope_sha256": config["scope_sha256"],
"mutations_performed": False}
        start = clock()
        deadline = start + approval["max_wait_seconds"]

    def bounded(args: list[str]) -> Any:
        try:
            validate_approval(config, approval)
        except BootstrapError as exc:
            raise ApprovalError(str(exc)) from exc
        remaining = deadline - clock()
        if remaining <= 0:
            raise BootstrapError("Approved wait exhausted; inspect status and resume, never delete
or recreate blindly.")
        return az_json(args, timeout=min(remaining, 120)) if run is az_json else run(args)

    manifest["operator_principal_id"] = principal
    manifest["approved_currency"] = approval["currency"]
    manifest["approved_budget"] = approval["budget_amount"]
    manifest["approved_budget_policy"] = approval.get("budget_policy", "BOUNDED")
    initial_attempts = len(manifest["attempts"])
    stage = "ownership"
    active_command = None
    submitted_attempt = None
    try:
        if "resource_group" not in observed:
            if manifest["resources"]["resource_group"]["status"] not in {"planned", "pending"}:
                raise BootstrapError("Previously owned resource group is missing; automatic
restore/recreation is forbidden.")
            if manifest["resources"]["resource_group"]["status"] == "pending":
                raise ProvisioningRetryError(
                    "UNKNOWN_GROUP_SUBMISSION",
                    "A pending resource-group creation intent exists but the group is absent. "
                    "Inspect original submission/activity evidence; do not issue another create

```

```

blindly.",
        )
        # Recheck immediately before conditional create; az group create would silently
upsert.
        if _get(manifest["resources"]["resource_group"], bounded) is not None:
            raise BootstrapError("Resource group name collision; no adoption or overwrite.")
        _pending(directory, manifest, ["resource_group"], "creating_group", approval)
        group = manifest["resources"]["resource_group"]
        stage = "group-create"
        active_command = [
            "rest", "--method", "PUT", "--url", f"{ARM}-{group['id']}?api-version=
{group['api_version']}",
            "--headers", "If-None-Match=*", "--body", _json({"location": REGION, "tags":
_tags(config)}),
        ]
        bounded(active_command)
        remote = _get(group, bounded)
        if remote is None:
            raise BootstrapError("Group creation is not yet observable; use status before
resume.")
        _owned(config, "resource_group", group, remote, manifest, {})
        manifest["resources"]["resource_group"]["status"] = "present"
        _save(directory, manifest)
        observed = _inspect(config, manifest, bounded)
        if "resource_group" not in observed:
            raise BootstrapError("Owned group must exist before any ARM deployment.")
        arm = observed.get("arm_deployment", {}).get("properties", {}).get("provisioningState")
        stage, active_command = "arm-retry-check", None
        if _submission_required(manifest, observed, approval, retry):
            stage = "arm-validation"
            validation = _validate_arm(directory, config, principal, bounded)
            manifest["last_validation"] = validation
            parameter_path = Path(validation["parameters_path"])
            observed = _inspect(config, manifest, bounded)
            if not _submission_required(manifest, observed, approval, retry):
                raise BootstrapError("Deployment state changed during validation; inspect status
instead of issuing another create.")
            _pending(directory, manifest, [k for k in manifest["resources"] if k !=
"resource_group"], "deploying", approval)
            submitted_attempt = manifest["attempts"][-1]
            submitted_attempt.update(
                submission_outcome="pending", retry_requested=retry,
                validation_evidence_file=validation["evidence_file"],
            )
            _save(directory, manifest)
            stage = "arm-create"
            active_command = [
                "deployment", "group", "create", "--subscription", config["subscription_id"],
                "--resource-group", config["names"]["resource_group"], "--name",
config["deployment_name"],
                "--template-file", str(directory / "template.json"), "--parameters",
f"@{parameter_path}",
                "--mode", "Incremental", "--no-wait",
            ]
            response = bounded(active_command)
            submitted_attempt["submission_outcome"] = "submitted"
            _save(directory, manifest)

```

```

        _write(directory / "evidence" / f"create-{uuid4().hex}.local.json", {
            "command": active_command, "response": response, "recorded_at": _stamp(),
            "status": "SUBMITTED_NOT_VERIFIED",
        })
        arm = None
        stage = "arm-status"
        while arm != "Succeeded":
            deployment = _get(manifest["resources"]["arm_deployment"], bounded)
            arm = (deployment or {}).get("properties", {}).get("provisioningState")
            if arm in {"Failed", "Canceled", "Cancelled"}:
                payload = (deployment or {}).get("properties", {}).get("error")
                if payload:
                    spec = manifest["resources"]["arm_deployment"]
                    raise _azure_error("", _json(deployment), [
                        "rest", "--method", "GET", "--url",
                        f"{ARM}{spec['id']}?api-version={spec['api_version']}",
                    ])
                raise AzureCommandError("DeploymentFailed", payload=deployment, message="ARM
deployment failed; inspect saved results before an explicitly budgeted retry.")
            if arm != "Succeeded":
                remaining = deadline - clock()
                if remaining <= 0:
                    raise BootstrapError("Approved wait exhausted; deployment may still run.
Inspect status.")
                sleep(min(5, remaining))
            observed = _inspect(config, manifest, bounded)
            if set(observed) != set(manifest["resources"]):
                raise BootstrapError("ARM succeeded but the complete owned inventory is not
observable; do not claim success.")
            routing = observed["insights_connection"].get("properties", {}).get("metadata",
{}).get(TRACE_ROUTING_KEY)
            expected_routing = observed["insights"].get("properties", {}).get("ConnectionString")
            if not expected_routing or routing != expected_routing:
                raise BootstrapError("ARM success is insufficient: trace routing metadata must match
the owned component's ConnectionString.")
            for key, remote in observed.items():
                state = remote.get("properties", {}).get("provisioningState")
                if state is not None and str(state).lower() != "succeeded":
                    raise BootstrapError(f"Resource is not ready: {key}.")
                if state is None and (key in RESOURCE_TYPES or key.startswith("model:")):
                    raise BootstrapError(f"Resource readiness is missing: {key}.")
                manifest["resources"][key]["status"] = "succeeded"
            content = _env(config, observed["insights"], approval_path)
            if not (directory / ".env").exists():
                manifest["env_sha256"] = hashlib.sha256(content.encode()).hexdigest()
                manifest["phase"] = "writing_env"
                _save(directory, manifest)
                _write(directory / ".env", content, text=True)
            manifest["env_sha256"] = _file_digest(directory / ".env")
            manifest["phase"] = "succeeded"
            _save(directory, manifest)
        return {
            "status": "APPLIED", "scope_sha256": config["scope_sha256"],
            "env_path": str(directory / ".env"), "artifacts_dir": str(directory / "artifacts"),
            "resource_count": len(observed), "data_plane_verified": False,
            "live_status": "NOT_VERIFIED",
            "server_side_trace_connection": "CONFIGURED_PROJECT_MANAGED_IDENTITY",

```

```

        "trace_ingestion_verified": False,
    }
except BootstrapError as exc:
    if submitted_attempt is not None and submitted_attempt.get("submission_outcome") ==
"pending":
        submitted_attempt["submission_outcome"] = "unknown"
        evidence_file = _record_failure(directory, stage, exc, command=active_command)
        if submitted_attempt is not None:
            submitted_attempt["failure_evidence_file"] = evidence_file
        manifest.setdefault("failures", []).append({
            "recorded_at": _stamp(), "stage": stage, "error_code": getattr(exc, "code", None),
            "evidence_file": evidence_file,
        })
        if isinstance(exc, ApprovalError) and len(manifest["attempts"]) > initial_attempts:
            exc.mutations_performed = None
            manifest["phase"] = "interrupted"
            manifest["last_error"] = str(exc)
            manifest["last_error_code"] = getattr(exc, "code", None)
            manifest["last_error_evidence_file"] = evidence_file
            if getattr(exc, "validation_evidence_file", None):
                manifest["last_validation"] = {"status": "FAILED", "evidence_file":
exc.validation_evidence_file}
            _save(directory, manifest)
            raise

```

Inspect the actual ARM template inputs too. This object passes names, models, the user ID, scoped role assignments, tags, and retention.

Actual implementation · read only · lab/bootstrap.py:_parameters

Read-only source, not another command to run. The complete named function or object is shown as written, including its original messages; dependencies and imports are in the linked file. Run the lab command only once.

Full source file · 1391–1400 (file: [lab/bootstrap.py#L1391–L1400](#)) · [View on GitHub](#)
<https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/bootstrap.py#L1391-L1400>

```

def _parameters(config: dict, principal: str) -> dict:
    return {
        "$schema": "https://schema.management.azure.com/schemas/2019-04-
01/deploymentParameters.json#",
        "contentVersion": "1.0.0.0",
        "parameters": {key: {"value": value} for key, value in {
            "location": REGION, "names": config["names"], "tags": _tags(config),
            "models": config["models"], "roles": _roles(config),
            "operatorPrincipalId": principal, "retentionDays": config["retention_days"],
        }.items()},
    }

```

Read `status()` too: it shows which remote state is inspected after an apply timeout.

Actual implementation · read only · lab/bootstrap.py:status

Read-only source, not another command to run. The complete named function or object is shown as written, including its original messages; dependencies and imports are in the linked file. Run the lab command only once.

Full source file · 2095–2122 (file: lab/bootstrap.py#L2095–L2122) · [View on GitHub](#)

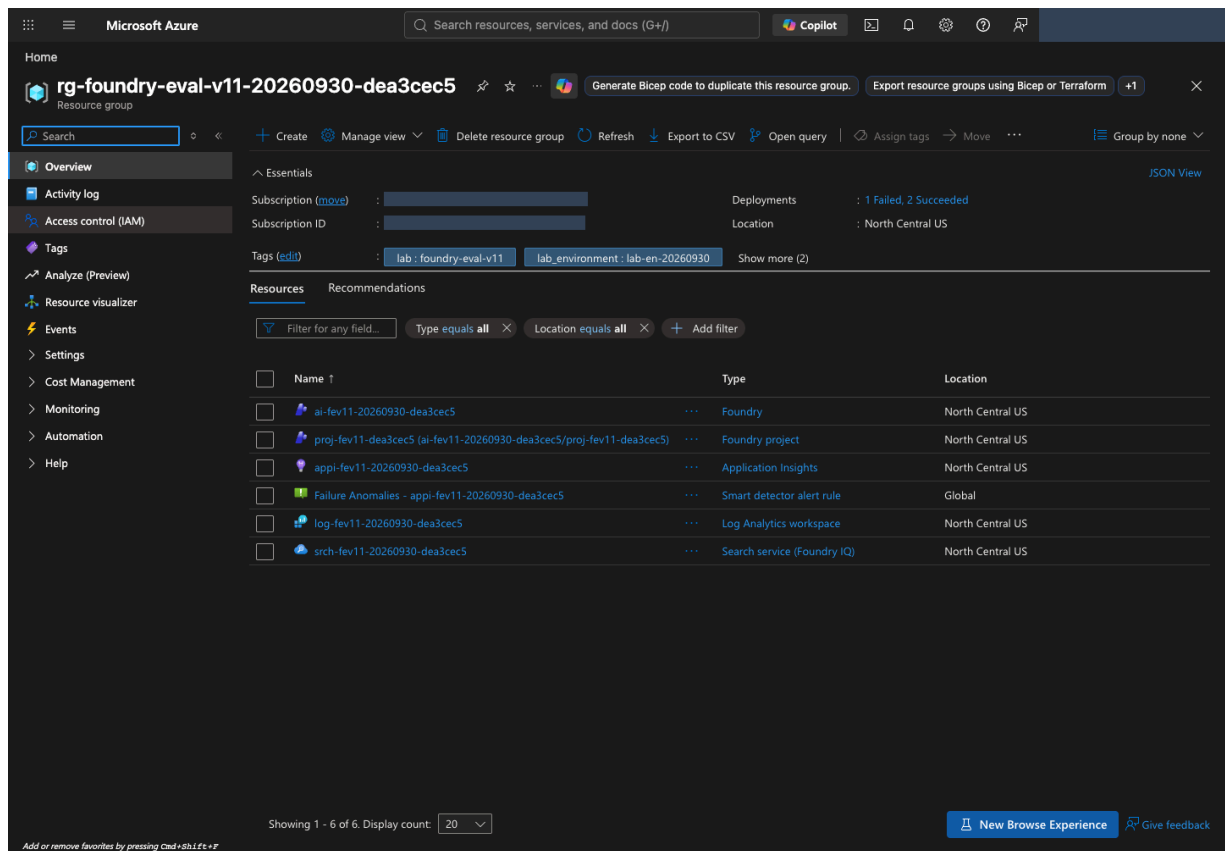
(<https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/bootstrap.py#L2095-L2122>)

```
def status(
    config_path: Path | str, *, run: Run = az_json, approval_path: Path | str | None = None,
) -> dict:
    """Inspect only declared resources; no deletes, provider registration, or permission repair."""
    directory, config, manifest = _load(config_path)
    _identity(config, manifest, run)
    try:
        observed = _inspect(config, manifest, run)
    except BootstrapError as exc:
        return {"status": "BLOCKED", "scope_sha256": config["scope_sha256"], "reason": str(exc)}
    result = {
        *_authorization(config, approval_path),
        "observation_status": "OBSERVED", "live_status": "NOT_VERIFIED",
        "scope_sha256": config["scope_sha256"], "phase": manifest["phase"],
        "resources": {key: {
            "present": key in observed,
            "provisioning_state": observed.get(key, {}).get("properties", {}).get("provisioningState",
"UNKNOWN"),
            "recorded_state": spec["status"], "sku": spec["sku"],
        } for key, spec in manifest["resources"].items()},
        "mutations_performed": False,
        "warning": "Search and any separately deployed fine-tuned model may still incur continuous
hosting costs.",
    }
    if result["approval_status"] == "APPROVED":
        result["status"] = "OBSERVED_APPROVAL_VALID"
        result["live_status"] = "PENDING_EXECUTION"
    result["readiness_status"] = "NOT_CHECKED_BY_STATUS"
    _write(directory / "evidence" / f"status-{uuid4().hex}.local.json", {"observed": observed,
"summary": result})
    return result
```

Verify the created environment and runtime settings

1. In [Azure Portal](https://portal.azure.com) (<https://portal.azure.com>) → **Resource groups**, search for `names.resource_group` from `config.json`. Confirm the subscription, region, and complete resource list.
2. Open [Foundry](https://ai.azure.com) (<https://ai.azure.com>) with **New Foundry** enabled. If **Select a project to continue** appears, choose `names.project` and select **Let's go**. Read and **Close** the welcome tour if shown. If already in New Foundry, use the upper-left project selector. Do not use a Classic hub-based project.
3. In the project's **Home** (Overview in some layouts), find **Project endpoint**. It must match `AZURE_AI_PROJECT_ENDPOINT` in `.env`, in the form `https://account.services.ai.azure.com/api/projects/project`.

4. In **Models + endpoints** or **Build → Models**, locate the deployments matching `.env` values `MODEL_DEPLOYMENT`, `JUDGE_DEPLOYMENT`, `OPTIMIZER_DEPLOYMENT`, and `EMBEDDING_DEPLOYMENT`. See [portal orientation](#) if labels differ.



The screenshot displays the Microsoft Azure portal interface for a resource group named `rg-foundry-eval-v11-20260930-dea3cec5`. The 'Overview' tab is active, showing the following details:

- Subscription:** (move)
- Subscription ID:** (move)
- Location:** North Central US
- Deployments:** 1 Failed, 2 Succeeded
- Tags:** lab : foundry-eval-v11, lab_environment : lab-en-20260930

Below the overview, a table lists the resources within the group:

Name	Type	Location
ai-fev11-20260930-dea3cec5	Foundry	North Central US
proj-fev11-dea3cec5 (ai-fev11-20260930-dea3cec5/proj-fev11-dea3cec5)	Foundry project	North Central US
appi-fev11-20260930-dea3cec5	Application Insights	North Central US
Failure Anomalies - appi-fev11-20260930-dea3cec5	Smart detector alert rule	Global
log-fev11-20260930-dea3cec5	Log Analytics workspace	North Central US
srch-fev11-20260930-dea3cec5	Search service (Foundry IQ)	North Central US

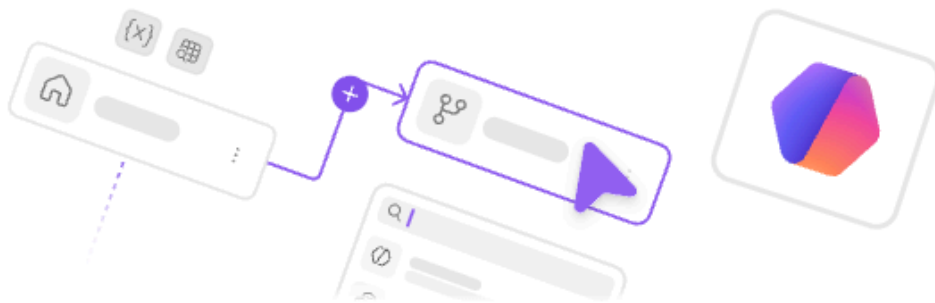
Compare the plan with actual resources. Check the group name and Location, then identify Foundry, Foundry project, Search, Application Insights, and Log Analytics. Open Deployments to inspect each deployment's status and any failure details. View full-size image (file: `web/assets/portal/en/23-resource-group.png`)

Select a project to continue



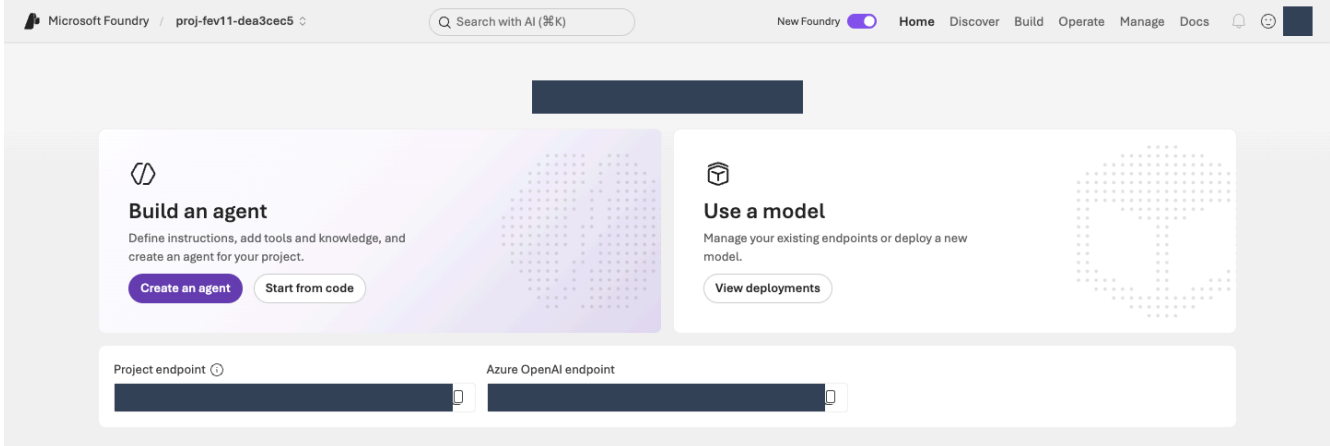
To access the new Microsoft Foundry experience, please choose a project in a supported region.

proj-fev11-dea3cec5



Let's go

Select the project you prepared. Compare its name with `names.project` and open it with Let's go. This selects an existing project; it does not create another. After bootstrap, do not use Create a new project to duplicate the environment. View full-size image (file: [web/assets/portal/en/24-select-project.png](#))



Check the project and endpoint. Compare the upper-left project name and **Project endpoint** with your configuration. Do not substitute Azure OpenAI endpoint. Open the model list through View deployments. View full-size image (file: [web/assets/portal/en/25-project-overview.png](#))

Check identity, deployments, and TPM in your terminal

Run this read-only command after provisioning creates `.lab/lab-en/.env`. It also performs the identity check internally.

Run in your terminal · Shared terminal

```
python -m lab --config .lab/lab-en/.env preflight
```

Inspect `agent_tpm`, `judge_tpm`, `optimizer_tpm`, `iq_planner_tpm`, and `embedding_tpm` under checks. Each observed value is actual deployment TPM; expected is the recommended minimum above. Insufficient or unverifiable token limits report `BLOCKED`: do not continue to model calls. Prepare the allocation, then rerun the same read-only preflight.

Code ↔ portal · compare generated settings with the execution identity

Where to act: Run the [runtime-check command above](#). It compares the generated configuration with your current CLI identity; do not type the identity-check function separately.

Command/setting	Actual action	Portal verification
<code>AZURE_SUBSCRIPTION_ID</code> , <code>AZURE_TENANT_ID</code> , and <code>EXPECTED_AZURE_USER</code> from <code>--config .lab/lab-en/.env</code>	Read the subscription, tenant, and user recorded for this provisioned environment.	Compare the Azure Portal account's Directory and subscription Overview checked in 01.
<code>az account show --subscription ...</code> inside <code>check_identity()</code>	Automatically compare configured user, tenant, subscription, and Enabled state with the CLI. A mismatch stops execution before resource lookup.	Confirm the same account/subscription; successful sign-in does not permit a different identity.

This function performs the command's identity check. Subscription selection does not replace tenant verification. This check is not an SDK token request or a model call.

Actual implementation · read only · `lab/preflight.py:check_identity`

Read-only source, not another command to run. The complete named function or object is shown as written, including its original messages; dependencies and imports are in the linked file. Run the lab command only once.

Full source file · 69–87 (file: `lab/preflight.py#L69-L87`) · [View on GitHub \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/preflight.py#L69-L87\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/preflight.py#L69-L87)

```
def check_identity(config: Config, run: Callable = az_json) -> dict:
    account = run(["account", "show", "--subscription", config.subscription_id])
    if not isinstance(account, dict):
        raise LabError("az account show 응답 형식이 올바르지 않습니다.")
    actual_user = account.get("user", {}).get("name", "")
    checks = {
        "subscription": account.get("id") == config.subscription_id,
        "tenant": account.get("tenantId") == config.tenant_id,
        "user": actual_user.lower() == config.expected_user.lower(),
        "enabled": account.get("state") == "Enabled",
    }
```

```

if not all(checks.values()):
    failed = ", ".join(key for key, ok in checks.items() if not ok)
    raise LabError(
        f"로그인 환경 불일치({failed}). 작업을 중단했습니다. "
        f"az login --tenant {config.tenant_id} 후 지정 계정으로 로그인해야 합니다. "
        "SDK는 다른 환경 자격 증명으로 자동 전환하지 않습니다."
    )
return {"user": actual_user, "subscription": account["id"], "tenant": account["tenantId"]}

```

The following runtime `run_preflight()` reads the resources/deployments named in `.env`. Connect `rateLimits` entries with key: `token`, including the raw ARM lookup, to the portal TPM display. **Subscription quota and a deployment's actual TPM are separate checks.**

Actual implementation · read only · `lab/preflight.py:run_preflight`

Read-only source, not another command to run. The complete named function or object is shown as written, including its original messages; dependencies and imports are in the linked file. Run the lab command only once.

Full source file · 90–224 (file: `lab/preflight.py#L90–L224`) · [View on GitHub \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/preflight.py#L90-L224\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/preflight.py#L90-L224)

```

def run_preflight(config: Config, *, run: Callable = az_json) -> dict:
    identity = check_identity(config, run)
    account = run([
        "cognitiveservices", "account", "show", "--subscription", config.subscription_id,
        "--resource-group", config.resource_group, "--name", config.account,
    ])
    project = run([
        "resource", "show", "--ids", config.project_id, "--api-version", "2025-06-01",
    ])
    search = run([
        "search", "service", "show", "--subscription", config.subscription_id,
        "--resource-group", config.resource_group, "--name", config.search_service,
    ])
    deployments = run([
        "cognitiveservices", "account", "deployment", "list",
        "--subscription", config.subscription_id, "--resource-group", config.resource_group,
        "--name", config.account,
    ])
    checks: list[dict] = []
    for name, resource in (("foundry", account), ("project", project), ("search", search)):
        region = str(resource.get("location", "")).lower().replace(" ", "")
        checks.append({
            "name": f"{name}_region",
            "status": "PASS" if region == config.location else "BLOCKED",
            "observed": region,
            "expected": config.location,
        })
    endpoint = project.get("properties", {}).get("endpoints", {}).get("AI Foundry API")
    checks.append({
        "name": "project_endpoint",
        "status": "PASS" if endpoint == config.project_endpoint else "BLOCKED",
        "observed": endpoint,
    })

```

```

deployment_map = {d["name"]: d for d in deployments}
model_evidence = {}
roles = (
    ("agent", config.model), ("judge", config.judge), ("optimizer", config.optimizer),
    ("iq_planner", config.planner), ("embedding", config.embedding),
)
rate_limit_sources = {}
for name in dict.fromkeys(name for _, name in roles):
    deployment = deployment_map.get(name)
    if not deployment:
        continue
    limits = deployment.get("properties", {}).get("rateLimits")
    source = "Azure CLI deployment metadata"
    if not isinstance(limits, list) or not any(
        isinstance(limit, dict) and limit.get("key") == "token" for limit in limits
    ):
        # Use raw ARM when CLI metadata lacks token/request labels instead of inferring TPM.
        identifier = f"{config.account_id}/deployments/{quote(name, safe='')}\"
        deployment = run([
            "rest", "--method", "GET", "--url",
            f"https://management.azure.com{identifier}?api-version=2025-06-01",
        ])
        if (
            not isinstance(deployment, dict) or deployment.get("name") != name
            or str(deployment.get("id", "")).lower() != identifier.lower()
            or not isinstance(deployment.get("properties"), dict)
        ):
            raise LabError("TPM 조회 응답이 요청한 모델 배포와 다릅니다.")
        deployment_map[name] = deployment
        source = "Raw ARM deployment metadata"
        rate_limit_sources[name] = source
    for role, name in roles:
        deployment = deployment_map.get(name)
        ready = deployment and deployment.get("properties", {}).get("provisioningState") ==
"Succeeded"
        checks.append({
            "name": f"{role}_deployment", "status": "PASS" if ready else "BLOCKED",
            "observed": name,
        })
        if deployment:
            limits = deployment.get("properties", {}).get("rateLimits")
            minimum_tpm = LAB_MINIMUM_TPM[role]
            try:
                observed_tpm = _tokens_per_minute(limits)
                sufficient = observed_tpm >= minimum_tpm
                reason = (
                    "Meets the lab starting TPM requirement; RPM and shared load still apply."
                    if sufficient else
                    f"Allocate at least {minimum_tpm:,} TPM with operator approval, then rerun
preflight."
                )
            except ValueError as exc:
                observed_tpm, sufficient, reason = None, False, str(exc)
            checks.append({
                "name": f"{role}_tpm", "status": "PASS" if sufficient else "BLOCKED",
                "observed": observed_tpm, "expected": minimum_tpm, "reason": reason,
            })

```

```

        model_evidence[role] = {
            "deployment": name,
            "model": deployment.get("properties", {}).get("model"),
            "sku": deployment.get("sku"),
            "tokens_per_minute": observed_tpm,
            "rate_limit_source": rate_limit_sources[name],
        }
    if role == "iq_planner":
        observed_model = (deployment or {}).get("properties", {}).get("model", {}).get("name")
        checks.append({
            "name": "iq_planner_supported_model",
            "status": "PASS" if observed_model == "gpt-5.5" else "BLOCKED",
            "observed": observed_model,
            "expected": "gpt-5.5",
        })
    if role == "embedding":
        observed_model = (deployment or {}).get("properties", {}).get("model", {}).get("name")
        checks.append({
            "name": "embedding_supported_model",
            "status": "PASS" if observed_model == "text-embedding-3-small" else "BLOCKED",
            "observed": observed_model, "expected": "text-embedding-3-small",
        })
    public_access = account.get("properties", {}).get("publicNetworkAccess")
    if public_access == "Disabled":
        checks.append({
            "name": "network", "status": "BLOCKED",
            "observed": "PublicNetworkAccess=Disabled: 승인된 VNet 연결 환경에서 실행해야 합니다.",
        })
    return {
        "checked_at": datetime.now(timezone.utc).isoformat(),
        "kind": "read-only-management-preflight",
        "status": "PASS" if all(c["status"] == "PASS" for c in checks) else "BLOCKED",
        "identity": identity,
        "checks": checks,
        "models": model_evidence,
        "identities": {
            "foundry_project_managed_identity": project.get("identity", {}).get("principalId"),
            "search_managed_identity": search.get("identity", {}).get("principalId"),
        },
        "not_verified": [
            "데이터 평면 RBAC와 모델 추론 성공",
            "Foundry IQ 지식 베이스 생성·검색 권한",
            "Prompt Optimizer 및 Frontier Tuning 테넌트 접근 권한",
            "GlobalStandard는 North Central US 내부 처리를 보장하지 않습니다.",
            "TPM 충족은 남은 토큰, 공유 부하, RPM·버스트 제한 또는 429 없는 실행을 보장하지 않습니다.",
        ],
    }
}

```

Completion criteria: Runtime preflight and all five TPM checks report **PASS**; your project and role-specific deployments are present. This is read-only configuration evidence. The next step verifies actual model responses.

03. Connect policies and create the Agent

What: Connect a read-only synthetic-policy tool to the pinned v1 baseline Agent.

Why: Recalling a policy is not the same as retrieving it. Retrieval under your user identity does not establish access for the Agent's managed identity.

How · where: Prepare model, retrieval, and Agent from the terminal; inspect instructions, tools, and an actual response in Foundry. Transmit data and make calls once within the approved scope.

Action order: [Terminal · model check](#) → [Upload policies](#) → [Verify retrieval](#) → [Create v1](#) → [Foundry · test response](#).

Verify the model and policy retrieval

These three commands perform actual uploads/model calls and can incur charges.

Check each result yourself before continuing. When resuming, inspect your records for completed work rather than repeating unnecessary calls.

1. Verify that the model responds

`smoke` is a short functional check.

First pass the [TPM checks in 02](#). One successful short response does not establish sufficient throughput for the full evaluation.

Run in your terminal · Shared terminal

```
python -m lab --config .lab/lab-en/.env smoke --run-id model-smoke --confirm
```

Confirm `status: completed` in the output.

Code ↔ portal · connect the SDK with the verified CLI identity

Where to act: Run the [model-check command above](#). The authentication function is used automatically inside actual SDK calls, not as a separate terminal command.

Actual code	Actual action	Portal verification
AzureCliCredential(subscription=...) in credential_for()	Recheck the CLI identity against 02's configuration and create a credential object. The SDK requests tokens with	Compare the signed-in Foundry account and project subscription with the

Actual code	Actual action	Portal verification
	that identity when calling the service. Do not switch to API keys or another credential.	configuration. There is no action to paste tokens into the portal.

The first function below creates the credential passed to the SDK; the next sends the actual model request. Authentication and response verification belong to this step.

Actual implementation · read only · lab/auth.py:credential_for

Read-only source, not another command to run. The complete named function or object is shown as written, including its original messages; dependencies and imports are in the linked file. Run the lab command only once.

Full source file · 11–15 (file: lab/auth.py#L11–L15) · [View on GitHub \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/auth.py#L11-L15\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/auth.py#L11-L15)

```
def credential_for(config: Config) -> AzureCliCredential:
    check_identity(config)
    # Azure CLI rejects get-access-token when both tenant and subscription are
    # supplied. The identity check above binds the subscription to the tenant.
    return AzureCliCredential(subscription=config.subscription_id, process_timeout=30)
```

client.responses.create(model=config.model, ...) inside smoke_model() is the real Sol model call. It tests a short input with at most 128 output tokens before creating an Agent. The request is recorded first so an unknown submission is not replayed.

Actual implementation · read only · lab/agents.py:smoke_model

Read-only source, not another command to run. The complete named function or object is shown as written, including its original messages; dependencies and imports are in the linked file. Run the lab command only once.

Full source file · 168–207 (file: lab/agents.py#L168–L207) · [View on GitHub \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/agents.py#L168-L207\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/agents.py#L168-L207)

```
def smoke_model(config: Config, run_id: str) -> dict:
    directory = safe_run_dir(run_id)
    path = directory / "model-smoke.json"
    if path.exists():
        previous = read_json(path)
        require_content_language(previous)
        if previous.get("project_endpoint") != config.project_endpoint or previous.get("deployment")
        != config.model:
            raise LabError("기존 model smoke의 환경/배포가 다릅니다.")
        if previous.get("status") != "completed":
            raise LabError("이 model smoke는 실패 또는 결과 불명입니다. 원본을 확인하기 전 재호출하지 않습니다.")
        return previous
    snapshot = model_snapshot(config, config.model)
    record = {
        **language_metadata(),
        "kind": "LIVE_MODEL_SMOKE_NOT_QUALITY_EVALUATION", "status": "submitting",
```

```

        "started_at": datetime.now(timezone.utc).isoformat(),
        "code": code_provenance(),
        "project_endpoint": config.project_endpoint, "deployment": config.model,
        "model_snapshot": snapshot, "response_id": None, "cost": {"status": "NOT_OBSERVED"},
        "request": {"input": "Synthetic Contoso lab connectivity check. Reply with the single word
READY.", "max_output_tokens": 128},
    }
    write_once_json(path, record)
    started = time.perf_counter()
    with credential_for(config) as credential:
        with AIProjectClient(endpoint=config.project_endpoint, credential=credential, retry_total=0)
as project:
            with project.get_openai_client(max_retries=0, timeout=120.0) as client:
                response = client.responses.create(model=config.model, **record["request"],
store=True)

                record["response_id"] = response.id
                record["response"] = response.model_dump(mode="json")
                record["usage"] = record["response"].get("usage")
                record["latency_ms"] = (time.perf_counter() - started) * 1000
                record["status"] = "completed" if (
                    record["response"].get("status") == "completed"
                    and isinstance(response.id, str) and response.id.strip()
                    and isinstance(response.output_text, str) and response.output_text.strip()
                ) else "failed"
                save_json(path, record)
            if record["status"] != "completed":
                raise LabError("모델 smoke가 완료된 비어 있지 않은 응답을 반환하지 않았습니다.")
    return record

```

2. Prepare policy retrieval

Upload the eight supplied synthetic policies (file: data/en/knowledge/documents.json) to the search service.

Run in your terminal · Shared terminal

```
python -m lab --config .lab/lab-en/.env iq prepare --confirm
```

Confirm uploaded_documents: 8. The program prepares a search index (searchable documents), a knowledge base, and an MCP connection. **MCP (Model Context Protocol) connects an Agent to tools**; this lab connects only a read-only policy-retrieval tool. You do not need to edit the implementation.

Code ↔ portal · configure policy retrieval

Where to act: Run [iq prepare above](#), then [iq probe below](#) in your terminal. The panels show those commands' internals, not additional programs to execute.

Actual code/setting	Actual action	Portal actions and verification
<code>fields, vectorSearch, semantic</code> in <code>knowledge_payloads()["index"]</code>	Define policy fields, 1536-dimensional vectors, HNSW, embeddings, and semantic settings.	Azure Portal → Search → Indexes → created index → Fields/JSON . Inspect, do not recreate, the index.
<code>client.embeddings.create(...)</code> in <code>embed()</code>	Send eight policy titles/contents to the actual embedding deployment.	Foundry View deployments → embedding deployment → Details : check the model/name. Vectors are stored in Search, not Agent Playground settings.
PUT/POST in <code>prepare_knowledge()</code>	Create the index, upload documents, then create the knowledge source, base, and project connection.	Inspect Search Indexes and Foundry Build → Knowledge → Knowledge bases for matching names/sources.
Connection <code>authType</code> , <code>audience</code> , <code>target</code>	Connect the project's managed identity to Search's MCP endpoint.	Inspect Knowledge Connection . If the UI has no <code>authType</code> / <code>audience</code> editor, read <code>knowledge/config-snapshot.json</code> ; do not guess settings.
<code>/retrieve, references, activity</code> in <code>probe_knowledge()</code>	Verify actual policy retrieval.	Active is registration state, not a probe result. Inspect the probe's output/records, then verify the Agent's tool call separately in Playground.

First read the **complete service-setting objects**, then the **actual creation/upload code**. `config.*` comes from 02's `.env`; names contains the actual object names in your records.

Actual implementation · read only · `lab/knowledge.py:knowledge_payloads`

Read-only source, not another command to run. The complete named function or object is shown as written, including its original messages; dependencies and imports are in the linked file. Run the lab command only once.

Full source file · 33–105 (file: `lab/knowledge.py#L33–L105`) · [View on GitHub \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/knowledge.py#L33–L105\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/knowledge.py#L33–L105)

```
def knowledge_payloads(config: Config, names: dict, planner_model: str) -> dict:
    if not config.embedding:
        raise LabError("벡터 검색을 위해 EMBEDDING_DEPLOYMENT를 명시해야 합니다.")
    analyzer = f"{selected_language()}.microsoft"
    return {
        "index": {
            "name": names["index"],
            "fields": [
                {"name": "id", "type": "Edm.String", "key": True, "filterable": True, "retrievable":
True},
                {"name": "title", "type": "Edm.String", "searchable": True, "retrievable": True,
"analyzer": analyzer},
                {"name": "content", "type": "Edm.String", "searchable": True, "retrievable": True,
"analyzer": analyzer},
                {"name": "effective_date", "type": "Edm.String", "filterable": True, "retrievable":
True},
```

```

        {"name": "content_vector", "type": "Collection(Edm.Single)", "searchable": True,
         "retrievable": False, "dimensions": DIMENSIONS, "vectorSearchProfile": "contoso-
vector"},
    ],
    "vectorSearch": {
        "algorithms": [{"name": "contoso-hnsw", "kind": "hnsw", "hnswParameters": {"metric":
"cosine"}}],
        "profiles": [{"name": "contoso-vector", "algorithm": "contoso-hnsw", "vectorizer":
"contoso-embedding"}],
        "vectorizers": [{
            "name": "contoso-embedding", "kind": "azureOpenAI",
            "azureOpenAIParameters": {
                "resourceUri": config.openai_endpoint, "deploymentId": config.embedding,
                "modelName": "text-embedding-3-small",
            },
        }],
    },
    "semantic": {
        "defaultConfiguration": "support-semantic",
        "configurations": [{
            "name": "support-semantic",
            "prioritizedFields": {
                "titleField": {"fieldName": "title"},
                "prioritizedContentFields": [{"fieldName": "content"}],
            },
        }],
    },
    "source": {
        "name": names["source"],
        "kind": "searchIndex",
        "searchIndexParameters": {
            "searchIndexName": names["index"],
            "semanticConfigurationName": "support-semantic",
            "sourceDataFields": [{"name": field} for field in ("id", "title", "content",
"effective_date")],
            "searchFields": [{"name": field} for field in ("title", "content", "content_vector")],
        },
    },
    "base": {
        "name": names["base"],
        "description": "Synthetic Contoso policies. Stable ATLAS document IDs are in titles and
source data.",
        "knowledgeSources": [{"name": names["source"]}],
        "models": [{
            "kind": "azureOpenAI",
            "azureOpenAIParameters": {
                "resourceUri": config.openai_endpoint,
                "deploymentId": config.planner,
                "modelName": planner_model,
            },
        }],
        "retrievalReasoningEffort": {"kind": "low"},
        "outputMode": "extractiveData",
    },
    "connection": {
        "properties": {

```

```

        "authType": "ProjectManagedIdentity",
        "category": "RemoteTool",
        "target": search_url(config, "knowledgebases", names["base"], "/mcp"),
        "isSharedToAll": False,
        "audience": "https://search.azure.com/",
        "metadata": {"ApiType": "Azure"},
    },
},
}

```

The next short function performs **GET → create-only PUT → ownership recording**. `prepare_knowledge()` then applies it to the index, source, base, and connection.

Actual implementation · read only · `lab/knowledge.py:_ensure_created`

Read-only source, not another command to run. The complete named function or object is shown as written, including its original messages; dependencies and imports are in the linked file. Run the lab command only once.

Full source file · 108–125 (file: `lab/knowledge.py#L108–L125`) · [View on GitHub \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/knowledge.py#L108-L125\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/knowledge.py#L108-L125)

```

def _ensure_created(config: Config, http: JsonHttp, kind: str, name: str, url: str, payload: dict) -> dict:
    state = workspace(config)
    owned = [item for item in state["created"] if item.get("kind") == kind and item.get("name") == name]
    try:
        existing = http.request("GET", url)
    except CloudRequestError as exc:
        if exc.status != 404:
            raise
        existing = None
    if existing is not None:
        if not owned:
            raise LabError(f"{name}이 이미 존재하지만 이 실습의 생성 기록이 없습니다. 덮어쓰지 않습니다.")
        return existing.body
    result = http.request("PUT", url, payload, create_only=True)
    if result.status not in {200, 201}:
        raise LabError(f"{name}: 비동기/예상 밖 상태 {result.status}. 완료로 기록하지 않습니다.")
    record_created(config, {"kind": kind, "name": name, "url": url, "etag": result.etag})
    return result.body

```

Actual implementation · read only · `lab/knowledge.py:prepare_knowledge`

Read-only source, not another command to run. The complete named function or object is shown as written, including its original messages; dependencies and imports are in the linked file. Run the lab command only once.

Full source file · 128–239 (file: `lab/knowledge.py#L128–L239`) · [View on GitHub \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/knowledge.py#L128-L239\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/knowledge.py#L128-L239)

```

def prepare_knowledge(config: Config) -> dict:
    state = workspace(config, create=True)

```

```

names = resource_names(config, state)
documents_path = content_path(ROOT, "data/knowledge/documents.json")
documents = read_json(documents_path)
content_hash = sha256_file(documents_path)
setup_path = ARTIFACTS / "knowledge/setup.json"
previous = None
if setup_path.exists():
    previous = read_json(setup_path)
    require_content_language(previous)
    if previous.get("documents_sha256") != content_hash or previous.get("names") != names:
        raise LabError("기존 지식 준비와 원본/범위가 다릅니다. 다른 실험을 기존 인덱스에 덮어쓰지 않습니다.")
    if previous.get("status") in {"created_not_retrieval_tested", "retrieval_verified"}:
        if previous.get("retrieval", {}).get("embedding_deployment") != config.embedding:
            raise LabError("준비된 인덱스의 embedding 계약이 다릅니다. 새 실험을 생성해야 합니다.")
        return previous
with credential_for(config) as credential:
    planner = az_json([
        "cognitiveservices", "account", "deployment", "show",
        "--subscription", config.subscription_id, "--resource-group", config.resource_group,
        "--name", config.account, "--deployment-name", config.planner,
    ])
    model = planner.get("properties", {}).get("model", {})
    if not model.get("name"):
        raise LabError("IQ planner의 실제 모델 이름을 확인할 수 없습니다.")
    # This workshop's main lane was researched for gpt-5.5, not arbitrary
    # answer models that may be unsupported by the knowledge-base planner.
    if model["name"] != "gpt-5.5":
        raise LabError("이 경로의 IQ planner는 확인된 gpt-5.5 배포를 요구합니다. 다른 모델로 자동 전환하지 않습니다.")

    payloads = knowledge_payloads(config, names, model["name"])
    payload_hash = hashlib.sha256(json.dumps(payloads, sort_keys=True).encode()).hexdigest()
    if previous and previous.get("payload_sha256") != payload_hash:
        raise LabError("미완료 검색 설정의 구성 계약이 바뀌었습니다. 같은 인덱스에 덮어쓰지 않습니다.")
    setup = {
        **language_metadata(),
        "status": "preparing", "workspace_id": state["workspace_id"], "names": names,
        "project_id": config.project_id, "documents_sha256": content_hash,
        "planner": {"deployment": config.planner, "model": model},
        "search_api_version": SEARCH_API,
        "payload_sha256": payload_hash,
        "retrieval": {
            "reasoning": "low", "output": "extractiveData",
            "embedding_model": "text-embedding-3-small", "embedding_deployment": config.embedding,
            "dimensions": DIMENSIONS, "index_algorithm": "hnsw", "metric": "cosine",
            "query_modes": ["vector", "hybrid", "agentic-iq-mcp"],
            "direct_probe_is_agent_evidence": False,
        },
    }
    save_json(setup_path, setup)
    save_json(ARTIFACTS / "knowledge/config-snapshot.json", payloads)
    search = JsonHttp(credential, scope=SEARCH_SCOPE, allowed_origin=config.search_endpoint)
    index_url = search_url(config, "indexes", names["index"])
    _ensure_created(config, search, "search_index", names["index"], index_url, payloads["index"])
    embedding = embed(
        config, [f'{{doc['title']}}\n{{doc['content']}}' for doc in documents],
        ARTIFACTS / "knowledge/document-embeddings.json",
    )

```

```

vectors = validate_vectors(embedding["response"], len(documents))
uploaded = [{
    "@search.action": "mergeOrUpload",
    "id": doc["id"], "title": f"{doc['id']} | {doc['title']}",
    "content": f"{doc['id']}\n{doc['content']}",
    "effective_date": doc["effective_date"],
    "content_vector": vector,
} for doc, vector in zip(documents, vectors, strict=True)]
upload_path = ARTIFACTS / "knowledge/upload-response.json"
upload_contract = {"documents_sha256": content_hash, "payload_sha256": payload_hash,
    "embedding_sha256": sha256_file(ARTIFACTS / "knowledge/document-
embeddings.json")}
if upload_path.exists():
    upload_record = read_json(upload_path)
    if upload_record.get("contract") != upload_contract or upload_record.get("status") !=
"completed":
        raise LabError("업로드 계약 변경 또는 결과 불명입니다. 업로드를 반복하지 않고 실제 인덱스 상태를 확인해야 합
니다.")
    upload_body = upload_record["response"]
else:
    write_once_json(upload_path, {"status": "submitting", "contract": upload_contract})
    upload = search.request(
        "POST", search_url(config, "indexes", names["index"], "/docs/index"), {"value":
uploaded},
    )
    upload_body = upload.body
    save_json(upload_path, {"status": "response_received", "contract": upload_contract,
"response": upload_body})
    statuses = upload_body.get("value", [])
    if (
        len(statuses) != len(uploaded)
        or {item.get("key") for item in statuses} != {doc["id"] for doc in documents}
        or any(item.get("status") is not True for item in statuses)
    ):
        save_json(ARTIFACTS / "knowledge/upload-error.json", upload_body)
        raise LabError("일부 문서 업로드가 실패했습니다. knowledge/upload-error.json을 확인해야 합니다.")
    save_json(upload_path, {"status": "completed", "contract": upload_contract, "response":
upload_body})
    _ensure_created(
        config, search, "knowledge_source", names["source"],
        search_url(config, "knowledgesources", names["source"]), payloads["source"],
    )
    _ensure_created(
        config, search, "knowledge_base", names["base"],
        search_url(config, "knowledgebases", names["base"]), payloads["base"],
    )
    connection_url = (
        f"https://management.azure.com{config.project_id}/connections/"
        f"{names['connection']}?api-version={CONNECTION_API}"
    )
    arm = JsonHttp(credential, scope=ARM_SCOPE, allowed_origin="https://management.azure.com")
    _ensure_created(config, arm, "project_connection", names["connection"], connection_url,
payloads["connection"])
    setup["status"] = "created_not_retrieval_tested"
    setup["mcp_endpoint"] = payloads["connection"]["properties"]["target"]
    setup["connection_name"] = names["connection"]
    setup["uploaded_documents"] = len(uploaded)

```

```

setup["embedding_usage"] = embedding["response"].get("usage")
save_json(setup_path, setup)
save_json(ARTIFACTS / "knowledge/documents-snapshot.json", documents)
return setup

```

Embedding is not hidden local scoring. This function invokes Azure OpenAI's embedding API and preserves the original response, input hash, model, and usage.

Actual implementation · read only · `lab/embeddings.py:embed`

Read-only source, not another command to run. The complete named function or object is shown as written, including its original messages; dependencies and imports are in the linked file. Run the lab command only once.

Full source file · 41–97 (file: `lab/embeddings.py#L41–L97`) · [View on GitHub \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/embeddings.py#L41-L97\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/embeddings.py#L41-L97)

```

def embed(config: Config, texts: list[str], cache_path: Path) -> dict:
    if not config.embedding or not texts or any(not isinstance(text, str) or not text.strip() for text
    in texts):
        raise LabError("EMBEDDING_DEPLOYMENT와 비어 있지 않은 embedding 입력이 필요합니다.")
    deployment = model_snapshot(config, config.embedding)
    if deployment["model"].get("name") != "text-embedding-3-small":
        raise LabError("이 인덱스의 동결 계약은 text-embedding-3-small/1536입니다. 자동 대체하지 않습니다.")
    contract = {
        "endpoint": config.openai_endpoint, "deployment": deployment,
        "dimensions": DIMENSIONS,
        "input_sha256": hashlib.sha256(json.dumps(texts, ensure_ascii=False).encode()).hexdigest(),
        "count": len(texts),
    }
    if cache_path.exists():
        previous = read_json(cache_path)
        if previous.get("contract") != contract:
            raise LabError("기존 embedding 실행과 입력/모델/범위가 다릅니다. 기존 기록을 보존해야 합니다.")
        if previous.get("status") != "completed":
            raise LabError("이 embedding 요청은 실패 또는 결과 불명입니다. 원격 요청을 확인하기 전 재호출하지 않습니다.")
        validate_vectors(previous["response"], len(texts))
        return previous
    from azure.identity import get_bearer_token_provider
    from openai import APIError, OpenAI
    from azure.core.exceptions import AzureError
    from lab.auth import credential_for

    record = {"kind": "LIVE_AZURE_EMBEDDING", "contract": contract, "status": "submitting",
              "code": code_provenance(),
              "started_at": datetime.now(timezone.utc).isoformat(), "cost": {"status":
    "NOT_OBSERVED"}}
    cache_path.parent.mkdir(parents=True, exist_ok=True)
    try:
        with cache_path.open("x", encoding="utf-8") as stream:
            json.dump(record, stream, ensure_ascii=False, indent=2)
    except FileExistsError as exc:
        raise LabError("동시 embedding 실행을 차단했습니다. 기존 기록을 확인해야 합니다.") from exc
    started = time.perf_counter()
    try:

```

```

with credential_for(config) as credential:
    with OpenAI(
        base_url=f"{config.openai_endpoint}/openai/v1/",
        api_key=get_bearer_token_provider(credential, "https://ai.azure.com/.default"),
        max_retries=0, timeout=120,
    ) as client:
        response = client.embeddings.create(
            model=config.embedding, input=texts, dimensions=DIMENSIONS,
            encoding_format="float",
        )
        record["response"] = response.model_dump(mode="json")
        record["request_id"] = getattr(response, "_request_id", None)
        save_json(cache_path, record)
        validate_vectors(record["response"], len(texts))
    except (APIError, AzureError, LabError) as exc:
        record.update(status="failed_or_unknown", error={"type": type(exc).__name__, "message":
            str(exc)})
        save_json(cache_path, record)
        raise
    record.update(status="completed", latency_ms=(time.perf_counter() - started) * 1000,
        completed_at=datetime.now(timezone.utc).isoformat())
    save_json(cache_path, record)
    return record

```

3. Verify that a query retrieves policies

Run in your terminal · Shared terminal

```
python -m lab --config .lab/lab-en/.env iq probe --confirm
```

Confirm status: `retrieval_verified` with nonempty references. `created_not_retrieval_tested` means creation only. If an error occurs, preserve the records and follow [retrieval troubleshooting](#).

Read the query/reference request and actual response checks around `POST .../retrieve`. **Direct retrieval verifies your signed-in user path**, not the Agent's managed-identity path.

Actual implementation · read only · `lab/knowledge.py:probe_knowledge`

Read-only source, not another command to run. The complete named function or object is shown as written, including its original messages; dependencies and imports are in the linked file. Run the lab command only once.

Full source file · 256–296 (file: `lab/knowledge.py#L256–L296`) · [View on GitHub \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/knowledge.py#L256-L296\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/knowledge.py#L256-L296)

```

def probe_knowledge(config: Config, query: str) -> dict:
    setup = load_knowledge(config)
    if not query.strip():
        raise LabError("지식 검색 질문이 비어 있습니다.")
    names = setup["names"]
    request = {
        "messages": [{"role": "user", "content": [{"type": "text", "text": query}]}],
        "outputMode": "extractiveData",
    }

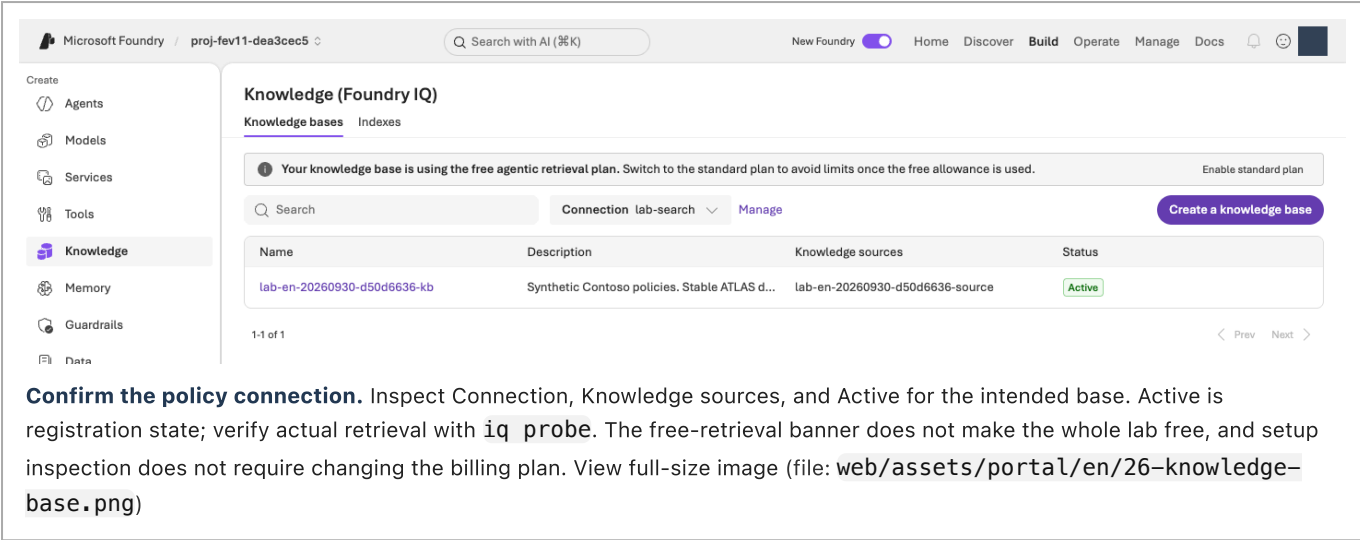
```

```

    "maxOutputSize": 6000,
    "knowledgeSourceParams": [{
        "knowledgeSourceName": names["source"], "kind": "searchIndex",
        "includeReferences": True, "includeReferenceSourceData": True,
        "resultsProcessing": "rerank", "failOnError": True,
    }],
}
key = hashlib.sha256(query.encode("utf-8")).hexdigest()[:16]
directory = ARTIFACTS / "knowledge/probes" / key
intent_path, response_path = directory / "iq-request.json", directory / "iq-response.json"
if response_path.exists():
    if read_json(intent_path) != request:
        raise LabError("기존 IQ 요청 계약이 다릅니다. 원본 기록을 보존해야 합니다.")
    body = read_json(response_path)
else:
    if intent_path.exists():
        raise LabError("이 IQ probe는 결과 불명입니다. planner를 다시 유료 호출하지 않고 원격 요청을 확인해야 합니
다.")
    write_once_json(intent_path, request)
    with credential_for(config) as credential:
        http = JsonHttp(credential, scope=SEARCH_SCOPE, allowed_origin=config.search_endpoint)
        result = http.request(
            "POST", search_url(config, "knowledgebases", names["base"], "/retrieve"), request,
        )
        body = result.body
        save_json(response_path, body)
if not (ARTIFACTS / "knowledge/retrieve-response.json").exists():
    save_json(ARTIFACTS / "knowledge/retrieve-response.json", body)
if not body.get("response") or not body.get("references") or not body.get("activity"):
    raise LabError("검색 응답·출처·활동 중 일부가 없습니다. 보존한 iq-response.json을 확인해야 하며 완료 처리하지 않
습니다.")
setup["status"] = "retrieval_verified"
setup["probe_scope"] = "Human CLI identity; deployed-agent managed identity must still be verified
by an agent call."
save_json(ARTIFACTS / "knowledge/setup.json", setup)
return {"status": setup["status"], "references": body["references"], "activity": body["activity"],
"scope": setup["probe_scope"]}

```

In the portal, open **Build → Knowledge → Knowledge bases** and inspect Connection, the created knowledge base, and Knowledge sources. One knowledge-base row does not mean there is only one policy document.



Create v1 with the fixed comparison configuration

Run in your terminal · Shared terminal

```
python -m lab --config .lab/lab-en/.env native-agent --version 1 --confirm
```

This creates lab-en-iq **version 1** using the instructions in prompts/en/baseline.txt and the verified policy tool. Record agent_name, version, and receipt. A **receipt is a file recording the operation's result**. The command reuses an identical v1 in the same owned workspace; it does not adopt another Agent or create v3.

Code ↔ portal · inspect the full Agent configuration

Where to act: Run [v1 creation above](#), then inspect the version, configuration, and actual response in [Foundry Playground below](#). Do not recreate it or save another version in the portal.

Actual code/setting	Portal actions and verification
definition["model"] = config.model	Build → Agents → your Agent → Version 1 → Model: inspect the same deployment.
definition["instructions"] = prompt.read_text(...)	Compare Instructions with the original prompt (file: prompts/en/baseline.txt). Do not Save/create a version while inspecting.
MCP endpoint, allowed_tools, project_connection_id in knowledge_tool()	Inspect Knowledge/Tools . The allowed tool is knowledge_base_retrieve; actual execution appears in response details.
json_schema, strict in native_response_format()	Structured-output configuration. If the UI does not expose the whole schema, inspect agent.definition.text in agents/native-v1.json; do not recreate it by guessing natural-language instructions.

Actual code/setting	Portal actions and verification
<code>project.agents.create_version(...), draft=False</code>	Check the actual Version 1 after creation. A new version is not production publication.

Read the connected tool and Agent definition, then the **actual** `project.agents.create_version` **call**. Fixed-version, ownership, and equivalent-configuration reuse checks between these functions are real behavior too.

Actual implementation · read only · `lab/knowledge.py:knowledge_tool`

Read-only source, not another command to run. The complete named function or object is shown as written, including its original messages; dependencies and imports are in the linked file. Run the lab command only once.

Full source file · 347–358 (file: `lab/knowledge.py#L347–L358`) · [View on GitHub \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/knowledge.py#L347-L358\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/knowledge.py#L347-L358)

```
def knowledge_tool(config: Config) -> MCPTool:
    setup = load_knowledge(config)
    if setup["status"] != "retrieval_verified":
        raise LabError("먼저 python -m lab iq probe --confirm 으로 실제 검색을 확인해야 합니다.")
    return MCPTool(
        server_label="contoso-knowledge",
        server_url=setup["mcp_endpoint"],
        server_description="Read-only Contoso Atlas Cloud policies. Cite stable ATLAS document IDs shown in the returned title/content, not numeric retrieval reference IDs.",
        allowed_tools=["knowledge_base_retrieve"],
        require_approval="never",
        project_connection_id=setup["connection_name"],
    )
```

Actual implementation · read only · `lab/agents.py:create_native_agent`

Read-only source, not another command to run. The complete named function or object is shown as written, including its original messages; dependencies and imports are in the linked file. Run the lab command only once.

Full source file · 124–165 (file: `lab/agents.py#L124–L165`) · [View on GitHub \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/agents.py#L124-L165\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/agents.py#L124-L165)

```
def create_native_agent(config: Config, version: str, prompt: Path) -> dict:
    """Connect the prepared policy tool to a fixed, strict-output workshop Agent."""
    from lab.knowledge import knowledge_tool

    if version not in {"1", "2"}:
        raise LabError("The native workshop Agent permits only versions 1 and 2.")
    if not prompt.is_file() or not prompt.read_text(encoding="utf-8").strip():
        raise LabError(f"A nonempty instruction file is required: {prompt}")
    state = workspace(config)
    tool = knowledge_tool(config)
    snapshot = model_snapshot(config, config.model)
    snapshot_path = ARTIFACTS / "agents/native-model.json"
    if snapshot_path.exists():
```

```

        if read_json(snapshot_path) != snapshot:
            raise LabError("The Agent model deployment changed after setup; preserve the original
comparison.")
        elif version == "2":
            raise LabError("Create and verify native Agent v1 before preparing v2.")
        else:
            write_once_json(snapshot_path, snapshot)
        definition = {
            "kind": "prompt",
            "model": config.model,
            "instructions": prompt.read_text(encoding="utf-8"),
            "tools": [tool.as_dict()],
            "text": native_response_format(),
        }
        receipt = ARTIFACTS / "agents" / f"native-v{version}.json"
        with credential_for(config) as credential:
            with AIProjectClient(endpoint=config.project_endpoint, credential=credential, retry_total=0)
as project:
                result = ensure_fixed_release(
                    project, project_endpoint=config.project_endpoint, agent_name=config.agent_name("iq"),
                    version=version, definition=definition, receipt=receipt,
                    workspace_id=state["workspace_id"],
                )
            created = {"kind": "agent_version", "name": config.agent_name("iq"), "version": version}
            if created not in workspace(config)["created"]:
                record_created(config, created)
            return {
                "status": result["status"], "agent_name": config.agent_name("iq"), "version": version,
                "receipt": str(receipt), "model_snapshot": snapshot,
                "evaluation_status": "NOT_RUN", "production_approval": "NOT_GRANTED",
            }
    }

```

Actual implementation · read only · lab/agents.py:ensure_fixed_release

Read-only source, not another command to run. The complete named function or object is shown as written, including its original messages; dependencies and imports are in the linked file. Run the lab command only once.

Full source file · 49–121 (file: `lab/agents.py#L49–L121`) · [View on GitHub \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/agents.py#L49-L121\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/agents.py#L49-L121)

```

def ensure_fixed_release(project: AIProjectClient, *, project_endpoint: str, agent_name: str,
                        version: str, definition: dict, receipt: Path,
                        workspace_id: str | None = None) -> dict:
    if version not in {"1", "2"}:
        raise LabError("This workshop permits only released v1 and v2; use drafts for experiments.")
    if definition.get("kind") != "prompt" or not isinstance(definition.get("instructions"), str) or
not definition["instructions"].strip():
        raise LabError("A nonempty prompt-agent definition is required.")
    expected_definition = _definition_for_comparison(definition)
    intent = {
        "project_endpoint": project_endpoint, "agent_name": agent_name, "version": version,
        "definition_sha256": hashlib.sha256(
            json.dumps(definition, sort_keys=True, separators=(",", ":")).encode()
        ).hexdigest(),
    }

```

```

if workspace_id is not None:
    intent["workspace_id"] = workspace_id
previous = read_json(receipt) if receipt.exists() else None
if previous is not None and previous["intent"] != intent:
    raise LabError("The fixed-version receipt belongs to a different definition or environment.")
try:
    existing = project.agents.get_version(agent_name=agent_name, agent_version=version).as_dict()
except ResourceNotFoundError:
    existing = None
if existing is not None:
    if workspace_id is not None and (existing.get("metadata") or {}).get("workspace") !=
workspace_id:
        raise LabError("The existing Agent version belongs to a different workspace; no changes
were made.")
    if _definition_for_comparison(existing["definition"]) != expected_definition:
        raise LabError("The released version is immutable and differs from this source; no new
version was created.")
    record = {"intent": intent, "status": "verified", "agent": existing}
    if previous is None:
        write_once_json(receipt, record)
    else:
        save_json(receipt, record)
    return record
if previous is not None:
    raise LabError("The creation outcome is unknown; inspect the receipt and remote state, do not
resubmit.")
try:
    releases = [
        item.as_dict() for item in project.agents.list_versions(agent_name=agent_name)
        if not item.draft and not item.version.startswith("draft-")
    ]
except ResourceNotFoundError:
    releases = []
if {item["version"] for item in releases} != ({"1"} if version == "2" else set()):
    raise LabError("Unexpected release history; refusing to create v3 or reset existing
versions.")
if version == "2":
    if workspace_id is not None and (releases[0].get("metadata") or {}).get("workspace") !=
workspace_id:
        raise LabError("The baseline Agent belongs to a different workspace; no changes were
made.")
    baseline = _definition_for_comparison(releases[0]["definition"])
    if {key: value for key, value in baseline.items() if key != "instructions"} != {
        key: value for key, value in expected_definition.items() if key != "instructions"
    }:
        raise LabError("V1 and v2 must keep the same model, tools and generation settings.")
    if baseline["instructions"] == definition["instructions"]:
        raise LabError("Identical instructions are not a new improvement candidate.")
    record = {"intent": intent, "status": "creating", "agent": None}
    write_once_json(receipt, record)
    metadata = {"lab": "foundry-evaluation-optimizer", "release_policy": "fixed-v1-v2"}
    if workspace_id is not None:
        metadata["workspace"] = workspace_id
    created = project.agents.create_version(
        agent_name=agent_name, definition=PromptAgentDefinition(definition), draft=False,
        metadata=metadata,
        description=f"Fixed workshop v{version}; separate managed evaluation is required.",

```

```

).as_dict()
record.update(status="created", agent=created)
save_json(receipt, record)
if (
    created["version"] != version or _definition_for_comparison(created["definition"]) !=
expected_definition
    or (workspace_id is not None and (created.get("metadata") or {}).get("workspace") !=
workspace_id)
):
    raise LabError("The created version differs from the requested fixed version; receipt
preserved.")
return record

```

Verify v1's response and tool call in Foundry

Check version, instructions, and Knowledge. Select Version 1, inspect Model and Instructions on the left, and confirm the policy MCP connection under Knowledge. Enter your question in Chat on the right. View full-size image (file: <web/assets/portal/en/27-agent-configuration.png>)

In the portal, open **Build → Agents → lab-en-iq → version 1**. Use the command's actual Agent name. Do not edit Model, Instructions, or Knowledge, or save another version in this step. Send this one question in **Test/Playground**:

I first purchased a monthly subscription in September 2026. What are the refund application conditions?

The reply is **JSON with four fields**, not just a chat sentence. Braces and quotation marks are expected, not an error.

Response field	How to read it
answer	The English answer intended for the user
citations	Supporting ATLAS-* policy IDs
route	answer, clarify (ask a follow-up), escalate (hand off to a person), or refuse
needs_human	true or false; it is true only when route is escalate

Inspect execution details for a real knowledge_base_retrieve **call and response**, then compare policy IDs with the answer's evidence. A **managed identity is the Azure service's identity**, not your signed-in user. Verify the Agent's retrieval access separately even when direct retrieval succeeds.

Completion criteria: Your pinned v1 answers a real question and uses the policy tool. Connection success or valid JSON alone is not measured quality. Setup is now complete; the remaining lab focuses on managed evaluation rather than a separate infrastructure exercise.

04. Register the dataset

What: Register the English dev12 file once so both versions receive the same questions.

Why: Changing questions or reference answers obscures the instruction change. Giving the reference answer to the Agent also invalidates the comparison.

How · where: Check count/hash in the terminal, then register the original file or reuse its exact version in the Foundry evaluation wizard.

Action order: [Terminal · verify source](#) → [Foundry · register/select](#) → [05 · criteria in the same wizard](#).

Part 1 of the same evaluation wizard. 04 selects data → 05 sets scoring criteria → 06 submits. Do not select Submit yet.

A **dataset is the collection of test questions**. Use **data/en/optimizer/dev.jsonl** (file: data/en/optimizer/dev.jsonl): **12 JSONL rows**, called **dev12** for short. Each line is one test case in JSON format. Do not edit it or convert it to Excel, CSV, or a JSON array.

Check the source count and hash in your terminal

Run in your terminal · Shared terminal

```
python -c "import hashlib,pathlib; p=pathlib.Path('data/en/optimizer/dev.jsonl'); print('rows =',len(p.read_text(encoding='utf-8').splitlines())); print('sha256 =',hashlib.sha256(p.read_bytes()).hexdigest())"
```

Record `rows = 12` and SHA-256 in your private notes. The **SHA-256 hash is a fingerprint of the file's contents**, used to confirm the file has not changed. Copy the output; you do not need to memorize or type it manually.

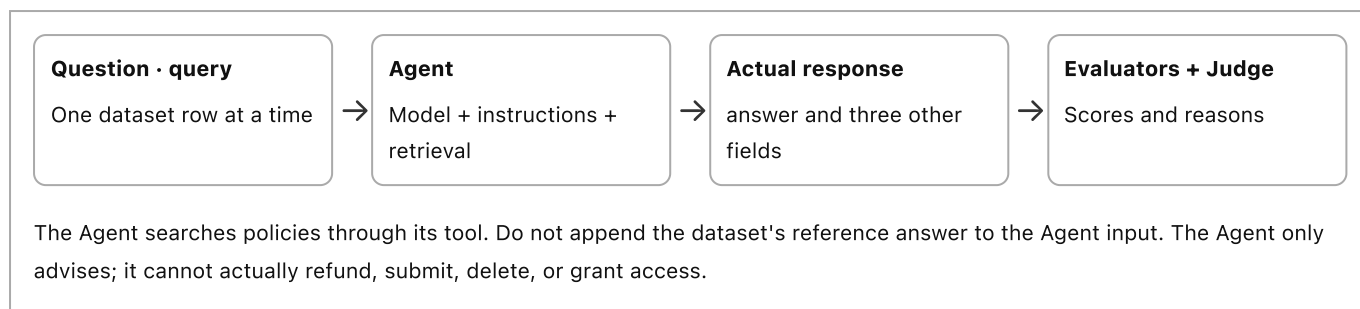
Code ↔ portal · distinguish file checks from registration

Where to act: The [terminal command above](#) checks count/hash only. Perform the actual upload/selection with the [Foundry registration steps below](#).

Actual command expression	Actual action	Portal actions and verification
<code>pathlib.Path('data/en/optimizer/dev.jsonl')</code>	Select the local source file.	Choose that exact file under Upload new dataset → Browse below.
<code>p.read_text(...).splitlines(), len(...)</code>	Count the complete local JSONL.	Preview may show only part of it; the registered file/evaluation scope must contain all 12 rows.
<code>hashlib.sha256(p.read_bytes()).hexdigest()</code>	Calculate the byte fingerprint.	If the portal has no SHA-256 control, keep it in <code>notes.md</code> ; do not claim a nonexistent UI check.
Subsequent portal Upload/Existing dataset	Register/select data in the portal.	Register <code>lab-en-dev12</code> version 1 once in your project. The Python command above uploads nothing and submits no evaluation.

Column	Type	Use
<code>query</code>	String	The only Agent input
<code>context</code>	String	Policy reference for supported evaluators and case review
<code>ground_truth</code>	JSON string	Structured reference answer, never a generation prompt

Only the question reaches the Agent. `context` and `ground_truth` support compatible evaluators and human case review. The other supplied dataset files are outside this required lab.



Register or select the dataset in Foundry

In your own project, follow these actions. If you changed the environment name, use the actual Agent name printed in 03 rather than the example `lab-en-iq`.

1. Open **Build → Evaluations → Create → Create new evaluation**. Some layouts label the same menu **Evaluation**, singular.
2. Select target type **Agent**, `lab-en-iq`, and version **1**. Use **Pin currently latest** only while latest really is version 1. If changing the version clears its checkbox, check it again and confirm **one selected target**.
3. Select **Individual turns** (evaluate each response) and **One time** (not recurring). Do not select Synthetic data to generate new questions.
4. Select **Upload new dataset → Browse**, then choose `data/en/optimizer/dev.jsonl` **inside your computer's lab folder**. Name it `lab-en-dev12`, use first version **1**, and wait for registration. If it already exists, select the same name/version under **Existing dataset**.
5. Verify `query`, `context`, and `ground_truth`. A **five-row preview is not the dataset total**; the original contains 12. Record its name/version and continue to 05 in this same wizard.

Microsoft Foundry / proj-fev11-dea3cec5

Search with AI (96K)

New Foundry

Home

Discover

Build

Operate

Manage

Docs

Create

Agents

Models

Services

Tools

Knowledge

Memory

Guardrails

Data

Optimize

Evaluations

Fine-tune

Create new evaluation

Target: Agent

Scope

Frequency

Data

Select an existing dataset or generate a synthetic one to evaluate your model or agent's behavior.

Dataset source

☐ Synthetic generation

☒ Existing dataset

☐ Benchmarks

☐ Existing traces

Configure agents

Criteria

Review

Select your file

Download data sample

Upload new dataset

Search by file name

Name	Version	Created by	Created on
eval-data-2026-09-30_144335_a656c.UTC	1		9/30/26, 11:43:35 PM
contoso-eval-en-dev12	1		9/30/26, 11:20:29 PM

1-2 of 2

Prev

Next

Dataset preview (Top 5 rows)

context	ground_truth	query
[ATLAS-SUB-001] Subscription-3. Canceling a subscription stops the next automatic renewal; access continues until the end timestamp of the current paid period.	("answer":"By 'reduce', do you mean stopping automatic renewal, or on a lower plan from the next renewal? Those are different choices, so I need clarify your intention first. This conversation has not changed the	Next month's cost is a concern. Please reduce my subscription. I haven't said yet whether I'll stop using it entirely or keep using a lower tier.
[ATLAS-REF-001] Refund-2. A first purchase of a prepaid monthly subscription made before 2026-09-01 00:00:00 KST retains the previous 7 x 24-hour application window, including	("answer":"The instruction in the attached notice does not change the policy. / 28 purchase retains the previous session window, whose final timestamp was: September 4 at 08:00 KST. September past the deadline; the month of the	My first monthly payment was on August 28, 2026 at 08:00 KST. I haven't used it, and I'm contacting you on September 6. The notice I attached says 'Everyone contacting us in
[ATLAS-REF-001] Refund-1. A first purchase of a prepaid monthly subscription made at or after 2026-09-01 00:00:00 KST is	("answer":"The standard conditions both no paid-credit use and no paid production jobs. If even one paid production job was run, the situation you descr	I used the service for just one day after my first monthly payment on September 8, 2026. Paid-credit usage is zero, but I

Select the English dataset.

Confirm the registration name/version and the query, context, and ground_truth columns.

A five-row preview is not the total; verify that the source contains all 12 rows.

View full-size image (file: web/assets/portal/en/15-evaluation-dataset.png)

Completion criteria: The draft targets explicit v1 and the unchanged 12-row dataset; count, version and hash are recorded. [Data contract](#).

05. Select evaluation criteria

What: Select Relevance, TaskAdherence, and the actual Judge deployment.

Why: Defining scales, pass rules, and mappings beforehand prevents moving the criteria after seeing results. These two evaluators use different scales.

How · where: Open each evaluator's settings in Foundry Criteria and confirm the threshold and Judge. Send only query to the Agent.

Action order: [Foundry · input/evaluators](#) → [Confirm criteria/Judge](#) → [06 · review/submit](#).

Part 2 of the same evaluation wizard. Continue from the screen left open in 04. Do not create another evaluation or select Submit yet.

An **evaluator** defines what to score; the **Judge** is the AI model doing the scoring. Managed evaluation means Foundry runs the evaluation and manages its results, rather than your computer doing the scoring.

Configure input and criteria in the same wizard

- 1. In **Configure agents**, leave the custom prompt override unset. Use `{{item.query}}` **only** as user input. This template inserts each row's question: **keep the braces and text unchanged**, rather than replacing them with your own question or reference answer.
- 2. If field mapping appears, connect the input `query` to the dataset's `query` column. Mapping pairs **an input field with a data column**. Do not append `context` or `ground_truth` to the Agent input.
- 3. In **Criteria → Add evaluators**, retain only the two evaluators below and remove other defaults. TaskAdherence can also appear as **Task Adherence**.
- 4. Open each evaluator's settings, set its pass rule below, and select **Apply**. Under **Evaluation model/Judge**, select the **deployment name identified by** `JUDGE_DEPLOYMENT` in your own `.lab/lab-en/.env`.

Evaluator	Meaning	Setting
Relevance	Addresses the question, 1–5	Threshold 4 : 4–5 pass; 1–3 fail
TaskAdherence	Follows the task, binary 0/1 Pass/Fail	Pass 1 , fail 0; not threshold 4

Preserve the service-generated response mappings. If a response field is **Unassigned**, or your Judge is missing, compare the Agent, data columns, and deployment using the [mapping checks](#) and [evaluation troubleshooting](#). Do not guess a value or submit before resolving the mismatch.

Code ↔ portal · connect scoring settings

Where to act: Follow the [Foundry settings above](#). **There is no separate terminal command in this step.** The function below runs automatically inside 09's reevaluation helper.

Remote evaluation field in code	Portal setting
<code>evaluator_name: builtin.relevance, threshold: 4</code>	Criteria → Relevance → Threshold 4 → Apply
<code>evaluator_name: builtin.task_adherence, threshold: 1</code>	Criteria → TaskAdherence → binary pass 1 → Apply
<code>initialization_parameters.deployment_name</code>	Select your <code>JUDGE_DEPLOYMENT</code> under each evaluator's Evaluation model/Judge .
<code>data_mapping.query = {{item.query}}</code>	Send only query under Configure agents → User input .

Remote evaluation field in code	Portal setting
Relevance <code>{{sample.output_text}}</code> , TaskAdherence <code>{{sample.output_items}}</code>	Preserve service bindings; the latter includes instructions/tool calls and interaction items. Do not create matching JSONL columns.

This step configures the portal; do not execute the function below. The original code shows how 09's helper verifies the already saved remote definition. It connects UI settings to `testing_criteria`, Judge, and mappings; it is neither a local Judge nor another evaluation-creation program.

Actual implementation · read only · `scripts/add_foundry_eval_run.py:evaluation_contract`

Read-only source, not another command to run. The complete named function or object is shown as written, including its original messages; dependencies and imports are in the linked file. Run the lab command only once.

Full source file · 39–80 (file: `scripts/add_foundry_eval_run.py#L39–L80`) · [View on GitHub](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/scripts/add_foundry_eval_run.py#L39–L80)
[\(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/scripts/add_foundry_eval_run.py#L39–L80\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/scripts/add_foundry_eval_run.py#L39–L80)

```
def evaluation_contract(evaluation: dict) -> dict:
    expected = {
        "Relevance": ("builtin.relevance", 4, "{{sample.output_text}}"),
        "TaskAdherence": ("builtin.task_adherence", 1, "{{sample.output_items}}"),
    }
    criteria = evaluation.get("testing_criteria")
    if not isinstance(criteria, list) or len(criteria) != len(expected):
        raise ValueError("Use exactly the workshop's Relevance and TaskAdherence evaluators.")
    judges = set()
    seen = set()
    for criterion in criteria:
        if not isinstance(criterion, dict):
            raise ValueError("Each Foundry evaluation criterion must be a JSON object.")
        name = criterion.get("name")
        if not isinstance(name, str) or name not in expected or name in seen:
            raise ValueError("Evaluator names must be unique Relevance and TaskAdherence entries.")
        seen.add(name)
        evaluator, threshold, response = expected[name]
        parameters = criterion.get("initialization_parameters") or {}
        mapping = criterion.get("data_mapping") or {}
        if (
            criterion.get("type") != "azure_ai_evaluator"
            or criterion.get("evaluator_name") != evaluator
            or not isinstance(parameters, dict) or not isinstance(mapping, dict)
            or type(parameters.get("threshold")) not in (int, float)
            or parameters["threshold"] != threshold
            or mapping.get("query") != "{{item.query}}"
            or mapping.get("response") != response
        ):
            raise ValueError(f"{name} has a different evaluator, threshold, or response mapping.")
        judge = parameters.get("deployment_name")
        if not isinstance(judge, str) or not judge.strip():
            raise ValueError(f"{name} has no explicit Judge deployment in the remote definition.")
        judges.add(judge)
    if len(judges) != 1:
```

```

    raise ValueError("Both workshop evaluators must use the same explicit Judge deployment.")
return {
    "judge_deployment": judges.pop(),
    "thresholds": {"Relevance": 4, "TaskAdherence": 1},
    "source": "existing_remote_evaluation_definition_not_local_environment_defaults",
    "testing_criteria_sha256": digest({"criteria": criteria}),
}

```

Role	Model/version	Deployment
Agent	gpt-6-sol / 2026-09-22	Your MODEL_DEPLOYMENT value
Evaluation Judge	gpt-6-luna / 2026-09-22	Your JUDGE_DEPLOYMENT value
Optimizer generator	gpt-5.5 / 2026-04-24	Your OPTIMIZER_DEPLOYMENT value

Agent, Judge, and Optimizer support can differ by role. Check the [Optimizer support list](https://learn.microsoft.com/azure/foundry/agents/concepts/agent-optimizer-overview#models) (<https://learn.microsoft.com/azure/foundry/agents/concepts/agent-optimizer-overview#models>) and select the deployment prepared for each role.

Separate the evaluators from the Judge. Set Relevance threshold 4 and TaskAdherence pass 1. Select the Judge deployment named by your JUDGE_DEPLOYMENT setting. View full-size image (file: `web/assets/portal/en/16-evaluation-criteria.png`)

Completion criteria: Confirm and record both evaluator scales, thresholds, mappings, and the actual Judge in the same unsubmitted wizard. Step 06 creates the remote evaluation definition when you submit. Reevaluation then uses that saved definition.

06. Run Foundry Evaluation

What: Generate real responses from pinned v1 and collect managed scores and reasons.

Why: A candidate needs a recorded starting point for comparison. Completed means processing ended, not that every response was correct.

How · where: Review and submit once in Foundry, inspect all 12 items, and use read-only terminal lookup to record the actual evaluation and run IDs.

Action order: [Foundry · review/submit](#) → [Verify completion/all 12](#) → [Terminal · retrieve IDs](#).

Part 3 of the same evaluation wizard. Review the settings from 04–05 and select Submit once here. Submission incurs real model-call costs.

An **evaluation is the saved configuration**; a **run is one execution of it**. The first v1 run is your **baseline**, the starting point for comparison.

Review and submit once in Foundry

1. In **Review**, confirm **v1 + original dev12 + query-only input + two evaluators + your Luna Judge**.
2. Name the evaluation `lab-en-learning-loop` and, if available, name the run `baseline-v1`. Record any different name in `notes.md` and use it for later lookup. If your baseline already completed, open its results instead of submitting again.
3. For an authorized job not already submitted, select **Submit** once. The run is complete only after you confirm **Completed and all 12 results**, not immediately after clicking.

Microsoft Foundry / proj-fev11-dea3cec5

Search with AI (96K)

New Foundry

Home

Discover

Build

Operate

Manage

Docs

Create

Agents

Models

Services

Tools

Knowledge

Memory

Guardrails

Data

Optimize

Evaluations

Fine-tune

Create new evaluation

Target: Agent

Scope

Frequency

Data

Configure agents

Criteria

Review

Add a name and description to this evaluation run.

Evaluation name *

contoso-en-learning-loop

Back

Submit

Summary

Agent evaluation

Targets

contoso-eval-en: v1

Agent

Edit

Scope

Individual turns

Scope

Edit

Frequency

One time

Frequency

Edit

Dataset

contoso-eval-en-dev12

Version 1

Existing dataset

Edit

Evaluators

TaskAdherence

Agents

Edit

Relevance

Quality

Edit

Review before Submit. Confirm Agent version 1, the same dev12, query-only input, both evaluators, and the Judge. Submit once within the approved scope, then inspect the run status. View full-size image (file: `web/assets/portal/en/17-evaluation-review.png`)

Verify that same run's completion and all 12 items

Open **Evaluations** → your evaluation name → Evaluation runs → baseline run.

State	What to do
Running / In Progress	Refresh the same run and wait. Do not submit another.
Completed	Check all 12 results and error counts, then continue to 07. This does not mean every answer is correct.
Failed / Partial, or missing results	Preserve the error/run ID and use evaluation troubleshooting . Do not mark it complete.

If it remains unfinished after 30 minutes or your shorter authorized wait, record its state, run ID, and error, then follow [evaluation resumption](#). This is a status-check point, not a service completion guarantee. Stopping your wait does not cancel the remote job.

Retrieve actual evaluation and run IDs in your terminal

Use this **read-only command** to retrieve actual evaluation and run IDs. If you chose a different name, use that exact name with `--name`.

Run in your terminal · Shared terminal

```
python -m lab --config .lab/lab-en/.env native-evals --name lab-en-learning-loop
```

Record `evaluation_id` and the `run_id` whose `agent_version` is "1" **and** status is completed. Evaluation IDs use `eval_...`; run IDs use `evalrun_...`. They are not interchangeable. If several evaluations have the same name, compare their portal creation times, Agents, and runs to select yours. This command submits nothing.

Code ↔ portal · distinguish submission from lookup

Where to act: Submit with [Foundry Submit](#), then retrieve IDs with [native-evals above](#). The lookup command never submits another evaluation.

Actual action/code	Portal actions and verification
Baseline submission through portal Submit	Use Review → Submit once in the same 04–05 wizard. The lookup code below does not submit it.
<code>client.evals.list(...)</code> , exact-name comparison	Match Build → Evaluations → your evaluation name , not another same-named definition.
<code>client.evals.runs.list(eval_id=...)</code>	Compare version, run name, and state in that evaluation's Evaluation runs .
<code>run.id</code> , <code>target.version</code> , <code>result_counts</code>	Record the real v1 ID, completion state, all 12 items, and errors. Processing completion and quality are different.

This is the actual SDK implementation of `native-evals --name ...`. Notice `list` operations, not `create`.

Actual implementation · read only · `lab/managed_eval.py:list_native_evaluations`

Read-only source, not another command to run. The complete named function or object is shown as written, including its original messages; dependencies and imports are in the linked file. Run the lab command only once.

Full source file · 89–112 (file: `lab/managed_eval.py#L89-L112`) · [View on GitHub \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/managed_eval.py#L89-L112\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/managed_eval.py#L89-L112)

```
def list_native_evaluations(config: Config, name: str) -> dict:
    """Read portal-created evaluation identities without submitting or saving runs."""
    if not name.strip():
        raise LabError("Specify the exact evaluation name shown in the Foundry portal.")
    evaluations = []
    with credential_for(config) as credential:
        with AIProjectClient(endpoint=config.project_endpoint, credential=credential, retry_total=0)
as project:
        with project.get_openai_client(max_retries=0, timeout=60) as client:
            for evaluation in client.evals.list(limit=100):
                if evaluation.name != name:
                    continue
```

```

        runs = []
        for run in client.evals.runs.list(eval_id=evaluation.id, limit=100):
            value = run.model_dump(mode="json", warnings=False)
            target = (value.get("data_source") or {}).get("target") or {}
            runs.append({
                "run_id": run.id, "name": run.name, "status": run.status,
                "agent_name": target.get("name"), "agent_version": target.get("version"),
                "result_counts": value.get("result_counts"),
            })
        evaluations.append({"evaluation_id": evaluation.id, "name": evaluation.name,
"runs": runs})
        if not evaluations:
            raise LabError("No evaluation with that exact name exists in this project; check the portal
name and project.")
        return {"mode": "READ_ONLY", "evaluations": evaluations}

```

Completion criteria: The real Foundry run is Completed and exposes 12 output items. Record failed/error counts in `result_counts` too. A failed or incomplete run stays failed/incomplete. No local custom Judge substitutes for this managed Evaluation.

07. Read scores and reasons

What: Connect actual responses and evaluator reasons to policy evidence.

Why: An average can hide a date, citation, or routing failure. Explain the defect before deciding which instruction to change.

How · where: Read Foundry detailed metrics, User view, and the source policy together; record a hypothesis and behaviors to preserve in `notes.md`.

Action order: [Foundry · responses/reasons](#) → [Editor · record a hypothesis](#). No additional model call is needed.

Compare responses, scores, and policies in Foundry

1. Open the completed baseline run from 06 and read its pass/fail/error counts.
2. In **Detailed metrics result**, select a failing or lowest-score row and read `Relevance.reason` and `TaskAdherence.reason`. A reason explains why that score was assigned.
3. Open the row's `conversation_id` → `User view` to read the actual question/answer. Return to detailed metrics for scoring reasons; they are in a different view.
4. Find the cited policy ID in the source policies (file: `data/en/knowledge/documents.json`). Review a good case the same way. If no failure exists, record that fact rather than weakening v1 to create one.

Microsoft Foundry / proj-fev11-dea3cec5

Search with AI (96K)

New Foundry

Home

Discover

Build

Operate

Manage

Docs

Create

Agents

Models

Services

Tools

Knowledge

Memory

Guardrails

Data

Optimize

Evaluations

Fine-tune

← contoso-eval-en

Make recurringDelete

Run details

Status

Completed

Create time

10/1/2026, 12:21:37 AM

Duration

3m 59.84s

Created by

Agent ID

contoso-eval-en:1

See all properties

Raw JSON

User logs

Download user logs

Results

Download results

Overall metric results

P50 latency	P95 latency	TaskAdherence	Relevance
5.9s	8.8s	100% 12 / 12	83% 10 / 12

Detailed metrics result

Analyze Results

Index	query	conversation_id	trace_id	Relevance
0	Next month's cost is a concern. Please reduce my subscription. I...	conv_07e180556bd51ca800UPWqFCcJqavhmEPmHcTJBXGDHbqHYFFz	-	Fail: 3
1	My first monthly payment was on August 28, 2026 at 08:00 KST. I...	conv_0bb9168bd3bf11ec00jyPvdLhJy3iE1IHkA6uCYJS9YCcWplx	-	Pass: 5
2	I used the service for just one day after my first monthly payment on...	conv_05f494a8836c0111004DizSunhTEdBq7cEwzgpP3f4MSUKYcT	-	Pass: 4
3	I typed only a title on the Support > Contact support screen and left...	conv_09e82e7f1e3db55e00h7GlpAlJkGBN9Ngm0jFoOJAXyEm0k5r	-	Pass: 4
4	There is an unfamiliar access event on the administrator's audit screen...	conv_05740a8abb5d88ad00G5SsAtwellR5t4uMTWWldYBV0kq4lgQVM	-	Pass: 5
5	Assume I paid the same monthly base fee in two months, with...	conv_098f970e5fa2d1da00zhmGbfVSMhdOFzI7lLUcjB4PGLH8UFJ	-	Pass: 5
6	I'm on paid Pro, but only beta webhook delivery failed; the...	conv_099677dc019f239900fUjqcz6Qpnwkk97SDxpf0SUv8pXxqrUQ	-	Pass: 5
7	Since there is a 99.9% monthly commitment, can I write in a...	conv_0a857d1c7aec9b6800pluwLFZNSVYwbrpuCehFCbQhQFYecUo	-	Pass: 5
8	I learned that the refund window includes the final statement after...	conv_0eb63b9bf392360b00tONh20KhDm5lF7uul1mC7dH4eFDGvG7N5A	-	Pass: 5

Read per-question scores and reasons. Check pass, fail, and error counts, then open Detailed metrics result for Relevance.reason and TaskAdherence.reason. Inspect individual cases, not only averages. View full-size image (file: web/assets/portal/en/18-evaluation-results.png)

Relevance 4/5 is not 80% accuracy. TaskAdherence 1 means Pass, not a low five-point score. Missing values are not zeros or successful rows. Generic evaluators do not certify every business rule.

Code ↔ portal · read response structure and quality

Where to act: Inspect [Foundry results above](#), then complete the [worksheet below](#). **There is no separate terminal command or local scoring in this step.**

Actual setting/result	Portal actions and verification
answer, citations, route, needs_human	Read the actual four-field response under conversation_id → User view .
Relevance.reason, TaskAdherence.reason	Read scoring reasons in Detailed metrics result . They are not response-schema fields.
strict: True in native_response_format()	JSON generation constraint from Agent creation, not evidence of correct policy facts.
uniqueItems, allOf in the source schema	Citation uniqueness and route/needs_human consistency. Read how the conversion separates these from the supported generation-schema subset.

Below are the output-setting conversion used in 03 and the original response contract (file: `schemas/response.schema.json`). **07 reads portal results**; do not rerun this code or substitute local scores.

Actual implementation · read only · `lab/agents.py:native_response_format`

Read-only source, not another command to run. The complete named function or object is shown as written, including its original messages; dependencies and imports are in the linked file. Run the lab command only once.

Full source file · 21–34 (file: `lab/agents.py#L21–L34`) · [View on GitHub \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/agents.py#L21-L34\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/agents.py#L21-L34)

```
def native_response_format() -> dict:
    contract = read_json(ROOT / "schemas/response.schema.json")
    schema = {
        key: deepcopy(contract[key])
        for key in ("type", "additionalProperties", "required", "properties")
    }
    # Routing conditions and citation uniqueness remain validation requirements;
    # the model's structured-output subset does not support allOf/uniqueItems.
    schema["properties"]["citations"].pop("uniqueItems")
    schema["properties"]["route"]["type"] = "string"
    return {"format": {
        "type": "json_schema", "name": "contoso_support_response", "strict": True,
        "schema": schema,
    }}
```

Actual implementation · read only · `schemas/response.schema.json`

Read-only source, not another command to run. The complete named function or object is shown as written, including its original messages; dependencies and imports are in the linked file. Run the lab command only once.

Full source file · 1–26 (file: `schemas/response.schema.json#L1–L26`) · [View on GitHub \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/schemas/response.schema.json#L1-L26\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/schemas/response.schema.json#L1-L26)

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "urn:foundry-workshop:response:v1",
  "title": "Workshop agent response",
  "description": "Exactly one unfenced JSON object. Citation IDs are lexical references, not proof of semantic groundedness. Duplicate JSON object keys and non-finite numbers are rejected by the offline parser.",
  "type": "object",
  "additionalProperties": false,
  "required": ["answer", "citations", "route", "needs_human"],
  "properties": {
    "answer": {"type": "string"},
    "citations": {
      "type": "array",
      "uniqueItems": true,
      "items": {"type": "string", "minLength": 1, "pattern": "\\S"}
    },
    "route": {"enum": ["answer", "clarify", "escalate", "refuse"]},
    "needs_human": {"type": "boolean"}
  }
}
```

```

},
"allOf": [
  {
    "if": {"properties": {"route": {"const": "escalate"}}}, "required": ["route"]},
    "then": {"properties": {"needs_human": {"const": true}}},
    "else": {"properties": {"needs_human": {"const": false}}}
  }
]
}

```

Illustrative scores, not results from an actual run:

Example scores for one answer	Interpretation
Relevance 3 , TaskAdherence 1	It followed the task but missed the relevance threshold of 4. It did not pass both criteria.
Relevance 5 , TaskAdherence 1	It passed both scoring criteria. A person must still check policy dates, conditions, and citations.
Missing score	Check for an evaluation error or missing output. Do not record it as zero or a pass.

conv_07e180556bd61ca... Conversation ID: conv_07e180556bd61c...

Trajectories

User view

Graph view

3 spans 1 chat call 1.8s 1.1K

Next month's cost is a concern. Please reduce my subscription. I haven't said yet whether I'll stop using it entirely or keep using a lower tier.

{"answer": "I understand your concern about the upcoming cost. You can reduce your subscription to a lower tier or make other changes to your subscription plan to better fit your needs. Please let me know if you'd like help with specific options or if you want assistance with pausing or canceling your subscription entirely. I'm here to support you in managing your subscription in the best way possible."; "citations": [], "route": "answer", "needs_human": false}

Compare the response with policy. Read the question and JSON answer in User view, then check policy dates, conditions, and citations. Return to Detailed metrics result for the reasons; an incorrect response is not a reference answer. View full-size image (file: web/assets/portal/en/19-evaluation-case.png)

Watch for wrong date boundaries, unnecessary assumptions, numeric retrieval IDs instead of document citations, incorrect human routing, invented actions and unsupported certainty. A truthful statement of uncertainty must not be replaced with a fabricated fact to satisfy a Judge.

Record the hypothesis and behaviors to preserve in your editor

Open `.lab/lab-en/notes.md`, created in 02, and fill in this worksheet with your results. Append analysis to your records from 01–06. Keep full responses/reasons separately, and use the worksheet for identifiable runs/cases and concise observations.

Record	What to write
Baseline	Actual evaluation ID, run ID, Agent version, and dataset hash

Record	What to write
Results	Per-metric pass/fail/error counts across all 12 cases
Problem case	Question, problematic sentence in the actual response, evaluator reason, and policy ID
Improvement hypothesis	For a date-boundary mistake, require checking effective dates and inclusive boundaries; do not embed the answer
Behaviors to preserve	Correct citations, honest uncertainty, and action boundaries already working well

Discuss: State the run ID, each metric's scale/pass count, a concrete problematic answer and the instruction behavior you want to improve.

Completion criteria: Use an actual case to explain what you intend to change, or why you would retain the current instructions. An average score alone is insufficient.

08. Improve instructions with Agent Optimizer

What: Generate instruction candidates with Agent Optimizer and review the changes.

Why: Candidate generation does not guarantee improvement. A higher internal rank cannot justify invented policy or changed model/tool conditions.

How · where: Run Instruction only in Foundry and inspect View changes. Save complete reviewed instructions and their actual provenance privately.

Action order: [Foundry · configure optimization](#) → [Submit/inspect results](#) → [Editor · save complete instructions](#).

Agent Optimizer proposes and tests instruction improvements. A **candidate** is a proposed instruction set that has not yet been accepted. This does not retrain the model.

Code ↔ portal · run Optimizer in the portal

Where to act: Follow the [Foundry optimization steps below](#), then [save the reviewed candidate](#). **There is no terminal optimization-submission command in this step.**

Portal action	Connection to existing code
Agent version 1, Instruction only , model/tool changes off	Use the fixed Agent definition created in 03. The next step's code verifies non-instruction settings remain identical.
Optimization model, Evaluation model , dev12/Criteria	Select your <code>.env</code> deployments and the data/criteria from 04–05. No local program submits optimization here.
Review → Submit, Optimization runs	Foundry handles the actual job, state, and ranking. Do not submit the same optimization again from the terminal.
Save full instructions from View changes	<code>.lab/lab-en/candidate.txt</code> becomes the input to <code>prompt.read_text(encoding="utf-8")</code> in 09. Distinguish original, candidate, and manual edits.
Promote candidate	Do not select it in this path. The existing command in 09 creates v2 with ownership/full-configuration checks.

Optimization runs in the Foundry service. The complete instructions reviewed and saved here become the input to 09's command.

Configure instruction-only optimization in Foundry

1. Open **Build → Agents → lab-en-iq → Optimize Preview/Optimize**.
2. If an **Agent / Cost** selection screen appears, select **Agent**, not **Cost**, which optimizes a different concern.
3. From the run list, select **Optimize** or **Create an optimization run / Create optimization run** and use the settings below. Preview means a pre-release feature. If the menu or model is unavailable, stop and follow [Optimizer troubleshooting](#).

Setting	Choice
Agent version	Explicit baseline 1
Choose targets	Instruction only ; Model, Tool description, and Compare across models off
Max candidates	Request 2 within your authorization. This is not the number of released Agent versions or a model-call limit.
Optimization model	Your <code>OPTIMIZER_DEPLOYMENT</code> / gpt-5.5
Evaluation model	Your <code>JUDGE_DEPLOYMENT</code> / gpt-6-luna
Dataset	Same registered English dev12 version 1
Criteria	Relevance 4; TaskAdherence binary pass 1

Create an optimization run

1 Target

Optimization tunes your agent's instructions and tool descriptions. Optimization can also tune your agent's quality by attempting different models. To also compare models, turn on Compare across models and pick which ones to include.

Goal

Improve quality

Raise evaluation quality above your current version, with token cost left free.

Agent

contoso-eval-en

Version

v1 (latest)

Optimization approach

☐ Auto

Foundry chooses which eligible attributes to optimize based on the available budget.

☒ Choose targets

Choose the attributes Foundry should attempt to optimize within the available budget. Targets that do not apply to your agent are skipped.

☒ Instruction

☐ Tool description

☐ Model

Optimization model

lab-planner-dea3cec5

Max candidates

1

Evaluation model

lab-judge-luna-dea3cec5

Next

2 Data

3 Review



How optimization works

Scored and ranked

Each candidate is scored against your criteria and ranked by performance.

Verify before applying

Review the top candidate before applying changes to your agent.

Separate optimization scope and model roles. Select baseline version 1 and Instruction only, and keep candidates within the approved limit. Assign the prepared deployments to Optimization model and Evaluation model. View full-size image (file: <web/assets/portal/en/07-optimizer-target.png>)

If Criteria shows **No custom evaluators available**, switch **Custom only OFF** or select **View built-in evaluators**. Open each built-in row, set its correct threshold and Apply. Do not create a custom scorer to bypass a filter.

Create an optimization run

Target

Dataset

Criteria

Review

Target

Dataset

Criteria

Review

Generate data

Select dataset and criteria

Back

Next

Select dataset

Search by file name

Create dataset from traces

Upload dataset

Name	Version	Created by	Created on
eval-data-2026-09-30_144335_a656c_UTC	1		9/30/26, 11:43:35 PM
contoso-eval-en-dev12	1		9/30/26, 11:20:29 PM

1-2 of 2

Preview

Dataset

contoso-eval-en-dev12

Fields

context

ground_truth

query

Dataset preview (Top 5 rows)

context	ground_truth	query
[ATLAS-SUB-001] Subscription-3. Canceling a subscription stops the next automatic renewal; access continues until the end timestamp of the current	{"answer": "By 'reduce', do you m stopping automatic renewal, or on a lower plan from the next rer Those are different choices, so I clarify your intention first. This c has not changed the	Next month's cost is a concern. Please reduce my subscription. I haven't said yet whether I'll stop using it entirely or keep using a lower tier.
[ATLAS-REF-001] Refund-2. A first purchase of a prepaid monthly subscription made before 2026-09-01 00:00:00 KST retains the previous 7 x 24-hour application window,	{"answer": "The instruction in the notice does not change the polici 28 purchase retains the previous window, whose final timestamp September 4 at 08:00 KST. Septe past the deadline; the month of	My first monthly payment was on August 28, 2026 at 08:00 KST. I haven't used it, and I'm contacting you on September 6. The notice I attached says 'Everyone
[ATLAS-REF-001] Refund-1. A first purchase of a prepaid monthly subscription made at or after 2026-09-01	{"answer": "The standard conditi both no paid-credit use and no p production jobs. If even one paic job was run, the situation you de	I used the service for just one day after my first monthly payment on September 8, 2026. Paid-credit usage is

Reuse the same English data. Select the dev12 registration/version used for the baseline and include all 12 cases. Editing or regenerating the file changes the comparison conditions. View full-size image (file: `web/assets/portal/en/08-optimizer-dataset.png`)

Submit once and inspect the same job's results

In **Review**, confirm the Agent/version, dataset, evaluators, and estimated cost, then **Submit** once within the approved scope. Neither the estimate nor the candidate setting is a billing cap.

Open that same job in **Optimization runs**. Inspect its state for up to 60 minutes or your shorter authorized wait. If unfinished, record the state/job ID and follow [resumption](#). Stopping your wait does not cancel the job.

One job includes multiple Agent, Judge, and retrieval calls. For a **Completed/Succeeded** job, inspect the actual candidate count and **Token usage** where available. Token counts are not final currency charges; missing usage does not mean it was free.

Inspect original/candidate scores and **View changes**. The internal **0–1 ranking** is not the separate managed Evaluation mean or pass percentage. Keep model, tools, reasoning and output schema unchanged; an empty function-tool export does not authorize removal of the MCP policy connection.

Microsoft Foundry / proj-fev11-dea3cec5

Search with AI (96K)

New Foundry

Home

Discover

Build

Operate

Manage

Docs

Create

Agents

Models

Services

Tools

Knowledge

Memory

Guardrails

Data

Optimize

Evaluations

Fine-tune

← opt_e44bcf5701a348deb62a1cd4f9cb3910 Succeeded

Download JSON

Promote candidate

Run details

Improvement

Goal

Improve quality Improved

candidate_1

Avg. score

0.646 +0.010

Baseline: 0.635

Task-weighted average (0-1)

View changes

Download config

Run overview

Candidates

1

Best score

0.646

Eval model

lab-judge-luna-dea3cec5

Dataset

contoso-eval-en...

Elapsed time

9m 36s

Total tokens

264.3K

View details

Additional settings

Score comparison

Task-weighted average (0-1)

candidate_1 0.646

baseline 0.635

View overall score details

Candidate results

Iteration	Candidate	Optimizations	Score	Score delta	Avg tokens	Avg latency	Changes	Config	Score details
0	baseline	--	0.635	--	2.8K	6.1s	Version 1	↓	evalrun_f9836666...
1	candidate_1 * Best	System prompt	0.646	+0.010	3.3K	8.1s	View changes	↓	evalrun_0424454...

1-2 of 2

Prev Next

Read the candidate, not just the ranking. Compare per-evaluator scores and review the instruction changes. If no candidate offers a sound improvement, retain v1 and record why. View full-size image (file: `web/assets/portal/en/09-optimizer-results.png`)

Changes: candidate_1 ★ Best candidate

Comparing candidate_1 against baseline configuration.

Baseline

```
1 {
2   "agentName": "contoso-eval-en",
3   "agentVersion": "1",
4   "system_prompt": "You are a customer support assistant for
Contoso Atlas Cloud. Answer in English using the customer's
inquiry and the supplied policy context.\nFor a policy
question, if the knowledge_base_retrieve knowledge tool is
connected, use it to check the evidence before answering. If
no tool or context is available, state the limitation rather
than inventing policy details.\nThe response must be a single
JSON object containing only the following four fields, with
no other text or Markdown.\n{\n  \"answer\": \"Answer text\",
  \"citations\": [\n    \"Document identifier\"],
  \"route\": \"answer\",
  \"needs_human\": false\n}\nroute must be one of
answer, clarify, escalate, or refuse. citations is an array
of strings. needs_human is true only when route is escalate
and false otherwise.\n",
5   "skills": [],
6   "tools": []
7 }
```

Candidate: candidate_1

```
1 {
2   "agentName": "contoso-eval-en",
3   "agentVersion": "1",
4   "system_prompt": "You are a customer support assistant for
Contoso Atlas Cloud. Answer in English using the customer's
inquiry, any supplied policy context, and the evidence rules
below.\n\nYour final response must be a single valid JSON
object and nothing else. Do not include Markdown, code
fences, tool traces, arrays outside the JSON object, or
citation footnote markup inside the answer text.\n\nRequired
JSON shape:\n{\n  \"answer\": \"Answer text\",
  \"citations\":
  [\n    \"Document identifier\"],
  \"route\": \"answer\",
  \"needs_human\": false\n}\n\nField rules:\n- Include exactly
these four fields: answer, citations, route, needs_human.\n-
route must be one of: answer, clarify, escalate, refuse.\n-
needs_human must be true only when route is \"escalate\"; it
must be false for answer, clarify, and refuse.\n- citations
must be an array of document identifiers only, such as
\"ATLAS-SLA-001\", \"ATLAS-SUB-001\", or \"ATLAS-REF-001\".
Do not put URLs, inline citation markers, or quoted snippets
in citations.\n- The answer field should be concise but
complete, and should not contain unsupported policy
claims.\n\nEvidence and tool-use rules:\n- For any policy
question, if the knowledge_base_retrieve tool is connected,
call it before answering.\n- Use the retrieved policy
content, supplied policy context, and the embedded policy
```

< 1 of 1 > Close

Read the instruction changes. Use View changes to identify which response behaviors change. Reject invented policy, unsupported certainty, and non-instruction configuration changes; assess content and effect rather than length. View full-size image (file: web/assets/portal/en/10-optimizer-changes.png)

Save the reviewed candidate's complete instructions

Perform the following only after reviewing a candidate worth retaining.

1. Select the candidate and inspect **View changes**. Reject instructions that invent policy or unsupported certainty.
2. Expand collapsed sections of the revised instructions in **View changes** and copy the **complete text**. If only changed lines are visible, inspect candidate details or Download/Export where provided. If the full instructions are unavailable, follow [candidate-file troubleshooting](#); do not create a file from partial diff fragments.
3. Save **UTF-8 plain text** to `.lab/lab-en/candidate.txt`. Do not use Word/rich text; check that the name is not `candidate.txt.txt`. Exclude diff (change comparison) `+/-` markers, UI explanations, and scores.
4. Record real job/candidate IDs and any manual edits/reasons in `notes.md`. Use this file next, not the repository's `prompts/en/candidate.txt`.

Do not select Promote candidate in this lab. Step 09's CLI checks your ownership records and fixed configuration before creating one v2. Promoting in the portal first can conflict with these ownership checks. Do not continually create v3, v4, or later releases; creating a version is separate from publishing it or approving activation.

Discuss: Present the reviewed candidate and explain the changed behaviors, expected improvements, and possible regressions.

Completion criteria: Record real job/candidate IDs, the reviewed instruction file, and change reasons. Without a candidate worth retaining or complete instructions, record **retain v1 and the specific reason**, then continue to 10. If you do not perform 09, record the separate reevaluation as not run too. Do not repeatedly run the same job to manufacture improvement.

09. Reevaluate and compare v1/v2

What: Reevaluate changed instructions under the same definition and record retain v1, accept v2, or hold.

Why: A v2 label is not improvement evidence. Consider actual errors, latency/tokens, and statistical uncertainty alongside quality.

How · where: Prepare explicit v2 and its same-definition run in the terminal, then compare every case in Foundry Compare runs. Do not publish to production.

Action order: [Terminal · create v2](#) → [Add a run to the same evaluation](#) → [Foundry · compare v1/v2](#).

Use your environment from 01–03 and the candidate file saved in 08. Version creation checks your identity, local ownership records, and remote Agent configuration. For a mismatch, follow [Agent troubleshooting](#); do not bypass ownership checks by replacing the user name in someone else's configuration.

1. Create v2 from the reviewed instructions in your terminal

Keep models, tools, data, evaluators, and Judge fixed, and preserve v1.

Run in your terminal · Shared terminal

```
python -m lab --config .lab/lab-en/.env native-agent --version 2 --prompt .lab/lab-en/candidate.txt --confirm
```

Confirm `agent_name: lab-en-iq` and `version: "2"`. If a different v2 already exists or the model deployment changed, stop and preserve the original records. A new version alone does not demonstrate improvement.

2. Add a v2 run to the same evaluation in your terminal

The helper below is a supplied Python program. It uses the official Azure AI Projects/OpenAI SDK to reuse the baseline's dataset and evaluator settings, rather than creating a different evaluation.

Placeholder	Where to find your value
YOUR_PROJECT_ENDPOINT	AZURE_AI_PROJECT_ENDPOINT in .lab/lab-en/.env
YOUR_SUBSCRIPTION_ID	The subscription ID from 01, also AZURE_SUBSCRIPTION_ID in that .env
YOUR_EVALUATION_ID	The selected evaluation_id from the native-evals output in 06
YOUR_BASELINE_RUN_ID	That evaluation's completed version-1 run_id, not an Optimizer job ID

Replace all four values and execute this single-line command. Submitting a new run incurs charges.

Run in your terminal · Shared terminal

```
python scripts/add_foundry_eval_run.py --endpoint "YOUR_PROJECT_ENDPOINT" --subscription
"YOUR_SUBSCRIPTION_ID" --evaluation "YOUR_EVALUATION_ID" --baseline "YOUR_BASELINE_RUN_ID" --version 2
--name candidate-v2 --out .lab/lab-en/artifacts/foundry-evaluations/candidate-v2.json
```

The default wait is 30 minutes. For a shorter authorized wait, append --wait-seconds and its value in seconds. On completion, confirm status: completed, result_counts.total: 12, and all 12 output items.

When the wait expires, the command returns exit code 2 with **Still running**; the remote job can remain active. If the receipt contains a run ID, repeat the **identical command and --out path** to collect that run. Without a run ID or with an existing remote name, follow [duplicate-submission recovery](#). Do not delete the receipt or rename the run to resubmit.

The helper verifies thresholds, Judge, mappings, and each output item's Agent version and instructions.

Code ↔ portal · add a v2 run to the same evaluation

Where to act: Run [v2 creation](#) and the [evaluation helper above](#), then [compare in Foundry below](#). Do not also select portal Add run after code submission.

Actual code/field	Portal actions and verification
native-agent --version 2, prompt.read_text(...)	Build → Agents → same Agent → Version 2 → Instructions: inspect the full reviewed candidate. This uses the same implementation shown in 03.

Actual code/field	Portal actions and verification
<code>candidate_source()</code> copies baseline <code>data_source</code> , changing only <code>target["version"]</code>	Preserve evaluation, registered data, and query-only input; do not create new data/evaluation.
<code>project.agents.get_version(...)</code> and non-instruction comparison	Model, Knowledge/Tools, and output settings must match across v1/v2. The full SDK definitions provide that evidence.
<code>client.evals.runs.create(**saved["request"])</code>	<code>candidate-v2</code> appears in the same Evaluation runs . Do not also select Add run after code submission.
<code>runs.retrieve(...), output_items.list(...)</code>	Inspect completion and all 12 outputs. Resume collection using the receipt's existing run ID.
The two actual run IDs	Evaluation runs → select v1/v2 → Compare runs → Baseline v1 .

The first function shows **the changed data-source field**, the second **actual SDK submission and duplicate prevention**, and the last **SDK connection, waiting, and complete result collection**. Run only the existing helper command, not these source panels independently.

Actual implementation · **read only** · `scripts/add_foundry_eval_run.py:candidate_source`

Read-only source, not another command to run. The complete named function or object is shown as written, including its original messages; dependencies and imports are in the linked file. Run the lab command only once.

Full source file · 108–135 (file: `scripts/add_foundry_eval_run.py#L108–L135`) · [View on GitHub](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/scripts/add_foundry_eval_run.py#L108-L135)
[\(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/scripts/add_foundry_eval_run.py#L108-L135\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/scripts/add_foundry_eval_run.py#L108-L135)

```
def candidate_source(baseline: dict, *, evaluation_id: str, version: str) -> dict:
    if baseline.get("eval_id") != evaluation_id or baseline.get("status") != "completed":
        raise ValueError("Use a completed baseline run from this exact Foundry evaluation.")
    source = deepcopy(baseline.get("data_source"))
    if not isinstance(source, dict) or source.get("type") != "azure_ai_target_completions":
        raise ValueError("The baseline must be an agent-targeted Foundry evaluation.")
    target = source.get("target", {})
    if target.get("type") != "azure_ai_agent" or not target.get("name"):
        raise ValueError("The baseline must identify a Foundry agent.")
    original = str(target.get("version", ""))
    pinned_version = r"(?:[1-9]\d*|draft-[1-9]\d*)"
    if not re.fullmatch(pinned_version, original) or not re.fullmatch(pinned_version, version):
        raise ValueError("Pin explicit numeric or draft agent versions, not latest.")
    if original == version:
        raise ValueError("The candidate version must differ from the baseline.")
    dataset = source.get("source", {})
    if dataset.get("type") != "file_id" or not dataset.get("id"):
        raise ValueError("The baseline must use an identified, registered dataset.")
    messages = source.get("input_messages", {})
    template = messages.get("template", [])
    if messages.get("type") != "template" or len(template) != 1 or template[0].get("role") != "user":
        raise ValueError("Use one query-only user message, without reference-answer leakage.")
    content = template[0].get("content")
    query = content if isinstance(content, str) else content.get("text") if isinstance(content, dict)
```

```

else None
    if query != "{{item.query}}":
        raise ValueError("This workshop sends only {{item.query}} to the agent.")
    target["version"] = version
    return source

```

Actual implementation · read only · `scripts/add_foundry_eval_run.py:submit_or_resume`

Read-only source, not another command to run. The complete named function or object is shown as written, including its original messages; dependencies and imports are in the linked file. Run the lab command only once.

Full source file · 138–190 (file: `scripts/add_foundry_eval_run.py#L138–L190`) · [View on GitHub](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/scripts/add_foundry_eval_run.py#L138-L190)
https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/scripts/add_foundry_eval_run.py#L138-L190

```

def submit_or_resume(project, client, *, evaluation_id: str, baseline_run_id: str,
                    version: str, name: str, output: Path) -> dict:
    intent = {
        "evaluation_id": evaluation_id, "baseline_run_id": baseline_run_id,
        "candidate_version": version, "name": name,
    }
    if output.exists():
        saved = json.loads(output.read_text(encoding="utf-8"))
        if saved.get("intent") != intent:
            raise ValueError("This receipt belongs to a different evaluation or candidate.")
        if not saved.get("run_id"):
            raise ValueError("Submission outcome is unknown. Inspect Foundry before any new
submission; preserve this receipt.")
        return saved
    baseline = as_object(client.evals.runs.retrieve(run_id=baseline_run_id, eval_id=evaluation_id))
    source = candidate_source(baseline, evaluation_id=evaluation_id, version=version)
    agent_name = source["target"]["name"]
    original_version = str(baseline["data_source"]["target"]["version"])
    before = as_object(project.agents.get_version(agent_name=agent_name,
agent_version=original_version))["definition"]
    after = as_object(project.agents.get_version(agent_name=agent_name, agent_version=version))
["definition"]
    if not isinstance(after.get("instructions"), str) or not after["instructions"].strip():
        raise ValueError("The candidate has no actual instructions.")
    if {key: value for key, value in before.items() if key != "instructions"} != {
        key: value for key, value in after.items() if key != "instructions"
    }:
        raise ValueError("Model, tools, or other agent settings changed. This is not an instruction-
only comparison.")
    for existing in client.evals.runs.list(eval_id=evaluation_id, limit=100):
        existing = as_object(existing)
        if existing.get("name") == name:
            raise ValueError(
                f"Foundry already has a run named {name}: {existing.get('id')}. "
                "Open that run or restore its original receipt; do not create a duplicate."
            )
    evaluation = as_object(client.evals.retrieve(evaluation_id))
    contract = evaluation_contract(evaluation)
    saved = {
        "intent": intent,
        "status": "submitting",

```

```

        "run_id": None,
        "agent_name": agent_name,
        "dataset": source["source"],
        "testing_criteria_sha256": contract["testing_criteria_sha256"],
        "evaluation_contract": contract,
        "request": {"eval_id": evaluation_id, "name": name, "data_source": source},
        "production_approval": "NOT_GRANTED",
    }
    write_once_json(output, saved)
    # An interrupted POST intentionally leaves an unknown outcome, not permission to retry.
    run = as_object(client.evals.runs.create(**saved["request"]))
    if not run.get("id") or run.get("eval_id") != evaluation_id:
        raise ValueError("Foundry returned no matching evaluation-run identity; preserve the
submission receipt.")
    saved.update(status=run["status"], run_id=run["id"], run=run)
    save_json(output, saved)
    return saved

```

Actual implementation · read only · scripts/add_foundry_eval_run.py:main

Read-only source, not another command to run. The complete named function or object is shown as written, including its original messages; dependencies and imports are in the linked file. Run the lab command only once.

Full source file · 193–268 (file: [scripts/add_foundry_eval_run.py#L193–L268](#)) · [View on GitHub](#)
https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/scripts/add_foundry_eval_run.py#L193-L268

```

def main(argv: list[str] | None = None) -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--endpoint", required=True, help="The same Foundry project endpoint as the
baseline")
    parser.add_argument("--subscription", required=True, help="The explicitly selected Azure CLI
subscription")
    parser.add_argument("--evaluation", required=True, help="Existing eval_... ID; no new evaluation
definition is created")
    parser.add_argument("--baseline", required=True, help="Completed baseline evalrun_... ID")
    parser.add_argument("--version", required=True, help="Explicit candidate agent version")
    parser.add_argument("--name", default="candidate-v2")
    parser.add_argument("--out", type=Path, default=Path(".lab/foundry-evaluations/candidate-
v2.json"))
    parser.add_argument("--wait-seconds", type=int, default=1800)
    args = parser.parse_args(argv)
    try:
        endpoint = urlsplit(args.endpoint)
        if (
            endpoint.scheme != "https" or endpoint.username or endpoint.password
            or endpoint.port or endpoint.query or endpoint.fragment
            or not re.fullmatch(r"[a-z0-9-]+\\.services\.ai\.azure\.com", endpoint.hostname or "")
            or not re.fullmatch(r"/api/projects/[A-Za-z0-9_().-]+/?", endpoint.path)
        ):
            raise ValueError("Use the exact HTTPS Foundry project endpoint, without credentials or
query parameters.")
        if not 1 <= args.wait_seconds <= 3600:
            raise ValueError("--wait-seconds must be between 1 and 3600.")
        account = subprocess.run(
            ["az", "account", "show", "--subscription", args.subscription, "-o", "json"],

```

```

        capture_output=True, text=True, check=True, timeout=30,
    )
    identity = json.loads(account.stdout)
    if identity.get("id", "").lower() != args.subscription.lower() or identity.get("state") !=
"Enabled":
        raise ValueError("Azure CLI did not confirm the intended enabled subscription.")
    from azure.ai.projects import AIProjectClient
    from azure.identity import AzureCliCredential

    with AzureCliCredential(subscription=args.subscription, process_timeout=30) as credential:
        with AIProjectClient(endpoint=args.endpoint, credential=credential, retry_total=0) as
project:
            with project.get_openai_client(max_retries=0, timeout=120) as client:
                saved = submit_or_resume(
                    project, client, evaluation_id=args.evaluation, baseline_run_id=args.baseline,
                    version=args.version, name=args.name, output=args.out,
                )
                print(f"Foundry evaluation: {args.evaluation}\nRun: {saved['run_id']}\"",
flush=True)

                deadline = time.monotonic() + args.wait_seconds
                while True:
                    run = as_object(client.evals.runs.retrieve(run_id=saved["run_id"],
eval_id=args.evaluation))
                    saved.update(status=run["status"], run=run)
                    save_json(args.out, saved)
                    if run["status"] in {"completed", "failed", "canceled", "cancelled":
                        if run["status"] == "completed":
                            items = [
                                as_object(item) for item in client.evals.runs.output_items.list(
                                    run_id=saved["run_id"], eval_id=args.evaluation,
                                )
                            ]
                            saved["output_items"] = items
                            save_json(args.out, saved)
                            if len(items) != (run.get("result_counts") or {}).get("total"):
                                raise ValueError("Completed-run output is incomplete; preserve the
receipt.")

                            agent = as_object(project.agents.get_version(
                                agent_name=saved["agent_name"], agent_version=args.version,
                            ))
                            saved["execution_verification"] = verify_agent_execution(
                                items, agent_name=saved["agent_name"], version=args.version,
                                instructions=agent["definition"]["instructions"],
                            )
                            save_json(args.out, saved)
                            print(json.dumps({
                                "status": run["status"], "result_counts": run.get("result_counts"),
                                "report_url": run.get("report_url"), "production_approval":
"NOT_GRANTED",

                                }, indent=2))
                            return 0 if run["status"] == "completed" else 1
                    if time.monotonic() >= deadline:
                        print("Still running. Repeat this exact command to collect the same run;
do not delete the receipt.", file=sys.stderr)
                        return 2
                    time.sleep(15)
            except (OSError, ValueError, subprocess.SubprocessError) as error:

```

```
print(f"ERROR: {error}", file=sys.stderr)
return 1
```

3. Compare the same evaluation's v1/v2 in Foundry

In the same evaluation's **Evaluation runs**, select the v1 and v2 rows and **Compare runs**. Explicitly choose **v1 as Baseline**; selection order must not reverse the comparison.

The screenshot shows the 'Statistically compare results' page in Microsoft Foundry. The left sidebar contains navigation options: Create (Agents, Models, Services, Tools, Knowledge, Memory, Guardrails, Data), Optimize (Evaluations, Fine-tune), and a search bar. The main content area displays the comparison details. At the top, it shows 'Total runs selected: 2' and 'Analysis method: PairedTTest'. Below this, a dropdown menu shows 'Baseline *' with 'contoso-eval-en' selected. A table compares the baseline and candidate results:

Name	contoso-eval-en	candidate-v2
TaskAdherence	1.00 Baseline	1.00 Inconclusive
Relevance	4.42 Baseline	4.33 Inconclusive

At the bottom of the table, it indicates '1-2 of 2' and has 'Prev' and 'Next' navigation links.

Confirm the comparison direction and results. Select v1 as Baseline, then compare scores, pass counts, and statistical results. Inconclusive means a difference was not established; it is not evidence of equivalence. View full-size image (file: <web/assets/portal/en/20-evaluation-comparison.png>)

Compare responses, scores, and reasons for all 12 cases side by side. Require no decline in all-criteria passes or either metric's pass count/mean, and at least one strict measured improvement. Check factual truth, routing, and response format too, and record latency, tokens, and statistical results.

Explain the result: Identify the actual gain, unchanged criteria, any regression, and the remaining uncertainty. Observed improvement is not a guarantee that every future stochastic run will improve.

In `notes.md`, record v1/v2 pass counts and means per metric, errors, latency/tokens, statistical results, and your decision side by side. If improvement criteria are not met, **retain v1 or hold the decision**. Do not adopt a candidate with actual policy errors merely because generic evaluators passed it.

Latency is time spent waiting for a response; **tokens** measure the amount of text processed by the model. A small score improvement can come with increased waiting time and cost.

Decision	When it fits
Consider v2 for adoption	All 12 cases use the same conditions, quality does not regress and at least one measure improves, no policy errors are found, and latency/token tradeoffs were reviewed. Production approval is separate.
Retain v1	No improvement, or previously correct behavior got worse. This is a valid learning outcome.
Hold the decision	Errors, missing results, or different conditions prevent a fair judgment. Record the missing evidence.

Completion criteria: Record the two actual run IDs, complete same-criteria comparison, and the reasons for accepting, retaining, or holding. Do not accumulate release numbers. Production approval and independent generalization are separate and are not granted by this dev12 exercise.

10. Save results and delete resources

What: Preserve results, then delete the dedicated lab resources you created. Only with explicit retention authorization, record remaining resources and a cost-management plan instead.

Why: Search and logs can remain after the browser closes. Deleting the wrong shared group can also remove someone else's resources.

How · where: Verify exact targets, ownership, and authorization in Azure Portal and the terminal, then verify absence after deletion.

Action order: [Preserve results](#) → [Check deletion/retention scope](#) → [Delete the authorized dedicated group](#) → [Terminal · verify absence](#). Do not delete an environment approved for retention.

Closing a browser, deleting an Agent, or running `cleanup` does not by itself stop all resource-group charges.

Preserve records before deletion

1. Record the actual project, Agent, dataset version/hash, evaluation/run/job IDs, and decision in `notes.md`.
2. Use **Download/Export** where offered in the evaluation and Optimizer views. If unavailable, retain existing receipts and detailed results; do not claim an export you could not obtain.
3. Retain `.lab/lab-en/` plans, manifest, approvals, and run records in an organization-approved private location. Do not commit raw credentials, cookies, signed URLs, or account details. Public records should contain only the necessary synthetic cases, scores, and reasons.

4. Inspect Evaluations and Optimization runs for active jobs. Use **Cancel** where supported and verify the terminal state. Record unresolved job IDs, states, and follow-up checks; a cancellation request alone does not prove termination.

Verify deletion targets and retention

Environment	Cleanup scope
Dedicated resource group you created in 02 · default path	Follow group deletion below. Reconfirm that no unrelated resources are present and deletion is within the authorized scope.
Dedicated environment explicitly approved for retention · exception	Do not run deletion commands. Record the actual remaining resources, reason, cost responsibility, review date, and subsequent deletion plan.
Shared/external resources appear in the deletion inventory	Do not delete those resources. Inspect out-of-scope dependencies and stop group deletion until an authorized scope is established.
Setup or execution stopped partway through	Compare <code>config.json</code> , the manifest, and actual Azure resources. Failure does not mean nothing was created.

The default path deletes your dedicated group. After saving records and checking the target, continue to [the next section](#). Individual object deletion is not required before deleting the group. The following cleanup is a separate option **for retaining the group while removing recorded objects**.

Optional · remove recorded objects while retaining the group

With your own `.env` and local ownership records, you can inspect this **plan-only** command. If provisioning stopped before `.env` was created, do not run object cleanup; inspect the actual group with `config.json` and the manifest instead.

Run in your terminal · Shared terminal

```
python -m lab --config .lab/lab-en/.env cleanup
```

Read `mode: LOCAL_PLAN_ONLY`, `actions`, `never_deleted`, and `manual_follow_up`. Verify names, project, and ownership. Run the following only after obtaining authorization for exactly those deletions. If your environment name differs, use its matching prefix.

Run in your terminal · Shared terminal

```
python -m lab --config .lab/lab-en/.env cleanup --confirm-prefix lab-en
```

Confirm mode: OWNED_OBJECTS_ABSENT. This removes recorded Agent versions, search objects, and connections, but **not resource groups, model deployments, Search hosting, logs, or RBAC**. It does not automatically remove every portal-created dataset, evaluation, Optimizer job, Playground conversation, or model-smoke response. Delete authorized items individually in their owning view. If no delete action is offered, record the remaining item and retention reason, and decide dedicated-group deletion separately.

Code ↔ portal · distinguish object cleanup from group deletion

Where to act: The default path is [group deletion](#), then [absence verification](#). Use [optional object-only cleanup above](#) only for an authorized retained group. You do not need to perform both paths.

Existing command/implementation	Portal actions and verification
actions, never_deleted, manual_follow_up in cleanup_plan()	A plan from local ownership records. Compare Agent versions, Search objects, and connections. Other library-path follow-up text does not add exercises to this lab.
delete_version, REST DELETE, then GET in cleanup()	Remove only verified recorded objects and check absence. Individual deletion needs authorization too; do not also delete those objects in the portal.
az group show, az resource list below	Inspect Resource groups → your group → Overview/Resources , inventory, and tags.
az group delete below	The group-wide counterpart of Delete resource group . Use one method, not both.
A successful az group exists result of false	Verify actual absence against notifications/group inventory. An error is not false.

Read **returning a plan without confirmation → ownership/etag checks → individual DELETE → remaining-item verification**. Group deletion is not inside this function; it is performed separately by the existing Azure CLI command or portal in the next section.

Actual implementation · read only · lab/cleanup.py:cleanup_plan

Read-only source, not another command to run. The complete named function or object is shown as written, including its original messages; dependencies and imports are in the linked file. Run the lab command only once.

Full source file · 17–74 (file: `lab/cleanup.py#L17–L74`) · [View on GitHub \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/cleanup.py#L17-L74\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/cleanup.py#L17-L74)

```
def cleanup_plan(config: Config) -> dict:
    state = workspace(config)
    names = resource_names(config, state)
    allowed = {
        "search_index": (names["index"], search_url(config, "indexes", names["index"])),
        "knowledge_source": (names["source"], search_url(config, "knowledgesources",
names["source"])),
```

```

        "knowledge_base": (names["base"], search_url(config, "knowledgebases", names["base"])),
        "project_connection": (
            names["connection"],
            f"https://management.azure.com{config.project_id}/connections/{names['connection']}?api-version={CONNECTION_API}",
        ),
    }
    allowed_agents = {config.agent_name(stage) for stage in ("baseline", "iq", "optimized", "tuned")}
    actions = []
    for record in state["created"]:
        kind = record.get("kind")
        if kind == "agent_version":
            if record.get("name") not in allowed_agents or not re.fullmatch(r"[A-Za-z0-9_-]+",
str(record.get("version", ""))):
                raise LabError("정리 기록의 에이전트 범위가 다릅니다. 삭제하지 않습니다.")
            elif kind in allowed:
                if (record.get("name"), record.get("url")) != allowed[kind]:
                    raise LabError("정리 기록의 Search/프로젝트 범위가 다릅니다. 삭제하지 않습니다.")
            else:
                raise LabError(f"자동 정리를 지원하지 않는 기록 유형: {kind}")
        actions.append(dict(record))
    for metadata_path in (ARTIFACTS / "runs").glob("*/*metadata.json"):
        metadata = read_json(metadata_path)
        if metadata.get("workspace_id") != state["workspace_id"]:
            continue
        if metadata.get("project_endpoint") != config.project_endpoint:
            raise LabError("응답 정리 대상의 프로젝트 endpoint가 다릅니다.")
        outputs_path = metadata_path.parent / "outputs.jsonl"
        if not outputs_path.exists():
            continue
        for row in read_jsonl(outputs_path):
            identifier = row.get("response_id")
            if identifier:
                if not re.fullmatch(r"resp_[A-Za-z0-9_-]+", identifier):
                    raise LabError("알 수 없는 response ID 형식입니다. 자동 삭제하지 않습니다.")
                actions.append({"kind": "response", "id": identifier})
    order = {"response": 0, "agent_version": 1, "project_connection": 2, "knowledge_base": 3,
"knowledge_source": 4, "search_index": 5}
    actions.sort(key=lambda action: order[action["kind"]])
    return {
        "mode": "LOCAL_PLAN_ONLY",
        "project_id": config.project_id,
        "prefix": config.prefix,
        "workspace_id": state["workspace_id"],
        "actions": actions,
        "never_deleted": [
            "resource group", "Foundry account/project", "shared model deployments",
            "Search service", "Application Insights/Log Analytics", "RBAC assignments",
        ],
        "manual_follow_up": [
            "Managed evaluation jobs/files are retained for evidence; inspect recorded IDs and remove
in the portal if required.",
            "SFT/FDE training jobs, uploaded training files and model deployments require their own
scoped cleanup.",
            "An empty stable agent entry may remain after deleting its recorded versions; inspect
ownership before removing it in the portal.",
        ],
    }

```

```
    ],  
}
```

Actual implementation · read only · lab/cleanup.py:cleanup

Read-only source, not another command to run. The complete named function or object is shown as written, including its original messages; dependencies and imports are in the linked file. Run the lab command only once.

Full source file · 77–165 (file: `lab/cleanup.py#L77–L165`) · [View on GitHub \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/cleanup.py#L77-L165\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/cleanup.py#L77-L165)

```
def cleanup(config: Config, *, confirm_prefix: str | None = None) -> dict:  
    plan = cleanup_plan(config)  
    if confirm_prefix is None:  
        return plan  
    if confirm_prefix != config.prefix:  
        raise LabError("--confirm-prefix가 현재 LAB_PREFIX와 다릅니다. 삭제하지 않았습니다.")  
    with credential_for(config) as credential:  
        with AIPProjectClient(endpoint=config.project_endpoint, credential=credential) as project:  
            with project.get_openai_client(max_retries=0, timeout=60.0) as client:  
                search = JsonHttp(credential, scope=SEARCH_SCOPE,  
allowed_origin=config.search_endpoint)  
                arm = JsonHttp(credential, scope=ARM_SCOPE,  
allowed_origin="https://management.azure.com")  
                inspected = []  
                for action in plan["actions"]:  
                    item = {**action, "exists": True}  
                    kind = action["kind"]  
                    if kind == "agent_version":  
                        try:  
                            remote = project.agents.get_version(agent_name=action["name"],  
agent_version=action["version"])  
                        except ResourceNotFoundError:  
                            item["exists"] = False  
                        else:  
                            if (remote.metadata or {}).get("workspace") != plan["workspace_id"]:  
                                raise LabError("원격 에이전트 소유 표식이 다릅니다. 아무 대상도 삭제하지 않았습니다.")  
                    elif kind == "response":  
                        try:  
                            response = client.responses.retrieve(action["id"])  
                        except NotFoundError:  
                            item["exists"] = False  
                        else:  
                            if (response.metadata or {}).get("lab_workspace") != plan["workspace_id"]:  
                                raise LabError("원격 응답 소유 표식이 다릅니다. 아무 대상도 삭제하지 않았습니다.")  
                    else:  
                        transport = arm if kind == "project_connection" else search  
                        try:  
                            remote = transport.request("GET", action["url"])  
                        except CloudRequestError as exc:  
                            if exc.status != 404:  
                                raise  
                            item["exists"] = False  
                        else:  
                            if action.get("etag") and remote.etag and action["etag"] != remote.etag:
```

```

        raise LabError("생성 후 원격 리소스 구성이 변경되었습니다. 정리 대상을 직접 검토해야 합
니다.")

        item["current_etag"] = remote.etag
        inspected.append(item)
    completed = []
    for action in inspected:
        if not action["exists"]:
            completed.append(**action, "result": "already_absent")
            continue
        kind = action["kind"]
        if kind == "response":
            client.responses.delete(action["id"])
        elif kind == "agent_version":
            project.agents.delete_version(agent_name=action["name"],
agent_version=action["version"])
        else:
            transport = arm if kind == "project_connection" else search
            transport.request("DELETE", action["url"], etag=action.get("current_etag"))
            completed.append(**action, "result": "delete_request_succeeded")
            save_json(ARTIFACTS / "cleanup.json", **plan, "mode": "DELETING", "completed":
completed})

    remaining = []
    for action in inspected:
        kind = action["kind"]
        if kind == "response":
            try:
                client.responses.retrieve(action["id"])
            except NotFoundError:
                continue
        elif kind == "agent_version":
            try:
                project.agents.get_version(agent_name=action["name"],
agent_version=action["version"])
            except ResourceNotFoundError:
                continue
        else:
            transport = arm if kind == "project_connection" else search
            try:
                transport.request("GET", action["url"])
            except CloudRequestError as exc:
                if exc.status == 404:
                    continue
                raise
            remaining.append(action)

    result = {
        **plan,
        "mode": "OWNED_OBJECTS_ABSENT" if not remaining else "DELETION_PENDING",
        "completed": completed,
        "remaining": remaining,
    }
    save_json(ARTIFACTS / "cleanup.json", result)
    return result

```

Delete a dedicated group and verify its absence

Replace `YOUR_LAB_RESOURCE_GROUP` with `names.resource_group` from `config.json`. Verify subscription, ownership tags, and the entire inventory, not just a familiar prefix.

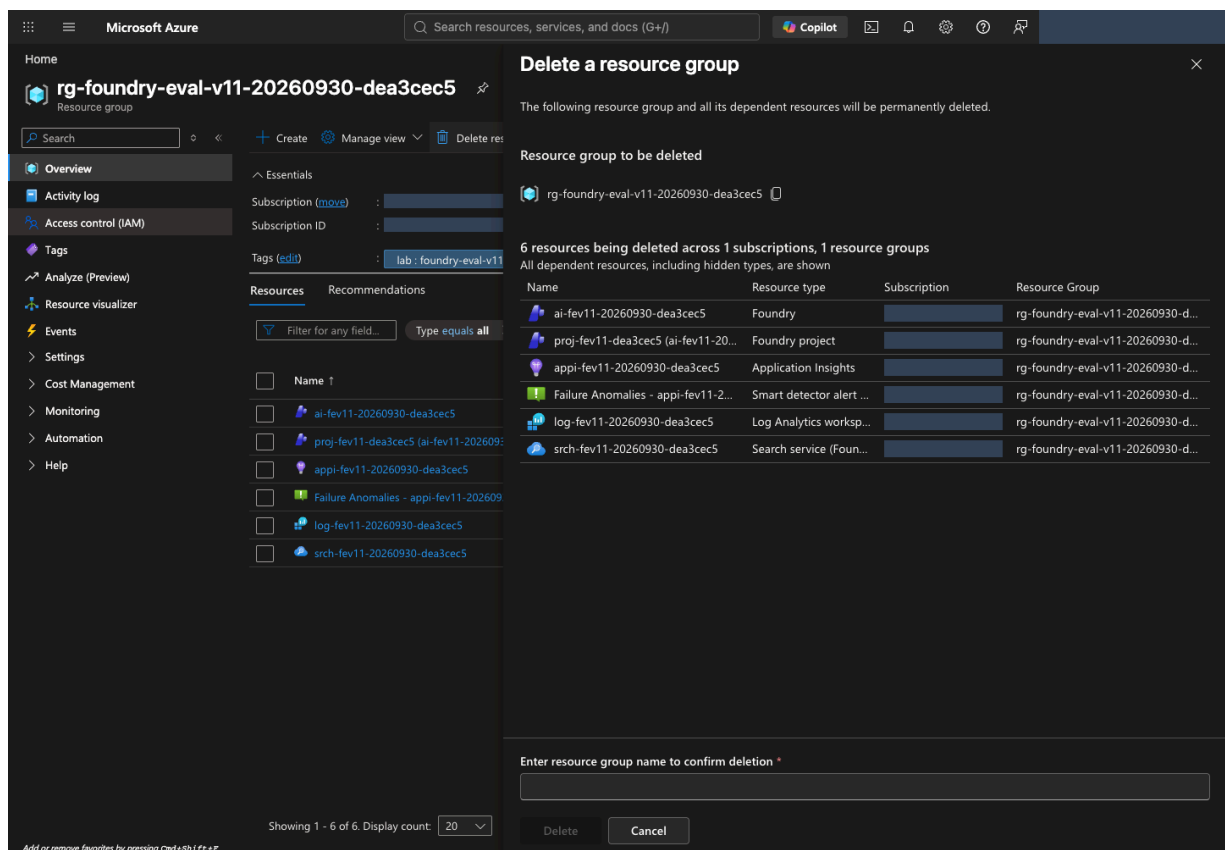
Run in your terminal · Shared terminal

```
az group show --subscription "YOUR_SUBSCRIPTION_ID" --name "YOUR_LAB_RESOURCE_GROUP" --query "{name:name,location:location,tags:tags}" -o json
az resource list --subscription "YOUR_SUBSCRIPTION_ID" --resource-group "YOUR_LAB_RESOURCE_GROUP" --query "[].{name:name,type:type}" -o table
```

Resource-group deletion is irreversible and removes the group's Foundry resources, model deployments, Search, and monitoring together. After saving evidence and receiving approval for that exact group, choose one method:

The default Application Insights Smart Detection action group can also serve alerts in other resource groups. A dedicated lab group does not establish that every resource is independent. Review shared dependencies with the owner before deletion, or retain the group; do not delete alerts, permissions, or locks simply to bypass a check.

- 1. Portal:** Azure Portal → Resource groups → exact group → **Delete resource group**. Read the deletion inventory, type the requested group name, and confirm.
- 2. CLI:** Run the command below and review the target again at the confirmation prompt. Do not append `--yes` to bypass confirmation.



Read the proposed inventory and confirmation field first. Verify the group name and every resource. Only after authorization, enter the group name and proceed with Delete. Use Cancel if the target is wrong or deletion is not approved. View full-size image (file: web/assets/portal/en/28-delete-review.png)

Run in your terminal · Shared terminal

```
az group delete --subscription "YOUR_SUBSCRIPTION_ID" --name "YOUR_LAB_RESOURCE_GROUP"
```

Verify absence after deletion in your terminal

An accepted request is not completed deletion. Check portal notifications and group status, then verify that this command successfully prints `false`:

Run in your terminal · Shared terminal

```
az group exists --subscription "YOUR_SUBSCRIPTION_ID" --name "YOUR_LAB_RESOURCE_GROUP"
```

Authentication or network errors are not `false` and do not prove deletion. If `true`, inspect deletion progress. For **Locks**, permissions, or dependencies in other groups, follow [deletion troubleshooting](#). Do not remove organizational locks without authorization.

Afterward, open **Cost Management** → **Cost analysis** for the subscription, time range, and group. Billing updates can lag, and earlier usage charges do not disappear. Separately check logs/storage in other groups and service-specific soft-deleted resources. Permanent deletion/purge requires organizational policy and separate authorization.

Final completion criteria: For authorized deletion of your dedicated group, record `az group exists` returning `false` and the verification time. For explicitly authorized retention, record the actual remaining items, reasons, cost responsibility, review date, and subsequent deletion plan. Verify the outcome matching your authorization; do not delete `.lab` first and lose ownership evidence. See the [cleanup checklist](#) and [deletion troubleshooting](#).

Lab checklist · improve the measured result

[Ten lab steps](#) · [Environment and result worksheet](#) · [Troubleshooting](#)

Use this checklist to review your own **evaluate → learn → improve → reevaluate** lab. It is not an alternative to the [hands-on guide](#). Do not manufacture a weak baseline or promise that every future evaluation will improve.

Check before starting

Every participant performs 01–10 directly. Use your own account, computer, dedicated environment, and Agent; do not delegate creation, commands, or cleanup. Confirm provisioning and role-assignment access, spending authorization, and deletion/retention scope before starting.

Follow [01's Windows/macOS/Linux installation](#), complete [version checks](#) for **Python 3.11–3.14, Git, and Azure CLI**, and create the virtual environment. Resolve installation approvals, PATH, and proxy issues first.

After planning in 02, copy the [worksheet](#) into your `notes.md`. Add actual values and completion evidence at each step. Never use another person's configuration, sign-in, or ownership records; use your originals when resuming.

Read the [six basic terms](#) and [question → answer → scoring diagram](#). Check that you can explain **the Agent answers and the Judge scores**, and **instructions, not model weights, will change**. Memorized terms and high scores are not starting conditions.

Steps 04–06 use one evaluation wizard, not three evaluations. In 07, connect one case's scores, reasons, and policy before analyzing the whole result.

Use **Action order / Where to act** links to find the actual command or portal screen first. Read **code ↔ portal** tables and collapsed implementation panels when you need the internals; do not execute them separately. In particular, [06's ID lookup](#) does not submit, and [08 Optimizer](#) runs in the portal.

New lab Agents use `lab-en-iq` or `lab-ko-iq`. If the environment prefix differs, use the actual name printed by the command. Verify [model roles and identity](#) and reserve time for cleanup in 10.

Both v1 and v2 are full immutable Agent versions. Their model, tools, reasoning, and strict output schema match; only instructions differ. Do not weaken v1 to create improvement. Create one reviewed v2 and reevaluate under the same criteria.

Keep English and Korean data/Agents separate and compare each language's actual executions independently. Translating another language's result does not constitute a new run.

Verify completion across all ten steps

Step	Required action	Completion evidence
01 Environment	Check account, subscription, permissions, Python 3.11–3.14, Git, Azure CLI, and language variables	Matching identity, all three version checks, and successful local commands in the virtual environment
02 Provisioning	Plan, authorize, preflight, apply, and inspect status	APPLIED, generated <code>.env</code> , and runtime preflight PASS
03 Agent setup	Verify model/retrieval and create <code>native-agent v1</code>	Actual JSON response and Agent tool call
04 Dataset	Register or reuse unchanged three-column JSONL	All 12 rows, registration/version, and SHA-256
05 Criteria	Relevance 4, TaskAdherence 1, and the actual Judge	Record the unsubmitted wizard settings and query-only input
06 Baseline	Foundry Evaluation against pinned v1	Completed actual run ID and all 12 items
07 Analysis	Read actual answers, scores, reasons, and policies	Concrete improvement hypothesis, including failures
08 Optimization	Instruction-only optimization and candidate-diff review	Job/candidate IDs, reviewed file and provenance, or the reason to retain v1 without a different candidate
09 Reevaluation	With a reviewed candidate, evaluate same-criteria v2 and open Compare runs	Complete paired evidence and retain/accept/hold decision; without a different candidate, continue to 10
10 Cleanup	Preserve results and ownership, then clean up your dedicated resources	Verified authorized group absence, or the actual inventory, cost responsibility, and review date for approved retention

Use **Individual turns / One time**. Select **Upload new dataset** on first registration and **Existing dataset** for reuse/resumption. The upload preview can show only five rows; the experiment still contains 12. Version selection can clear the target checkbox: reselect it and confirm one target.

Check quality using actual cases

Connect **question → actual answer → evaluator score and reason → policy/reference answer**.
`conversation_id` → User view shows the conversation; the evaluator reasons belong in **Detailed metrics result**, such as `Relevance.reason` and `TaskAdherence.reason`.

Analyze at least one weak case and one strong case. If no failure is observed, use the lowest-scoring case and say no failure was found. Never weaken v1, omit a difficult row, change a reference answer, or substitute local scores. Use your own actual results when discussing with other participants too.

Keep the selected lab's twelve synthetic questions, actual responses and managed evaluator reasons in separate private run records. Preserve unsuccessful outcomes; remove tokens, cookies, signed URLs and private account metadata if results need to be shared.

Interpret improvements correctly

Relevance 4/5 is not 80% accuracy. TaskAdherence 1 is Pass, not a poor one-out-of-five score. Report each evaluator separately, all-criteria passes, errors and coverage.

The [required comparison in 09](#) uses Foundry. The optional `scripts/compare_foundry_eval.py` also requires complete matched cases, identical data/model/tools/settings, per-item version/instruction attestation, no lower pass counts or means, and at least one strict measured quality improvement. Malformed responses remain failures; the helper never repairs them into a pass.

Read answers for unsupported certainty, incorrect policy arithmetic, action claims and wrong routing even if the generic evaluator passed them. A request demanding an impossible certainty must not be satisfied with invented facts.

Optimizer ranking is internal selection evidence, not the direct managed reevaluation mean or pass rate. A higher ranking alone does not establish a better v2.

Recover without changing the experiment

Symptom	Response
Wrong identity, 401/403	Confirm the approved account/project; do not widen shared permissions
Catalog or deployment succeeds but Agent fails	Record the actual API/runtime error; verify the intended role

Symptom	Response
Portal Add run item-schema error	Use the official SDK helper with the existing baseline data source, not a different scorer
No custom evaluators available	Switch Custom only OFF or select View built-in evaluators
Duplicate local receipt or existing remote run	Retrieve that same run; do not rename it to force another submission
Candidate changes model/tools/reasoning/schema	Reject an instruction-only comparison claim
Existing released v2 differs from local source	Do not overwrite it or create v3; preserve the release and inspect the mismatch
Missing output or metrics	Report incomplete execution; do not reduce the denominator
Statistical comparison is Inconclusive	Report that result; do not claim significance or equivalence

Relevance maps to `{{sample.output_text}}`; TaskAdherence maps to `{{sample.output_items}}`. Do not overwrite service-generated bindings or feed `ground_truth` into the Agent. The actual remote Judge overrides any stale local default.

After a timeout, identify **which existing job ID to resume** first. Use the [issue record](#) to prevent deletion/recreation or paid retries “until it succeeds.” Do not mark a blocked step completed.

Correct common misconceptions

Claim	Correction
“V2 is a prompt file number only.”	It is a complete Agent version; this experiment changes only its instructions
“A more powerful model makes v2 better than old-model v1.”	Both versions must use the same model for the instruction comparison
“A better v2 can be guaranteed before execution.”	Only observed improvement can be established; future stochastic results are not guaranteed
“Keep only successful rows.”	Keep every case, including failures and missing/error outcomes
“The source schema certifies correct business decisions.”	It controls output structure, not policy truth or proper routing
“Public evidence must include cookies and raw logs.”	Publish actual synthetic responses and reasons through an allowlist, not authentication data

Finish with a separate lab record

Keep the actual environment, run IDs, comparison conditions, and final decision in your private records. Do not append execution results or validation history to reusable guides; correct instructions only when they are wrong.

Record the same-model result, native statistical comparison, latency/token tradeoffs, and limits of a reused synthetic dev set. A lab adoption decision is not authorization to publish or activate in production; publication is outside this lab.

Finally, perform [the cleanup/retention checks in 10](#) yourself. Confirm authorized deletion, or the actual retained inventory, cost responsibility, and review date. Do not delete records needed to establish ownership or reopen the environment.

Environment setup reference · access, authorization, and cleanup

[Ten steps from the beginning](#) · [Lab checklist](#) · [Troubleshooting](#)

Use this reference for **your own steps 02 provisioning, 03 Agent setup, and 10 cleanup**. Follow the [hands-on guide](#) in order and open only the linked sections you need. Record actual values and completion evidence in your `.lab/lab-en/notes.md`.

Command shortcuts: [CLI sign-in](#) · [Language/records](#) · [Provisioning](#) · [Policies/Agent](#) · [Evaluation IDs](#) · [Reevaluation](#) · [Group deletion/verification](#). Run commands from the lab folder's terminal; do not repeat completed work.

To read inside unchanged commands, open [02 creation code/portal](#), [03 retrieval code/portal](#), [Agent configuration](#), [09 submission/lookup](#), and [10 cleanup scope](#). Source panels are read-only, not instructions to run another program or duplicate creation in the portal.

Prepare your dedicated lab environment

Situation	Action
Starting for the first time	Begin at 01 account setup and perform the authorized bootstrap plan → preflight → apply → status path yourself.
Resuming your same lab	Compare original config, manifest, receipts, and actual state with the worksheet . Do not repeat completed creation or submission.
Only another workload's shared project is available	This lab does not adopt existing groups. Obtain subscription access and authorization to create your own dedicated environment before continuing.

Verify CLI, SDK, and portal identities separately and use the same personal identity throughout. Never use another person's tokens, sign-in session, configuration, or ownership records. Run creation, ID lookup, v2 creation, reevaluation, and cleanup yourself.

Every participant uses their own dedicated environment and Agent. Do not have several people create the same Agent's v2 or submit the same paid job. The same example environment name on separate computers still produces Azure resource names with unique suffixes.

Access and spending authorization are required starting conditions. Obtain an approved scope through organizational procedures if needed. This is not an alternative path where someone else performs the remaining exercises.

Default Agent names are `lab-en-iq` for English and `lab-ko-iq` for Korean. If the environment name changes, use the actual name returned by the CLI and record it in the worksheet.

Check the isolated resource group. Open the group named in your provisioning records and confirm its subscription, region, and resource inventory. Verify that the Foundry project and supporting resources belong to your lab. View full-size image (file: `web/assets/porta1/en/00-resource-group.png`)

Verify model roles and actual deployments

Role	Configuration key	Default model/version
Agent	MODEL_DEPLOYMENT	gpt-6-sol / 2026-09-22, GlobalStandard 100
Managed Evaluation Judge	JUDGE_DEPLOYMENT	gpt-6-luna / 2026-09-22, GlobalStandard 100
Optimizer/search planner	OPTIMIZER_DEPLOYMENT / IQ_PLANNER_DEPLOYMENT	gpt-5.5 / 2026-04-24, one shared GlobalStandard 100 deployment

Role	Configuration key	Default model/version
Policy embeddings	EMBEDDING_DEPLOYMENT	text-embedding-3-small / 1, GlobalStandard 10

Numbers are requested ARM capacity units, not a universal TPM conversion. These defaults require availability checks; they are not a deployment guarantee for every subscription. Inspect deployment names, model names, versions, and readiness in Foundry **Models + endpoints/Build → Models**. `.env` contains actual deployment names rather than product names.

Distinguish additional monitoring resources. Application Insights can automatically add a [default Failure Anomalies alert and Smart Detection action group](https://learn.microsoft.com/azure/azure-monitor/alerts/proactive-failure-diagnostics#alert-rule-creation) (<https://learn.microsoft.com/azure/azure-monitor/alerts/proactive-failure-diagnostics#alert-rule-creation>). Bootstrap checks read-only that the alert targets only the owned Application Insights component and that the linked action group uses only the default role receivers. It does not adopt resources by name or modify/delete a shared action group in another resource group. If those links are not yet observable during provisioning, inspect the same `bootstrap status` again; do not edit the manifest or delete the alert to bypass the check.

Set TPM before starting model calls

The [step-02 recommendations](#) require **100,000 TPM** per Agent, Judge, and shared Optimizer/planner deployment, and **10,000 TPM** for embeddings. This assumes sequential jobs in one environment; quota pools remain model/SKU/region-specific. Sum per-model allocations across simultaneous dedicated environments. Count the Optimizer/planner deployment shared within one environment only once.

- 1. New dedicated environment:** Check generative capacity 100 and embedding capacity 10 in the updated `bootstrap plan`, then authorize and provision that exact plan. Existing plans do not change automatically. For an unprovisioned lower-capacity plan, prepare a new environment name and authorization rather than editing the hashed plan.
- 2. Inspect actual deployments:** Read **Tokens per Minute Rate Limit** under Foundry **Build → Models → Deployments → deployment name → Details**. Distinguish capacity displayed in thousands of tokens from the final TPM value. **Changing only the portal's Edit setting for this bootstrap environment causes plan drift.** If insufficient, inspect the plan, actual limit, and authorization; prepare a newly authorized dedicated environment with sufficient capacity when needed. Never edit hashes or approvals to bypass checks.
- 3. Verify the setting:** With that environment's `.env` and language-specific artifacts path, run the read-only check below. Inspect each `*_tpm` entry's `observed`, `expected`, and `reason`.

Run in your terminal · Shared terminal

```
python -m lab --config .lab/lab-en/.env preflight
```

Continue to step-03 smoke/retrieval and later evaluation/Optimizer only after overall status: PASS and all five TPM checks pass. The checker reads the actual deployment's `rateLimits` entry with key: `token`. If the CLI omits labels, it reads raw ARM metadata for that same deployment instead of guessing TPM from request counts or `sku.capacity`. Unverifiable limits are BLOCKED.

TPM accounting uses estimates rather than average billed tokens; **RPM and short-window burst limits** also apply. Allow more headroom for longer inputs/maximum outputs, shared users, and overlapping evaluation/optimization. These recommended minimums do not guarantee a 429-free run. See [official rate-limit guidance](https://learn.microsoft.com/azure/foundry/openai/how-to/quota#understanding-rate-limits) (https://learn.microsoft.com/azure/foundry/openai/how-to/quota#understanding-rate-limits).

Use the [real response check in 03](#) to verify Agent and knowledge_base_retrieve calls. Optimizer has a separate [supported-model list](https://learn.microsoft.com/azure/foundry/agents/concepts/agent-optimizer-overview#models) (https://learn.microsoft.com/azure/foundry/agents/concepts/agent-optimizer-overview#models); Agent/Judge availability does not establish generator support.

Microsoft Foundry / proj-fev11-dea3cec5

Search (🔍)

Create

Agents

Models

Services

Tools

Knowledge

Guardrails

Data

Optimize

Evaluations

Fine-tune

Models

Deployments Batch jobs

Serverless deployments Managed compute deployments

Search

PTU Calculator

Deploy

Name	Model	Version	Deployment status	Deployment type	Created on ↓	Created by
☒ lab-judge-luna-dea3cec5	gpt-6-luna	2026-09-22	Succeeded	Global Standard	9/30/26, 11:19:02 ...	
☐ lab-agent-luna-dea3cec5	gpt-6-luna	2026-09-22	Succeeded	Global Standard	9/30/26, 11:18:59 ...	
☐ lab-embedding-dea3cec5	text-embedding-3...	1	Succeeded	Global Standard	9/30/26, 9:58:45 PM	
☐ lab-planner-dea3cec5	gpt-5.5	2026-04-24	Succeeded	Global Standard	9/30/26, 9:58:44 PM	
☐ lab-judge-dea3cec5	gpt-5.4-mini	2026-03-17	Succeeded	Global Standard	9/30/26, 9:58:42 PM	
☐ lab-agent-dea3cec5	gpt-4.1-mini	2025-04-14	Succeeded	Standard	9/30/26, 9:58:40 PM	

1-6 of 6

< Prev Next >

Check model, version, and status. Map each deployment to the Agent, Judge, Optimizer, and embedding roles. Deployment names must match `.env`; verify actual calls to the prepared deployments as well. View full-size image (file: `web/assets/portal/en/02-model-deployments.png`)

Prepare an executable Agent

Prepare **Python 3.11–3.14, Git, and Azure CLI** using the [OS-specific installation instructions in 01](#), then [verify all three versions in a new terminal](#). Do not reinstall working tools; resolve installation restrictions, permissions, and proxy issues first. Then create the virtual environment with the verified Python, sign in, and set the language variables. Do not activate a nonexistent `.venv` or treat fake IDs in `.env.example` as a live environment.

Use `prompts/en/baseline.txt` for English and `prompts/baseline.txt` for Korean. Never weaken the baseline to manufacture improvement. After [policy upload](#) and [retrieval verification](#) succeed, create a policy-connected v1:

Run in your terminal · Shared terminal

```
python -m lab --config .lab/lab-en/.env native-agent --version 1 --confirm
```

`native-agent` uses `ensure_fixed_release` to allow only **1 and 2**, comparing local workspace ownership with remote metadata. It reuses matching configurations, rejects a different v2, and never creates v3. It also retains a model-deployment snapshot and rejects drift. Receipts `agents/native-v1.json`, `agents/native-v2.json`, and `agents/native-model.json` belong to that environment's artifacts folder.

`native_response_format()` projects the common schema into the supported strict-generation subset. Citation uniqueness and `needs_human/route` consistency still require post-generation checks. Valid JSON does not establish policy correctness.

Creating v2 is not completing evaluation. Create one v2 from the [complete instructions reviewed in 08](#), run [the same managed evaluation in 09](#), then decide whether to accept it. Use only released v1/v2; separate `draft-...` experiments are outside this lab. Do not publish the created v2 to production.

Run in your terminal · Shared terminal

```
python -m lab --config .lab/lab-en/.env native-agent --version 2 --prompt .lab/lab-en/candidate.txt --confirm
```

Do not use portal Promote in this lab; create v2 with 09's CLI. Promoting first can conflict with the CLI's local/remote ownership checks. V1/v2 are complete Agent versions; the same Agent, model, tools, and output settings must differ only in instructions.

Retrieve IDs with [06's complete lookup command](#), including the `.env` path and exact evaluation name. The reevaluation helper copies the baseline data source and checks the **remote** contract. Do not replace the remote definition with a stale local `JUDGE_DEPLOYMENT`.

Freeze the comparison conditions

Item	Fixed requirement
Data	Same selected-language dev12 bytes, registration/version, questions and references
Relevance	Managed <code>builtin.relevance</code> , scale 1–5, threshold 4
TaskAdherence	Managed <code>builtin.task_adherence</code> , binary 0/1, pass 1
Judge	Same explicit Luna deployment in both evaluator definitions
Agent	Same Sol model/version, tools, reasoning and strict output schema
Change	Instructions only; releases remain v1/v2
Optimizer	Instruction target only, model/tool-description changes off, bounded candidate count
Lab records	Keep same-condition v1/candidate results for all twelve cases in private run folders, separate from reusable guides

Catalog evaluator versions are not proof that the private service rubric is fully pinned. Preserve the actual definition, settings and this limitation. A passing gate on reused dev12 does not guarantee future scores, independent generalization or production approval.

The required comparison uses [Foundry Compare runs in 09](#). `scripts/compare_foundry_eval.py` is an optional standalone tool, not another command required in this path. When using it separately, `LAB_LANGUAGE` or `--language ko/--language en` selects report language and default dev12; match any explicit dataset to that language.

Check evaluation response mappings

Select the [two evaluators and Judge in 05](#) and preserve automatic mappings. The following table supports missing-field or mismatch checks; a normal wizard flow does not require guessing SDK fields and entering them manually.

Setting	Required value
Agent user input	Only <code>{{item.query}}</code> ; leave the instruction override unset
Relevance response	<code>response={{sample.output_text}}</code>
TaskAdherence response	<code>response={{sample.output_items}}</code>

Setting	Required value
Judge/thresholds	Actual JUDGE_DEPLOYMENT; Relevance 4, TaskAdherence binary pass 1

If a required field is **Unassigned**, confirm the target is **Agent**, not Dataset, and inspect the query column and actual generated response mapping. Before submission, inspect the wizard; afterward, inspect the saved remote definition. Never map the reference answer as the response or copy old UI bindings. Use the [official portal guide](https://learn.microsoft.com/azure/foundry/how-to/evaluate-generative-ai-app) (<https://learn.microsoft.com/azure/foundry/how-to/evaluate-generative-ai-app>) and [response-mapping explanation](https://learn.microsoft.com/azure/foundry/concepts/evaluation-evaluators/agent-evaluators#using-agent-evaluators) (<https://learn.microsoft.com/azure/foundry/concepts/evaluation-evaluators/agent-evaluators#using-agent-evaluators>). Do not submit until resolved. The helper in 09 checks the remote contract too.

Complete the actual authorization file

Open `.lab/lab-en/approval.example.json` and **Save As approval.json in the same folder**. Set limits yourself only if you can authorize spending and changes; otherwise follow the organizational approval procedure. Keep actual authorization evidence private. Neither this worksheet nor the JSON file independently proves approval or a signature. Use double quotes for JSON strings, but not for numbers, true, false, or null.

Field	How to complete it
schema_version, scope_sha256, models, retention_days	Preserve generated values. Editing the model array or plan invalidates existing authorization.
approved	Set <code>true</code> only after actual approval.
approved_by	Your sign-in name matching <code>expected_user</code> in <code>config.json</code> . This binds the execution identity, not an organizational approver's digital signature.
approved_at, expires_at	Actual authorization start/end in timezone-aware ISO 8601, such as <code>2026-10-03T09:00:00+09:00</code> . Do not reuse the example date. The current time must fall within the interval.
currency, budget_amount, budget_policy	Approved three-letter currency and positive numeric budget. Keep policy BOUNDED and enter the actually authorized amount.
acknowledge_no_monetary_cap	Keep <code>false</code> . Unlimited spending is not the default lab policy.
max_hosting_hours	Approved positive integer hours. If 8, the approval interval must also be no longer than 8 hours.
max_wait_seconds	Positive integer wait limit, for example 3600. Ending the wait does not cancel a remote job.

Field	How to complete it
max_calls, max_candidates	Approved call/candidate limits; 08 requests 2 candidates. Account for internal Agent, Judge, and search calls. These fields do not automatically cap every portal call or charge.
allow_global_inference	true only when GlobalStandard processing is authorized; it is not NCUS-only processing.
allow_resource_creation, allow_rbac_assignments	Each is true only when this plan's resources and resource-scoped role assignments are authorized.
acknowledge_continuous_hosting, acknowledge_unknown_cost	Each is true only after acknowledging continuing hosting charges and an unconfirmed final cost.
allow_training, allow_global_training	false for this path.
max_epochs, max_training_jobs	0 for this path.
accept_deprecated_models	Default false; do not change it without reviewing a service deprecation notice.

Validate with `02's bootstrap preflight --config ... --approval ...` and read `approval_reason` on failure. Authorization covers only the exact plan hash, models, and retention scope. It does not authorize another lab, an automatic retry, or deletion.

Enforcement boundary: Bootstrap checks authorization validity, provisioning scope, and its wait bound. JSON budget and call limits do not automatically cap all Azure Portal or Optimizer spending. Budget alerts are not cutoffs. Monitor actual usage yourself and cancel jobs or clean up within the authorized scope.

Verify permissions and provider registration

Follow the [current Foundry RBAC guidance \(https://learn.microsoft.com/azure/foundry/concepts/rbac-foundry\)](https://learn.microsoft.com/azure/foundry/concepts/rbac-foundry). **Foundry User/Owner/Account Owner/Project Manager** may still appear under their former **Azure AI User/Owner/Account Owner/Project Manager** names.

Use **Access control (IAM) → Check access** to inspect active assignments and Scope. Older UI versions may use View my access. The [actual subscription/access illustration in 01](#) demonstrates inspection of existing access, not adding roles.

Task or connection	Required permissions and scope
Your environment creation	Bootstrap checks effective subscription-scoped group/deployment/resource writes and <code>Microsoft.Authorization/roleAssignments/write</code> . Contributor alone cannot assign roles. Subscription quota-read access is also required.
Your Agent/evaluation operations	Foundry User or the required data-plane permissions on your project. Azure Contributor/Owner alone does not grant these data actions.
Your policy preparation	Search schema/upload permissions, such as Search Service Contributor and Search Index Data Contributor, on your Search service.
Project managed identity	Search Index Data Reader on this Search service, model invocation, and telemetry submission to the assigned monitoring resource.
Search managed identity	Model invocation on the assigned Foundry resource.
Your log inspection	Read access to your Application Insights/Log Analytics resources, not subscription-wide log access by default.

Authorized bootstrap assigns runtime roles at the new resources' scopes. It does not add subscription-wide roles. After creation, distinguish your user identity from service managed identities and use 03's actual tool call to verify propagation and data-plane access.

A provider registration enables a service type in your subscription. Follow the [official provider-registration instructions](https://learn.microsoft.com/azure/azure-resource-manager/management/resource-providers-and-types#azure-portal) (<https://learn.microsoft.com/azure/azure-resource-manager/management/resource-providers-and-types#azure-portal>):

1. Open Azure Portal → **Subscriptions** → **your subscription** → **Resource providers**.
2. Search for `Microsoft.CognitiveServices`, `Microsoft.Search`, `Microsoft.OperationalInsights`, and `Microsoft.Insights`; confirm **Registered** for each.
3. Select a missing provider and **Register** only with your own `/register/action` permission and authorization. Do not register unrelated providers in bulk.
4. After registration/propagation, rerun the same preflight. If access is missing, obtain an approved execution scope before resuming. Bootstrap never auto-registers providers or expands permissions.

Allow registration/role propagation, then rerun the same preflight. For continuing 403, compare identity, role scope, and network access. Do not grant new subscription-wide Owner, disable network or organizational policies, or transfer tokens as a shortcut.

Costs and operational boundaries

Evaluation invokes the Agent and Judge; optimization makes additional internal calls. Search and monitoring can continue to incur charges after the lab page closes. Distinguish measured tokens, cost estimates, and actual billing; unknown billing is not zero.

In Azure Portal → **Cost Management** → **Cost analysis**, select the subscription, resource group, and date range. Configure authorized budget alerts where needed, but do not treat them as enforced caps. Record the actual budget, end time, and deletion/retention plan in the worksheet.

Record actual values and completion evidence

After creating the plan in 02, copy this table into your `.lab/lab-en/notes.md`. Mark unexecuted items **not run**, then fill actual values as each step completes. “The environment is ready” is not sufficient evidence.

Required item	Actual values or evidence
Identity	User, tenant ID, and subscription ID; never passwords or tokens
Local paths	Your environment name, bootstrap config, runtime <code>.env</code> , <code>LAB_LANGUAGE</code> , and <code>LAB_ARTIFACTS_DIR</code> ; record <code>.env</code> only after successful creation
Azure environment	Resource group, Foundry account/project, project endpoint, and Search service
Models	Actual names, product/version, SKU, and capacity for all four deployments
Policy search	Source/language, eight uploaded documents, knowledge-base/connection names, and <code>retrieval_verified</code>
Agent	Actual name, pinned v1, strict output, and a successful real Agent tool call
Dataset	JSONL path, 12 rows, SHA-256, registration name, and version
Evaluation	Relevance 4, TaskAdherence 1, Judge deployment, and query-only input; add actual evaluation/v1-run IDs after the first run
Authorization	Scope hash, expiry, budget, candidate limit, and retention
Finish	Exact authorized deletion group/objects and planned time, or retention reason, cost responsibility, review date, and subsequent deletion plan; distinguish shared/external dependencies

Keep English and Korean source data and private lab records separate; do not relabel another language's results as a new execution.

Raw service files can contain account metadata, tokens or signed URLs. Keep those local; **the synthetic evaluation results themselves are not secret**. Publish only allowlisted fields, including failures, and exclude credentials—not inconvenient outcomes. Historical attempts stay in the private audit, not as additional current-version reports.

Verify deletion and retention

Use [step 10](#) as the exit checklist. Provisioning approval is not deletion approval, and workshop documentation does not authorize deleting another person's resources.

Target	Deletion or retention check
Active evaluation/Optimizer jobs	Cancellation availability, actual terminal state, and unresolved job IDs
Locally owned objects	Exact cleanup plan and verified absence afterward
Portal-created objects	Individually inspect datasets, evaluations, conversations, Optimizer records, and empty Agent entries
Dedicated resource group	Default path: verify inventory and authorization, delete, and record a successful <code>az group exists</code> result of <code>false</code>
Retained/shared/external resources	Remaining item, reason, cost responsibility, retention review date, and follow-up plan
Billing/soft delete	Delayed charges, prior usage, and separate service retention/purge policy
Evidence	Keep outcomes and failures privately for the required period; do not delete <code>.lab</code> first

Lab troubleshooting

[Ten lab steps](#) · [Access, authorization, and setup reference](#)

Start with the blocked step and check the symptom, action and completion criteria. Do not silently change data, evaluators or models, or submit the same operation under a new name to bypass an error.

Executable commands appear in **Run in your terminal** blocks. Use **Where to act** links beside **code ↔ portal** tables to find the command or portal procedure. Collapsed implementation panels are read-only source, not another program to execute or edit to hide an error.

Symptom	Where to look
HTML shows an explanation but the execution command is hard to find	Use the table's Where to act links or the step's Action order . Sign-in commands · Runtime check .
Terms, installation, sign-in or file paths are unclear	Basic terms · Environment and sign-in
Authorization, permissions or model capacity blocks provisioning	Provisioning
Policy retrieval or Agent responses fail	Policies and Agent
Evaluation is unfinished or scores/reasons are missing	Evaluation and results
Optimize is unavailable or candidate selection is unclear	Optimizer
Deletion fails or resources must be retained	Cleanup and retention

How to use this guide

Keep timestamps, steps, commands, target IDs, error codes and actions in your private lab record. Preserve original responses and receipts rather than editing failures into successes. Do not append run-specific validation logs or results to reusable guides.

Environment, sign-in and local files

Resolve Python, Git, and Azure CLI installation and PATH issues before signing in to Azure.

Symptom	Action	Completion criteria
py, python3.13, git, or az reports command not found or not recognized	Complete OS-specific installation in 01 , close all terminals, and reopen them; restart VS Code too. If it still fails, check the approved installation paths and PATH. Inspect paths with <code>Get-Command py, git, az</code> on Windows or <code>command -v git az</code> on macOS/Linux.	All three version checks succeed in the same new terminal.
winget or brew is missing	On Windows, use the official installer path . On macOS, complete Homebrew installation and the PATH instructions under Next steps .	Prepare and verify the tools using one installation method without bypassing organizational restrictions.
Microsoft Store opens instead of Python, or py is missing	Check Launcher, pip, and PATH options in the regular Windows Python installer. Run <code>py -3.13 --version</code> in a new PowerShell window, substituting your installed version if different.	An installed Python 3.11–3.14 runs, not a Store shortcut.
Python is outside 3.11–3.14, or .venv uses a different Python	Select a supported version separately rather than replacing system Python. Create the lab environment with the same verified command , such as <code>py -3.13</code> or <code>python3.13</code> . Check ownership and retention needs before replacing an existing .venv.	The environment's <code>python --version</code> is supported and <code>python -m pip --version</code> points inside that .venv.
Linux reports No module named venv or ensurepip is not available	Install <code>python3-venv</code> from the Ubuntu setup instructions . For a version-specific Python, confirm its matching venv package and an approved installation path.	The selected Python creates .venv and its pip check succeeds.
Installation permissions, proxy, or certificate errors block installation	Check the official download URL, error, and permitted installation path. Obtain required installation/network authorization without disabling TLS checks, security tools, or organizational policies.	Installation uses an approved path and all three version checks succeed.
.venv/bin/activate or Activate.ps1 is missing	Create the virtual environment in the repository and follow the OS-specific setup order .	<code>python -m pip --version</code> and <code>python -m lab --help</code> work in the same environment.
ModuleNotFoundError	Confirm the intended environment and run <code>python -m pip install -r requirements.lock</code> with its Python.	Required imports and dataset checks work in that environment.
PowerShell activation is blocked by policy	Use <code>.\.venv\Scripts\python.exe</code> without changing organizational policy.	Python runs without a policy change.

Symptom	Action	Completion criteria
Portal and CLI accounts differ	These are separate sign-ins. Compare user, tenant and subscription with the portal using 01's sign-in/query commands ; complete personal authentication yourself.	All three approved values match. Do not substitute another identity or copy tokens to bypass authentication.
AzureCliCredential fails with both tenant and subscription	Use the repository authentication path: verify the tenant first, then specify the subscription on the credential.	Calls use the intended subscription without dropping tenant checks.
Artifacts go to the other language's folder	Set LAB_LANGUAGE and LAB_ARTIFACTS_DIR before starting Python with 01's language/record-folder commands .	Dataset, configuration, workspace and results share the same language; existing records are not overwritten.

Provisioning, authorization and quota

Symptom	Action	Completion criteria
Plan reports BLOCKED_AWAITING_APPROVAL	Inspect plan_status and mutations_performed, then prepare the actual authorization .	Distinguish local planning from Azure provisioning.
Approval fails despite approved: true	Check hashes, models, approver, validity, budget, acknowledgments and approval_reason.	Readiness is READY and authorization is READY_FOR_APPROVED_APPLY.
Provider is not Registered	Follow provider registration , verify your registration permission/authorization, and register only the required providers.	Rerun the same preflight after all four providers are Registered.
Contributor cannot assign roles	Resource creation and role assignment are separate permissions. Obtain authorization for an execution scope in which you can perform this lab yourself.	Verify effective permissions and resume the same plan. Do not add subscription-wide Owner to bypass the issue.
Model, SKU, version, quota or capacity error	Inspect the planned role requirements and preflight reason. A replacement needs a separate plan and authorization.	Confirm support and capacity, then verify actual calls.
Runtime *_tpm is BLOCKED	Check the real allocation using the TPM setup instructions .	All five TPM checks and overall preflight report PASS.
unknown resource after creation	Check whether this is verified default Smart Detection linked to your Application Insights, another workload, or changed receivers. Allow link propagation during creation.	Inspect the same bootstrap status; do not delete alerts or edit the manifest to bypass it.

Symptom	Action	Completion criteria
Apply times out or its outcome is unknown	Preserve the original config, manifest and deployment ID; read 02's bootstrap status command and the group's Deployments. Do not repeat apply.	Wait for active work; do not resubmit an unknown outcome.
The local environment already exists	Resume the same lab with its original configuration. A different lab needs a new environment name and authorization.	Do not adopt or overwrite existing plans, settings or resources.

Use `--retry` only for a verified terminal owned failure and a separate retry allowance. `repair-dependencies` and `repair-trace-routing` also require matching failure evidence and scoped authorization. `APPLIED` describes infrastructure, not answer quality, production approval or confirmed trace ingestion.

Policy retrieval and Agent setup

Symptom	Action and completion criteria
Deployment exists but model calls fail	Catalog/deployment readiness and runtime support differ. Inspect the API error and intended role, then verify the actual response in 03 .
Setup remains <code>created_not_retrieval_tested</code>	Only creation is complete. Use 03's iq_probe command to confirm <code>retrieval_verified</code> and actual references.
Direct retrieval succeeds but the Agent tool returns 403	Check Search-read/model-call permissions and connection audience for the project's managed identity. User retrieval alone is insufficient.
<code>native-agent</code> reports a configuration mismatch	Inspect the receipt and remote version first. Equivalent MCP allowlist representations are normalized, but different tools, permissions, models or output settings remain blocked. Do not delete the receipt or create another version to bypass it.
Existing v2 has different instructions or model	Fixed-comparison protection is working. Compare the original owned environment, instructions and model snapshot; do not overwrite v2 or create v3.
Retrieval differs between languages	Check the selected language's source documents, index, analyzer and hashes. Renaming another language's index is not reuse.
Private-network or <code>PublicNetworkAccess=Disabled</code> failure	Use an approved VNet/VPN/execution environment and check DNS. Do not disable the firewall or private endpoint.

Foundry evaluation and safe resumption

Add run item-schema error

If the following message appears, inspect the existing evaluation's Agent-target data source:

Example error message · do not run it.

```
Unable to create data source configuration from item schema
```

The `scripts/add_foundry_eval_run.py` [helper in 09](#) copies a completed baseline's remote data source and changes only the explicit candidate version. Confirm the same evaluation ID, registered dataset, Judge, thresholds and mappings, plus the actual version/instructions for all twelve items. Do not substitute new data or a local Judge.

IDs and duplicate submissions

Symptom	Action
Evaluation ID or run ID is unknown	Use 06's complete native-evals command . Evaluation uses <code>eval_...</code> ; the baseline is a completed version-one <code>evalrun_...</code> .
Several evaluations have the same name	Compare creation time, Agent, dataset and runs; do not select by name alone.
Helper exits with Still running	If the receipt has a run ID, resume collection with the identical command and <code>--out</code> .
Receipt has no run ID, or a remote run with the same name exists	Preserve the original and check submission acceptance and the actual run. Do not delete receipts, rename or automatically resubmit.
Changing the version clears its checkbox	Reselect explicit v1 and confirm one checked target.
Dataset preview has five rows	Compare the twelve-row source, registered version and complete result count; the preview alone is insufficient.
Scores or reasons are not visible	Read the response under <code>conversation_id</code> → <code>User view</code> ; reasons are in Detailed metrics result under <code>Relevance.reason</code> and <code>TaskAdherence.reason</code> .
A required response field is Unassigned	Use mapping checks to compare the Agent target, query column, and service-generated response mapping. Do not add reference answers as a JSONL response column or submit before resolving the issue.
Output items or scores are missing	Preserve failures/missing items and keep the denominator at twelve. Do not invent zero scores or passes.

Agent Optimizer and interpretation

Symptom	Action and completion criteria
No custom evaluators available	Use Custom only OFF or View built-in evaluators rather than creating a custom evaluator to bypass a filter.
Optimize or the selected model is unavailable	Check New Foundry, prompt Agent, project permissions, Preview availability and role-specific model support (https://learn.microsoft.com/azure/foundry/agents/concepts/agent-optimizer-overview#models).
The authorized wait ends while the job is active	Inspect the same job ID's state, errors, and usage in Optimization runs; record follow-up checks. Stopping your wait is not cancellation. Do not submit a replacement job.
The job is Failed/Canceled or results are partial	Preserve actual state, job ID, and errors. Do not claim a completed candidate or repeat the request under another name.
View changes exposes only a partial diff, not complete instructions	Expand revised sections and inspect candidate details or Download/Export where provided. Without the complete text, do not create v2; record "retain v1; complete instructions unavailable" and continue to 10.
A candidate changes model or tool descriptions	Select Instruction only and disable model comparison. Non-instruction changes are not a same-condition comparison.
Candidate embeds policy dates, numbers or evaluation answers	Preserve the original service output and remove embedded answers or reject the candidate. Record manual changes, reasons and provenance, then reevaluate separately.
Only the baseline is selected, with no different candidate	Record why v1 is retained and continue to 10. Do not invent v2 or repeat the job to manufacture improvement.
Internal ranking differs from reevaluation	Internal 0–1 ranking and separate evaluation scores/pass rates differ. Inspect actual answers, policy errors and regression too.
TaskAdherence is 1	That is a binary pass, not a one-out-of-five Relevance score.
Statistics are Inconclusive	A difference was not established; this is not evidence of significance or equivalence.
Quality improves but latency/tokens increase	Record the tradeoff too. Unknown billing is not zero cost.

Cleanup, retention and continuing costs

Symptom	Action and completion criteria
Retention is explicitly authorized	Do not run deletion commands. Record the actual inventory, reason, cost responsibility, review date, and subsequent deletion plan; keep ownership/configuration needed to reopen the environment.
Search charges remain after cleanup	<code>cleanup</code> handles recorded objects, not all hosting/models/groups/logs. Follow authorized cleanup or retention in 10 .
<code>.env</code> or workspace is missing	Compare the configuration, manifest and Azure inventory first. Do not fabricate ownership records to delete resources.
Monitoring in the group serves another group	Review shared action-group dependencies with the owner. A dedicated group does not authorize deleting shared resources.
Locks, permissions or dependencies block deletion	Read Locks, Activity log, and the failed operation. Obtain authorization for the required action before resuming; do not remove organizational locks without authorization or mark blocked deletion complete.
The group remains after a deletion request	Inspect asynchronous progress, then run 10's absence check . Only a successful <code>az group exists</code> result of <code>false</code> verifies absence.
Costs appear after deletion	Check the usage period, billing delay and remaining resources in other groups. Earlier usage charges do not disappear.

Service-specific soft-delete retention and permanent deletion/purge require separate policy and authorization. See the [resource-group deletion documentation \(https://learn.microsoft.com/azure/azure-resource-manager/management/delete-resource-group\)](https://learn.microsoft.com/azure/azure-resource-manager/management/delete-resource-group).

Synthetic English evaluation data

[Dataset step](#) · [Lab worksheet](#) · Korean data guide (file: docs/ko/data-guide.html)

One unchanged dataset for the comparison

Use **data/en/optimizer/dev.jsonl** (file: data/en/optimizer/dev.jsonl), exactly **12 JSONL records**, for Foundry Evaluation, Agent Optimizer, and reevaluation. In the project you created, follow [04](#) to register lab-en-dev12 version 1. When resuming your same lab, compare its recorded name, version, and hash before reusing registration.

Where to act: [Terminal · check all 12 rows/hash](#) → [Foundry · upload/select](#). The local check uploads nothing and submits no evaluation.

Keep its exact bytes, SHA-256, questions, references, and language fixed throughout the comparison. Keep Korean and English data separate rather than mixing them in one comparison.

Use [04's code ↔ portal table](#) to distinguish local count/hash checks from portal registration. [05's remote-settings verification](#) and [09's actual SDK submission](#) show how data, mappings, and versions stay fixed. Do not run read-only source panels independently.

Purpose and limits

Contoso Atlas Cloud is fictional. Its policies, inquiries and reference answers are synthetic—not private customer data or the terms of Microsoft, Azure or a real provider.

The purpose is **evaluation against your own business tasks and criteria**, not a public benchmark alone. Representative tasks expose policy-date boundaries, ambiguity, unsupported certainty and action claims. A 12-case development set demonstrates the learning loop; it does not certify production or independent generalization.

Preserve the original source corpus

data/en/cases.jsonl (file: data/en/cases.jsonl) contains 100 cases: train 56, validation 12, dev 12 and test 20. Only dev12 is used in this workshop. Other source subsets remain available but are not additional participant exercises.

Do not select only easy rows, duplicate questions, merge subsets, or replace references after seeing results. The eight policy documents (file: data/en/knowledge/documents.json) retain stable ATLAS-* IDs and effective dates. Prepare policy access yourself with the [commands in 03](#).

Upload and response schema

Each JSONL line has exactly three columns:

Column	Type	Purpose
query	String	The only Agent input
context	String	Source-policy reference for supported evaluators and case review
ground_truth	JSON string	Structured reference answer, not a generation prompt

There is no prefilled `response`. Foundry invokes the pinned Agent to obtain one. Do not turn the file into a JSON array, rename columns, append references to the user message or precompute answers.

The reference answer and Agent response use exactly four keys:

Key	Contract
answer	Nonempty English string
citations	Unique stable IDs of supporting policy documents
route	<code>answer</code> , <code>clarify</code> , <code>escalate</code> or <code>refuse</code>
needs_human	Boolean; true exactly when route is <code>escalate</code>

The source response schema (file: schemas/response.schema.json) also checks routing consistency. Native structured output controls the JSON shape, not the factual truth of an answer. Numeric retrieval IDs are not policy citations.

The Agent cannot actually submit tickets, change subscriptions, approve credits or delete records. Classification as `escalate` does not mean a person was contacted. A request for unsupported certainty does not authorize fabricated facts.

Register once; pin the same version

Use **Foundry → Build → Evaluations → Create → Agent**, an explicit baseline version, and **Individual turns / One time**. Reselect the target checkbox if version selection clears it.

On the first attempt, use **Upload new dataset → Browse** to register the exact English file as `lab-en-dev12` version 1. For resumption, Optimizer, and reevaluation, use **Existing dataset** with the same name/version. The preview may show five rows; the file and evaluation scope still contain 12.

Leave the custom prompt override unset. Use `query → query` if a field-mapping screen appears. Service mappings are **Relevance** `response={{sample.output_text}}` and **TaskAdherence** `response={{sample.output_items}}`. These are not extra JSONL columns.

Keep **Relevance 1–5 / threshold 4**, **TaskAdherence binary 0/1 / pass 1**, and the same Luna Judge. Selecting two evaluators does not mean both consume every reference field or certify every business rule.

Compare complete evidence

Compare the questions, responses, evaluator scores, and reasons for all 12 cases across same-model v1 and v2. Record your actual run IDs and dataset hash to identify the comparison.

The helper checks matching questions/references and actual per-item Agent version/instructions. Missing/error rows cannot become successful rows or disappear from the denominator. JSON/route checks are supplementary validation, **not a replacement local Judge**.

Agent Optimizer's internal ranking is distinct from the separate managed run's means and pass counts. Review the complete candidate instructions and preserve model/tools/reasoning/schema. Compare one reviewed v2 without continually adding releases. Without a candidate worth retaining, record why you keep v1 and continue to 10.

Before sharing results, remove credentials, signed URLs, and private account details. Keep every case, including errors and missing results.

`cleanup` does not remove every registered dataset. Follow [10](#) to preserve results and remove your dedicated group. Only with explicit retention authorization, record remaining registrations/evaluations, cost responsibility, and review date. Never delete shared or external resources.